

计算机组成原理与系统结构

第4章 运算方法 与运算器





- ❖ 4.1 计算机中的运算操作
- ❖ 4.2 定点数的加减运算
- ❖ 4.3 定点数的乘法运算
- ❖ 4.4 定点数除法运算
- ❖ 4.5 定点运算器的组成与结构
- ❖ 4.6 浮点运算及运算器
- ❖ 4.7 现代计算机的运算器
- ❖ 本章小结



4.1 计算机中的运算操作

- ❖ **运算器的硬件功能：**取决于指令系统的运算需求
- ❖ **指令系统：**体现高级语言编写的软件的需求
- ❖ **高级语言中的运算：**通过底层的运算类机器指令，最终在硬件上实现的

❖ 4.1.1 从高级语言到机器语言的运算

❖ 4.1.2 移位运算





4.1.1 从高级语言到机器语言的运算

运算类型	C语言运算操作符	RISC-V机器指令/汇编指令	硬件运算
算术运算	+ (加) - (减) * (乘) / (除)	add/addi sub/subi mul/mulh/mulhu/mulhsu div/divu	加法 减法 乘法 除法
按位运算	& (位与) (位或) ~ (取反) ^ (位异或)	and/andi or/ori xori(0xFFFF FFFF) xor/xori	位与 位或 取反/位异或 位异或
移位运算	<< (左移) (unsigned)>> (右移) (signed)>> (右移)	sll/slli srl/srli sra/srai	左移 逻辑右移 算术右移
逻辑运算	! (非) && (与) (或)	sltiui(1) 基于beq(0)的程序段 基于beq(0)/bne(0)程序段	比较小于置数 相等比较 不相等比较

❖ **i**: 参加运算的操作数有一个是常数

❖ **u**: 无符号运算

4.1.1 从高级语言到机器语言的运算

- ❖ (1) 算术运算：最基本的加、减、乘、除四则运算
 - 操作数：有符号数/无符号数的**整数**算术运算、**浮点数**的运算
- ❖ (2) 按位运算：按位进行逻辑运算——与、或、非/取反、异或
 - C语言：**取反运算符** “~”
 - 机器指令：x86 → **NOT**指令；RISC-V指令集用异或指令xori代替实现：xori x6,x6, -1
 - 硬件：与门、或门、非门、异或门
- ❖ (3) 移位运算：左移——数值**乘以2**；右移——数值**除以2**
 - **逻辑移位**：操作数是**无符号数**——**最高位补零**
 - **算术移位**：操作数是**有符号数**——**最高位保持不变**
 - C语言：左移操作符 “<<”；右移操作符 “>>”；几乎所有的编译器和计算机系统都遵循如下规则：对有符号数执行算术右移；对无符号数执行逻辑右移
 - 机器指令：**左移**——sll/slli；**右移**——sra/srai（算术）、srl/srli（逻辑）
 - Java语言则明确：“>>”操作符指定算术右移，“>>>”操作符指定逻辑右移

4.1.1 从高级语言到机器语言的运算

- ❖ (4) 逻辑运算：将操作数当作命题进行逻辑“真”与“假”的判定，逻辑运算后返回true (1) 或者false (0)
 - C语言：操作数的值非零即真，零即假
 - &&和||运算没有对应的机器指令，编译器一般都会将其翻译成一段机器语言程序完成

C语言程序	RISC-V机器语言程序
int x=0;	li t0,0 #变量x在寄存器t0中, t0=0
int y=-1;	li t1,-1 #变量y在寄存器t1中, t1=-1
int z1=!x;	sltiu s1,t0,0x001 #if (t0<0x001) s1=1 else s1=0, 变量z1存放在s1寄存器
int z2=x&& y;	beq t0,x0,L1 #if (t0==0) goto L1, x0是全0寄存器, 判断x是否为0
	beq t1,x0,L1 #if (t1==0) goto L1, 判断y是否为0
	li s2,1 #t0和t1 (x和y) 都非0, 则&&结果=1, s2是变量z2
	j L2 #goto L2, 跳至后面程序
	L1: li s2,0 #x=0或y=0时, 跳到此处, &&的结果=0, 变量z2在s2中
	L2:





4.1.2 移位运算

❖ 移位是计算机中常用的一种运算操作：

- 二进制位每相对于小数点左移1位：乘以2，数据放大了1倍
- 每相对于小数点右移1位：除以2，数据缩小了1/2倍
- 比采用乘除法指令进行乘以或除以 2^n ，运算速度快很多
- 可以采用移位器和加减法器来构造乘除运算器

❖ 1、移位运算类型：逻辑移位、算术移位、循环移位

❖ x86 CPU：指令最全

- 算术、逻辑和循环移位指令
- 将移出的数据位置入CF标志位
- 循环移位：
 - 大循环：数据与CF一起循环
 - 小循环：数据位循环

移位类型	移位对象	左移方法	右移方法
逻辑移位	无符号数据	高位移出，低位补0	低位移出，高位补0
算术移位	有符号数据（补码）	高位移出，低位补0	低位移出，高位补符号位
	IEEE754浮点数尾数（原码）	符号位不变，数值位高位移出，低位补0	符号位不变，低位移出，数值位高位补“0”
循环移位	无符号数据	最高位移入最低位	最低位移入最高位



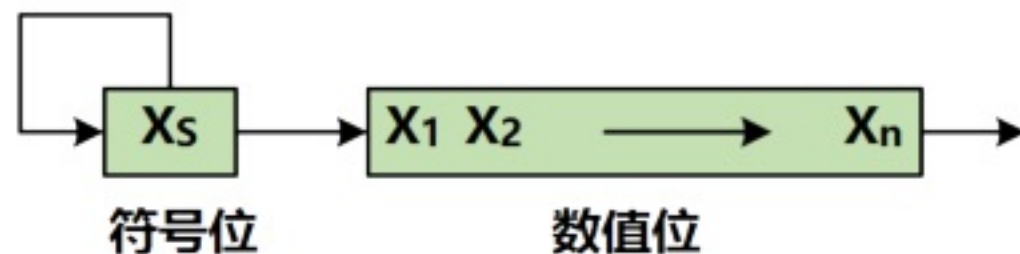
4.1.2 移位运算

- ❖ 计算机算术移位指令：大多都采用补码的移位规则，移位的对象为补码机器数
- ❖ 补码机器数的算术移位：



算术左移

移位前： $X_s \neq X_1$ ，则左移后会发生溢出



算术右移

移位后： $X_1 = X_s$ ，即：高位添符号位



4.1.2 移位运算

- ❖ 【例4.1】 $X=+13$, $Y=-5/16$, 它们是5位机器数。先写出X和Y的原码和补码, 再通过移位求出 $X/2$ 、 $2X$ 、 $Y/2$ 、 $2Y$ 的机器数。

$[X]_{\text{原}}=0,1101$	$[X/2]_{\text{原}}=0,0110$	$[2X]_{\text{原}}=0,1010$ (溢出)
$[X]_{\text{补}}=0,1101$	$[X/2]_{\text{补}}=0,0110$	$[2X]_{\text{补}}=1,1010$ (溢出)
$[Y]_{\text{原}}=1.0101$	$[Y/2]_{\text{原}}=1.0010$	$[2Y]_{\text{原}}=1.1010$
$[Y]_{\text{补}}=1.1011$	$[Y/2]_{\text{补}}=1.1101$	$[2Y]_{\text{补}}=1.0110$

- ❖ 对于补码, 硬件上直接进行左移, 所以符号位由0变成了1
- ❖ C语言不会检测 “<<” 操作符左移是否溢出, 需要程序员格外小心

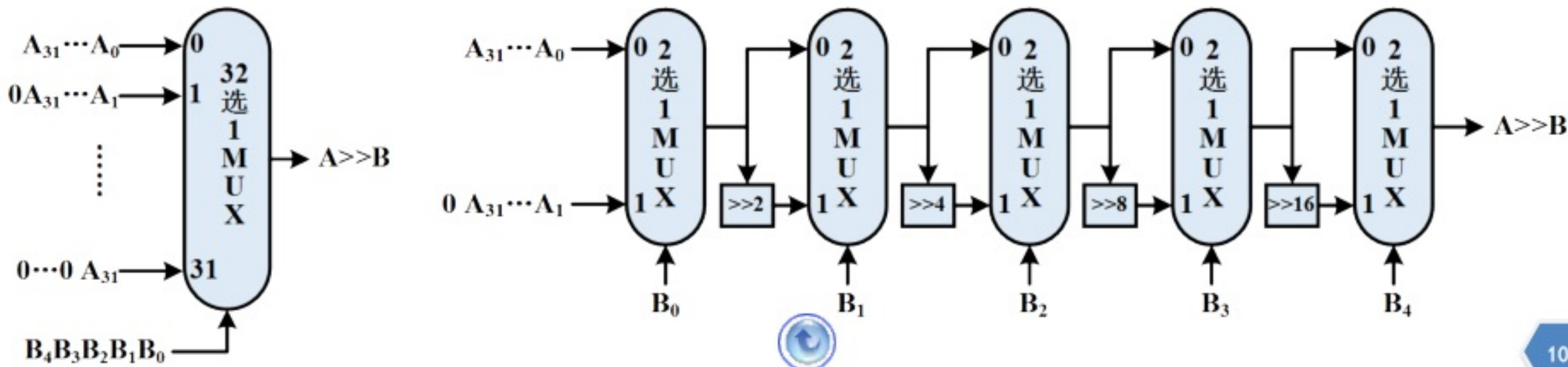
4.1.2 移位运算

❖ 2、移位器设计

- **移位寄存器**：带移位功能的寄存器，在每个时钟的有效边沿，将数据左移或者右移一位
- **多功能移位寄存器**：能选择完成左移、右移、置数等多种功能
- 它们是**时序逻辑电路**，每个时钟移位位数固定，需要多个时钟完成多位的移位，速度较慢
- 将寄存器各个触发器数据输入端的逻辑电路独立出来，就是桶形移位器

❖ **桶形移位器**：属于组合逻辑电路，能支持**可变的移位位数**和**不同移位功能**，速度较快

对32位数据 $A = A_{31} \dots A_0$ 实现 $A \gg B$ 操作





4.2 定点数的加减运算

❖ 4.2.1 补码加减运算与运算器

❖ 4.2.2 移码加减运算与判溢

❖ 4.2.3 十进制加法运算





4.2.1 补码加减运算与运算器

- ❖ 1、补码加减运算方法
- ❖ 2、补码加减运算的溢出判断
- ❖ 3、补码加减运算器的实现





1、补码加减运算方法

❖ 补码的加减运算的公式是：

- $[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}} \quad (\text{mod } 2^{n+1})$
- $[X-Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}} \quad (\text{mod } 2^{n+1})$

$[X]_{\text{原}} = 1,0011$	-3
$+ [Y]_{\text{原}} = 0,1001$	+9
1,1100	-12

❖ 公式可以用于原码、反码、移码吗？

- No No No

❖ 意义：

- 使用补码进行加减运算，符号位和数值位一样参加运算
- 补码的减法可以用加法来实现：任意两数之差的补码等于被减数的补码与减数相反数的补码之和



1、补码加减运算方法

❖ 补码的加法运算公式:

$$\blacksquare [X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$$

证明:

$$[X]_{\text{补}} = 2^{n+1} + X \pmod{2^{n+1}}$$

$$[Y]_{\text{补}} = 2^{n+1} + Y \pmod{2^{n+1}}$$

$$\begin{aligned} [X]_{\text{补}} + [Y]_{\text{补}} &= 2^{n+1} + X + \underline{2^{n+1}} + Y \pmod{2^{n+1}} \\ &= 2^{n+1} + (X + Y) \pmod{2^{n+1}} \\ &= [X+Y]_{\text{补}} \pmod{2^{n+1}} \end{aligned}$$

❖ 补码的减法运算公式:

$$\begin{aligned} \blacksquare [X-Y]_{\text{补}} &= [X + (-Y)]_{\text{补}} \\ &= [X]_{\text{补}} + [-Y]_{\text{补}} \end{aligned}$$

❖ 任意两数之和的补码 = 两数的补码之和

❖ 任意两数之差的补码 = 被减数的补码与减数相反数的补码之和



求补运算: $[Y]_{\text{补}} \rightarrow [-Y]_{\text{补}}$

❖ 求补规则: 将 $[Y]_{\text{补}}$ 包括符号位在内每一位取反, 末位加1。

❖ 若 $[Y]_{\text{补}} = Y_S, Y_1 \dots Y_n$, 则:

$$\blacksquare [-Y]_{\text{补}} = \overline{Y_S} \overline{Y_1} \dots \overline{Y_n} + 1$$

❖ 若 $[Y]_{\text{补}} = Y_S.Y_1 \dots Y_n$, 则:

$$\blacksquare [-Y]_{\text{补}} = \overline{Y_S} \overline{Y_1} \dots \overline{Y_n} + 0.0 \dots 01$$

❖ 例: $[X]_{\text{补}} = 0.1101$, 则: $[-X]_{\text{补}} = 1.0011$

$[Y]_{\text{补}} = 1.1101$, 则: $[-Y]_{\text{补}} = 0.0011$



补码加减运算举例

❖ 【例4.2】 已知 $X=+1011$, $Y=-0100$, 用补码计算 $X+Y$ 和 $X-Y$ 。

■ 先写出补码:

$$[X]_{\text{补}} = 0,1011 \quad [Y]_{\text{补}} = 1,1100$$

$$[-Y]_{\text{补}} = 0,0100$$

■ 再计算:

$$\begin{array}{r} 0,1011 \\ + 1,1100 \\ \hline 0,0111 \end{array}$$

$$[X+Y]_{\text{补}} = 0, 0111$$

$$\begin{array}{r} 0,1011 \\ + 0,0100 \\ \hline 0,1111 \end{array}$$

$$[X-Y]_{\text{补}} = 0, 1111$$



补码加减运算举例

❖ 【例4.3】 $X=+1000$, $Y=+1001$, 求 $X+Y$;  正数

❖ 写出补码:

■ $[X]_{\text{补}} = 0,1000$ $[Y]_{\text{补}} = 0,1001$

$$\begin{array}{r} 0,1000 \\ + 0,1001 \\ \hline 1,0001 \end{array}$$

$[X]_{\text{补}} + [Y]_{\text{补}} = 1,0001$  真值 = -1111

$[X+Y]_{\text{补}} \neq [X]_{\text{补}} + [Y]_{\text{补}}$ why? 因为溢出!





2、补码加减运算的溢出判断

❖ **溢出**：运算结果超出机器数的表示范围

❖ 计算机硬件必须具备：

- 能够**检测**运算结果是否发生溢出；
- 能够**指示**运算结果是否发生溢出；

❖ 对于加减运算，**可能发生溢出的情况**：

- 同号（两数）相加
- 异号（两数）相减

Why?

❖ **补码加减运算判溢方法**：

- 单符号位判溢
- 进位判溢
- 双符号位判溢

❖ **一定会发生溢出的情况**：

- 正数相加，且运算结果符号位为1
- 负数相加，且运算结果符号位为0
- 正数 - 负数，且运算结果符号位为1
- 负数 - 正数，且运算结果符号位为0



溢出情况举例

❖ 【例4.3】

- (1) $X=+1000$, $Y=+1001$, 用补码计算 $X+Y$ 。
- (2) $X=-1000$, $Y=-1001$, 用补码计算 $X+Y$ 。
- (3) $X=1000$, $Y=-1001$, 用补码计算 $X-Y$ 。
- (4) $X=-1000$, $Y=1001$, 用补码计算 $X-Y$ 。

(1) 正数相加

$$\begin{array}{r} [X]_{\text{补}} \quad 0, 1000 \\ + [Y]_{\text{补}} \quad 0, 1001 \\ \hline [X+Y]_{\text{补}} \quad 1, 0001 \end{array}$$

❖ 正确结果: $8+9=17$

❖ 运算结果: -15

❖ 正溢出

(2) 负数相加

$$\begin{array}{r} [X]_{\text{补}} \quad 1, 1000 \\ + [Y]_{\text{补}} \quad 1, 0111 \\ \hline [X+Y]_{\text{补}} \quad 0, 1111 \end{array}$$

❖ 正确结果:

■ $(-8)+(-9)=-17$

❖ 运算结果: +15

❖ 负溢出

(3) 正数减负数

$$\begin{array}{r} [X]_{\text{补}} \quad 0, 1000 \\ + [-Y]_{\text{补}} \quad 0, 1001 \\ \hline [X-Y]_{\text{补}} \quad 1, 0001 \end{array}$$

❖ 正确结果:

■ $(+8)-(-9)=17$

❖ 运算结果: -15

❖ 正溢出

(4) 负数减正数

$$\begin{array}{r} [X]_{\text{补}} \quad 1, 1000 \\ + [-Y]_{\text{补}} \quad 1, 0111 \\ \hline [X-Y]_{\text{补}} \quad 0, 1111 \end{array}$$

❖ 正确结果:

■ $(-8)-(+9)=-17$

❖ 运算结果: +15

❖ 负溢出

常用的判溢方法 (补码加减运算)

(1) 单符号位判溢方法

$\overline{ADD}/SUB = 0$: 做加法

$\overline{ADD}/SUB = 1$: 做减法

❖ 【例4.3】的4种情况:

$$[X]_{\text{补}} = X_s X_1 \dots X_n$$

$$[Y]_{\text{补}} = Y_s Y_1 \dots Y_n$$

$$[X \pm Y]_{\text{补}} = F_s F_1 \dots F_n$$

运算	操作数X	X_s	操作数Y	Y_s	运算结果F	F_s	溢出情况
$X+Y$	正数	0	正数	0	负数	1	正溢出
$X+Y$	负数	1	负数	1	正数	0	负溢出
$X-Y$	正数	0	负数	1	负数	1	正溢出
$X-Y$	负数	1	正数	0	正数	0	负溢出

正数相加: 结果符号位为1

负数相加: 结果符号位为0

正数 - 负数: 结果符号为1

负数 - 正数: 结果符号为0

$$V = \overline{ADD}/SUB(\overline{X_s} \overline{Y_s} F_s + X_s Y_s \overline{F_s}) + \overline{ADD}/SUB(\overline{X_s} Y_s F_s + X_s \overline{Y_s} \overline{F_s})$$

$V=1$: 溢出



常用的判溢方法（补码加减运算）

(2) 进位判溢方法

- 最高有效位产生的进位 C_1 和符号位产生的进位 C_s 不同时，表明加减运算发生了溢出

$$V = C_1 \oplus C_s$$

最高有效位运算产生的进位

符号位运算产生的进位

	$\overline{ADD}/SUB$	X_s	Y_s	C_1	F_s	C_s	V	说明
加法	0	0	0	0	0	0	0	无溢出
	0	0	0	1	1	0	1	正溢出
	0	1	1	0	0	1	1	负溢出
	0	1	1	1	1	1	0	无溢出
减法	1	0	1	0	0	0	0	无溢出
	1	0	1	1	1	0	1	正溢出
	1	1	0	0	0	1	1	负溢出
	1	1	0	1	1	1	0	无溢出



常用的判溢方法（补码加减运算）

(2) 进位判溢方法

- 最高有效位产生的进位 C_1 和符号位产生的进位 C_S 不同时，表明加减运算发生了溢出

$$V = C_1 \oplus C_S$$

❖ 例： $X = +1000$, $Y = +1001$, 求 $X + Y$;

❖ $[X]_{\text{补}} = 0,1000$ $[Y]_{\text{补}} = 0,1001$

$$\begin{array}{r} 0 \ 1000 \\ + \ 0 \ 1001 \\ \hline 1 \ 0001 \end{array}$$

$C_1 = 1, C_S = 0$: 溢出



常用的判溢方法（补码加减运算）

变形补码

(3) 双符号位判溢方法

- X和Y采用双符号位补码参加运算，正数的双符号位为00，负数的双符号位为11；当运算结果的两位符号 F_{S1} F_{S2} 不同时（01或10），发生溢出。

$$\begin{array}{r} X_S X_S X_1 X_2 \dots X_n \\ + Y_S Y_S Y_1 Y_2 \dots Y_n \\ \hline F_{S1} F_{S2} F_1 F_2 \dots F_n \end{array}$$

$$\begin{array}{r} X_S X_1 X_2 \dots X_n \\ + Y_S Y_1 Y_2 \dots Y_n \\ \hline F_S F_1 F_2 \dots F_n \end{array}$$

$$F_{S1} = X_S \oplus Y_S \oplus C_S$$

$$F_{S2} = F_S$$

$$\blacksquare V = F_{S1} \oplus F_{S2}$$

$$\blacksquare V = F_{S1} \oplus F_{S2} = X_S \oplus Y_S \oplus C_S \oplus F_S$$

- $F_{S1} F_{S2} = 01$ ，则正溢出； $F_{S1} F_{S2} = 10$ ，则负溢出。



判溢方法举例

❖ 【例4.4】：

- (1) $X=+1000$, $Y=+1001$, 用补码计算 $X+Y$ 和 $X-Y$, 通过进位判溢法判溢。
- (2) $X=-1000$, $Y=1001$, 用双符号位补码计算 $X+Y$ 和 $X-Y$, 并判溢。

■ (1) 进位判溢法判溢

- 写出 $[X]_{\text{补}}$ 、 $[Y]_{\text{补}}$ 、 $[-Y]_{\text{补}}$
- 加法计算
- 判溢：比较符号位进位和次高位的进位是否相同

$$\begin{array}{r} [X]_{\text{补}} \quad 0, 1000 \\ + [Y]_{\text{补}} \quad 0, 1001 \\ \hline [X+Y]_{\text{补}} \quad 1, 0001 \end{array}$$

$C_s=0$, $C_1=1$
不同, 溢出

$$\begin{array}{r} [X]_{\text{补}} \quad 0, 1000 \\ + [-Y]_{\text{补}} \quad 1, 0111 \\ \hline [X-Y]_{\text{补}} \quad 1, 1111 \end{array}$$

$C_s=0$, $C_1=0$
相同, 无溢出



判溢方法举例

❖ 【例4.4】：

- (1) $X=+1000$, $Y=+1001$, 用补码计算 $X+Y$ 和 $X-Y$, 通过进位判溢法判溢。
- (2) $X=-1000$, $Y=1001$, 用双符号位补码计算 $X+Y$ 和 $X-Y$, 并判溢。

■ (2) 双符号位补码判溢

- 写出双符号的 $[X]_{\text{补}}$ 、 $[Y]_{\text{补}}$ 、 $[-Y]_{\text{补}}$
- 加法计算 (包含双符号位)
- 判溢: 比较运算结果两个符号位是否相同

$$\begin{array}{r} [X]_{\text{补}} \quad 11, 1000 \\ + [Y]_{\text{补}} \quad 00, 1001 \\ \hline [X+Y]_{\text{补}} \quad 00, 0001 \end{array}$$

$F_{S1} F_{S2} = 00$
无溢出

$$\begin{array}{r} [X]_{\text{补}} \quad 11, 1000 \\ + [-Y]_{\text{补}} \quad 11, 0111 \\ \hline [X-Y]_{\text{补}} \quad 10, 1111 \end{array}$$

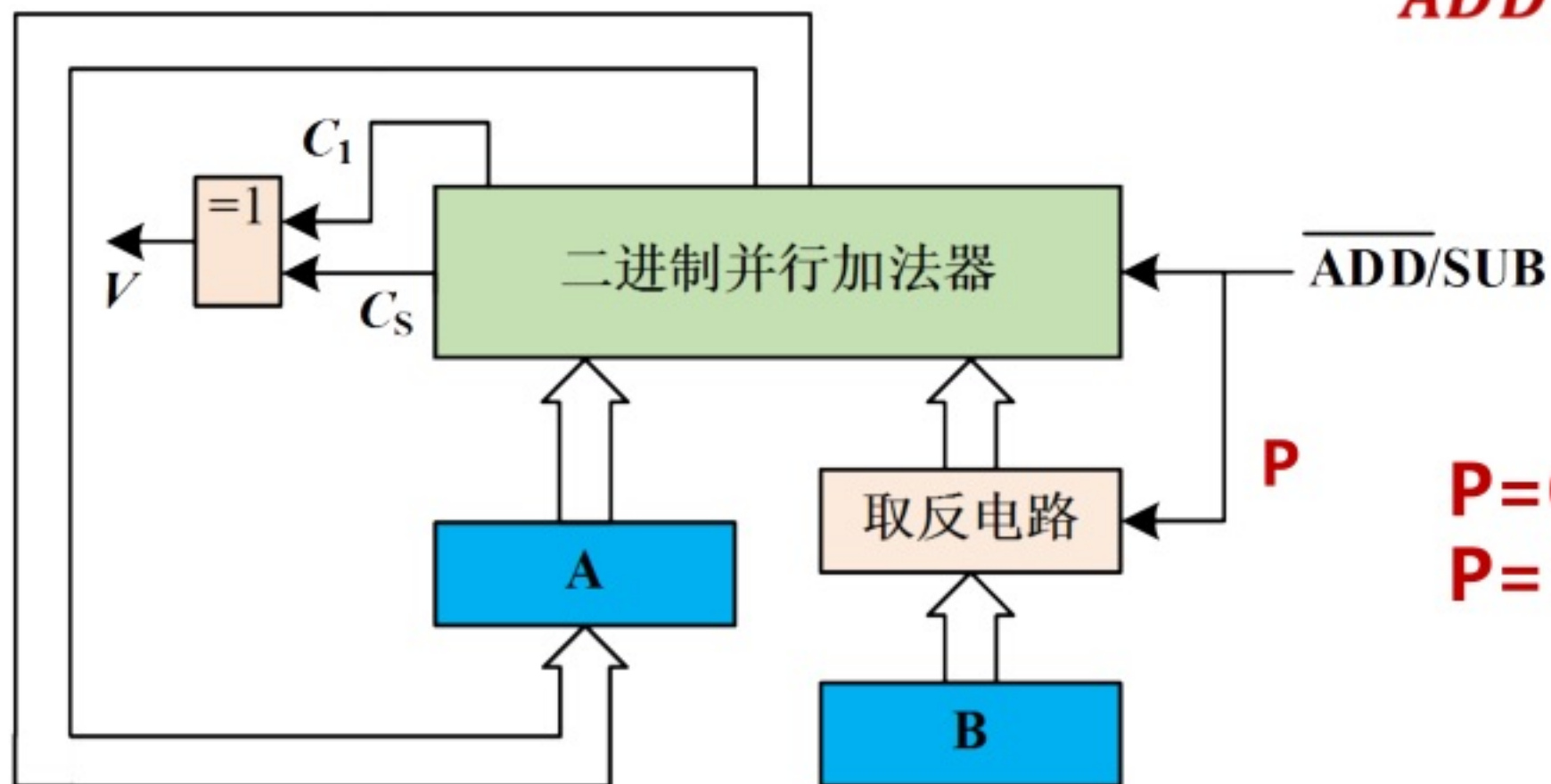
$F_{S1} F_{S2} = 10$
负溢出





3、补码加减运算器

$\overline{ADD}/SUB=0$: 加法
 $\overline{ADD}/SUB=1$: 减法



$P=0$: 不取反
 $P=1$: 取反



3、补码加减运算器的实现

- ❖ **核心部件**：一个普通的二进制并行加法器。
- ❖ **A**：累加器，存放 $[X]_{\text{补}}$
- ❖ **B**：寄存器，存放 $[Y]_{\text{补}}$
- ❖ **取反电路**：
 - 控制信号 $P=1$ ，取反电路工作； $P=0$ ：不工作（不取反）
 - $P = \overline{ADD/SUB}$
- ❖ **工作原理**：
 - $\overline{ADD/SUB} = 0$ ：补码加法器，B寄存器→并行加法器（ $P=0$ ）；
 - $\overline{ADD/SUB} = 1$ ：补码减法器，**B取反**→并行加法器（ $P=1$ ），且并行加法器的**最低位的进位=1**，即**B取反加1**；相当于 $[X]_{\text{补}}$ 加 $[-Y]_{\text{补}}$ ，→减法运算。





4.2.2 移码加减运算与判溢

❖ 移码和移码计算

$$[X]_{\text{移}} = 2^n + X \quad [Y]_{\text{移}} = 2^n + Y \quad -2^n \leq X \leq 2^n - 1$$

$$\begin{aligned} [X]_{\text{移}} + [Y]_{\text{移}} &= 2^n + X + 2^n + Y = 2^n + (2^n + X + Y) \\ &= 2^n + [X+Y]_{\text{移}} \end{aligned}$$

$$[X]_{\text{移}} + [-Y]_{\text{移}} = 2^n + [X-Y]_{\text{移}}$$

结果符号位
取反

❖ 移码和补码混合计算

$$[Y]_{\text{补}} = 2^{n+1} + Y \pmod{2^{n+1}}$$

$$[X]_{\text{移}} + [Y]_{\text{补}} = [X+Y]_{\text{移}}$$

$$[X]_{\text{移}} + [-Y]_{\text{补}} = [X-Y]_{\text{移}}$$



4.2.2 移码加减运算与判溢

❖ 移码运算结果判溢：双符号位

$$\begin{array}{r}
 [X]_{\text{移}} \quad 0 \quad X_S \quad X_1 \quad X_2 \quad \dots \quad X_n \\
 + [\pm Y]_{\text{补}} \quad Y_S \quad Y_S \quad Y_1 \quad Y_2 \quad \dots \quad Y_n \\
 \hline
 [X \pm Y]_{\text{移}} \quad F_{S1} \quad F_{S2} \quad F_1 \quad F_2 \quad \dots \quad F_n
 \end{array}$$

$[X]_{\text{移}}$: 最高符号位=0
 $[Y]_{\text{补}}$: 正常双符号位

❖ 移码运算结果溢出的判断条件是：

- $F_{S1} F_{S2} = 0 \times$ 时：结果正确
- $F_{S1} F_{S2} = 10$ 时：结果正溢出
- $F_{S1} F_{S2} = 11$ 时：结果负溢出

❖ 移码运算用于浮点数的阶码：

- 阶码运算结果：正溢出时，浮点数上溢
- 阶码运算结果：负溢出时，浮点数下溢，当作机器零处理



4.2.2 移码加减运算与判溢

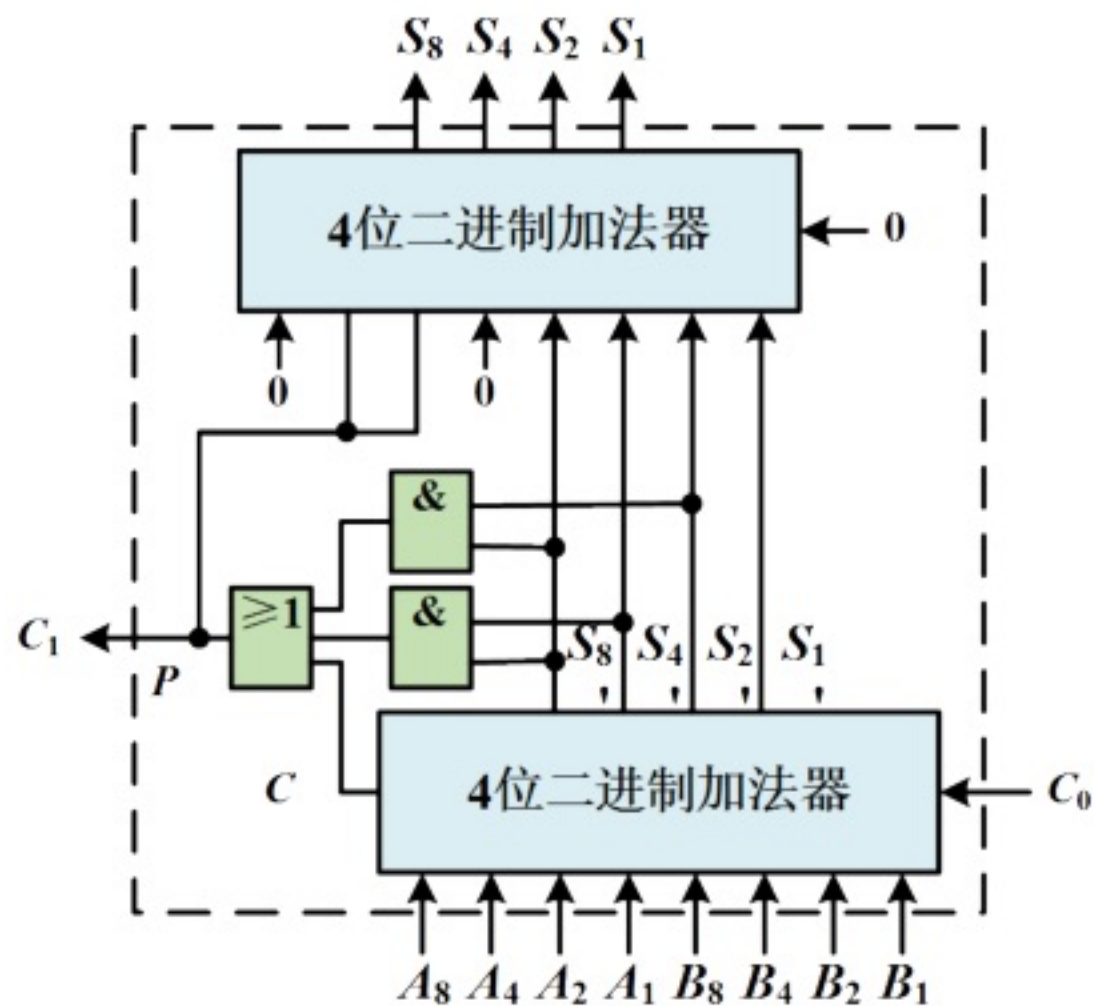
- ❖ 【例4.5】 $X=+1000$, $Y=+1001$, 用移码计算 $X+Y$ 和 $X-Y$.
- ❖ $[X]_{\text{移}} = 01, 1000$
- ❖ $[Y]_{\text{补}} = 00, 1001$
- ❖ $[-Y]_{\text{补}} = 11, 0111$
- ❖ $[X+Y]_{\text{移}} = [X]_{\text{移}} + [Y]_{\text{补}} = 01, 1000 + 00, 1001 = 10, 0001$
 - ❖ 最高符号位为1, 溢出, 最低符号位为0, 正溢出
- ❖ $[X-Y]_{\text{移}} = [X]_{\text{移}} + [-Y]_{\text{补}} = 01, 1000 + 11, 0111 = 00, 1111$
 - ❖ 最高符号位为0, 无溢出, $X-Y = -1$



4.2.3 十进制加法运算

- ❖ 十进制加法器实现：二进制加法器+校正电路
- ❖ 当相加的两数之和 $S > 9$ 时，加6校正；当 $S \leq 9$ 时，结果正确，不需校正。
- ❖ 对于8421 BCD码，校正条件为：
 - $S_8'S_4'S_2'S_1' \geq 1010$
 - 或 $C=1$
- ❖ 校正条件的逻辑式：

$$P = S_8' S_4' + S_8' S_2' + C$$





4.2.3 十进制加法运算

❖ 余3码加法器：校正条件=进位C

- C=0：减3校正（即加1101校正）
- C=1，加3校正（即加0011校正）

❖ 目前的计算机硬件中：**一般不设置专用的BCD码运算器**

- 先使用二进制的加减乘除运算指令进行运算
- 再用BCD码校正指令对运算结果进行调整

❖ 例如Intel X86 CPU指令系统：

- 设置了6条十进制调整指令：DAA/AAA、DAS/AAS、AAM、AAD，分别对二进制加减乘除运算指令的结果进行修正





4.3 定点数的乘法运算

❖ 4.3.1 乘除法实现方法概述

❖ 4.3.2 原码串行乘法器

❖ 4.3.3 补码串行乘法器

❖ 4.3.4 阵列乘法器

串行乘法
算法





4.3.1 乘除法实现方法概述

- ❖ 计算机的软硬件在逻辑上具有一定的等价性
- ❖ 实现乘除法运算，有三种方式：
- ❖ (1) 用软件实现
 - 硬件上：设计简单，没有乘法器和除法器。
 - 指令系统：没有乘除指令，但有加/减法和移位指令
 - 实现：乘除运算通过编制一段子程序来实现
 - 算法：程序中运用串行乘除运算算法，循环累加、右移指令→乘法，循环减、左移指令→除法。
 - 运算速度：较慢。
 - 适用场合：早期的单片机。



4.3.1 乘除法实现方法概述

❖ (2) 用串行硬件乘法器和除法器实现

- **硬件上：**设置有并行加法器、移位器和若干循环、计数控制逻辑电路搭成的**串行乘除法器**。
- **指令系统：**具有乘除法指令。
- **实现：**乘除运算通过**微程序一级（硬件 + 微程序）**来实现。
- **算法：**在微程序中**依据串行乘除运算算法**，循环累加、右移指令→乘法，循环减、左移指令→除法。
- **运算速度：**有所提高，硬件设计相对复杂。
- **适用场合：**低性能CPU

❖ (3) 用高速的阵列乘法器和阵列除法器实现

- **硬件上：**设置有专用的、并行运算的**阵列乘法器和阵列除法器**。
- **指令系统：**具有乘除法指令。
- **实现：****完全通过硬件**来实现。
- **算法：**并行乘/除法。
- **运算速度：**很快，但硬件设计相当复杂。
- **适用场合：**高性能CPU。



乘除法运算的3种实现方法

对比	软件实现	串行乘除法器	并行乘除法器
运算器硬件	只有加减法器和移位器	有乘法器和除法器	并行乘、除法器
乘除指令	无，但有加减法指令和移位指令	有	有
实现方法	使用子程序实现	硬件 + 微程序	硬件
工作原理	串行乘除法算法	串行算法	并行算法
运算速度	慢	较慢	快
适用场合	早期单片机	低性能单片机	高性能CPU





4.3.2 原码串行乘法器





1、手工乘法算法

❖ 手工计算 1011×1101 ，步骤：

❖ 手工算法：

- 对应每一位乘数求得1项位积
- 将位积逐位左移
- 将所有的位积一次相加，得到最后的乘积

❖ 在计算机上能否实现？如何实现？

- 能进行4个操作数一次相加的加法器？
- 加法器的宽度为8位

$$\begin{array}{r} 1101 \\ \times 1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline 10001111 \end{array}$$

Arrows indicate the shifting of partial products: from the first '1' of the multiplier to the first partial product, from the second '1' to the second, from the third '0' to the third, and from the fourth '1' to the fourth.

$$\begin{array}{r} 1011 \\ \times 1101 \\ \hline 1011 \\ 0000 \\ 1011 \\ 1011 \\ \hline 10001111 \end{array}$$



改造手工算法适合计算机硬件实现：

手工算法

将所有位积一次相加

位积左移

机器算法

位积逐次累加

部分积

部分积右移

即：累加，右移，
构成循环



2、原码一位乘法算法：累加、右移

- ❖ $[X]_{\text{原}} = X_s X_1 X_2 \dots X_n$
 - ❖ $[Y]_{\text{原}} = Y_s Y_1 Y_2 \dots Y_n$
 - ❖ $P = X \cdot Y$, P_s 是积的符号
- ① 符号位单独处理 $P_s = X_s \oplus Y_s$
 - ② 绝对值进行数值运算 $|P| = |X| * |Y|$
 - ③ 初始部分积为0, $Y_i = 1$, 部分积加 $|X|$, $Y_i = 0$, 部分积加0, 累加结果右移一位, 得新部分积。
 - ④ 累加右移n次, 即 $i = n, n-1, \dots, 2, 1$

举例

❖ 【例4.6】 $X=+1011$, $Y=-1101$
，用原码一位乘法计算 $P=X \cdot Y$ 。

❖ ①先写出X和Y的原码、X和Y的绝对值，并求出乘积的符号位

- $[X]_{\text{原}}=0, 1011$
- $[Y]_{\text{原}}=1, 1101$
- $P_s = X_s \oplus Y_s = 0 \oplus 1 = 1$

❖ ②进行绝对值的乘法运算，得到积的绝对值： $|P| = |X| \cdot |Y|$

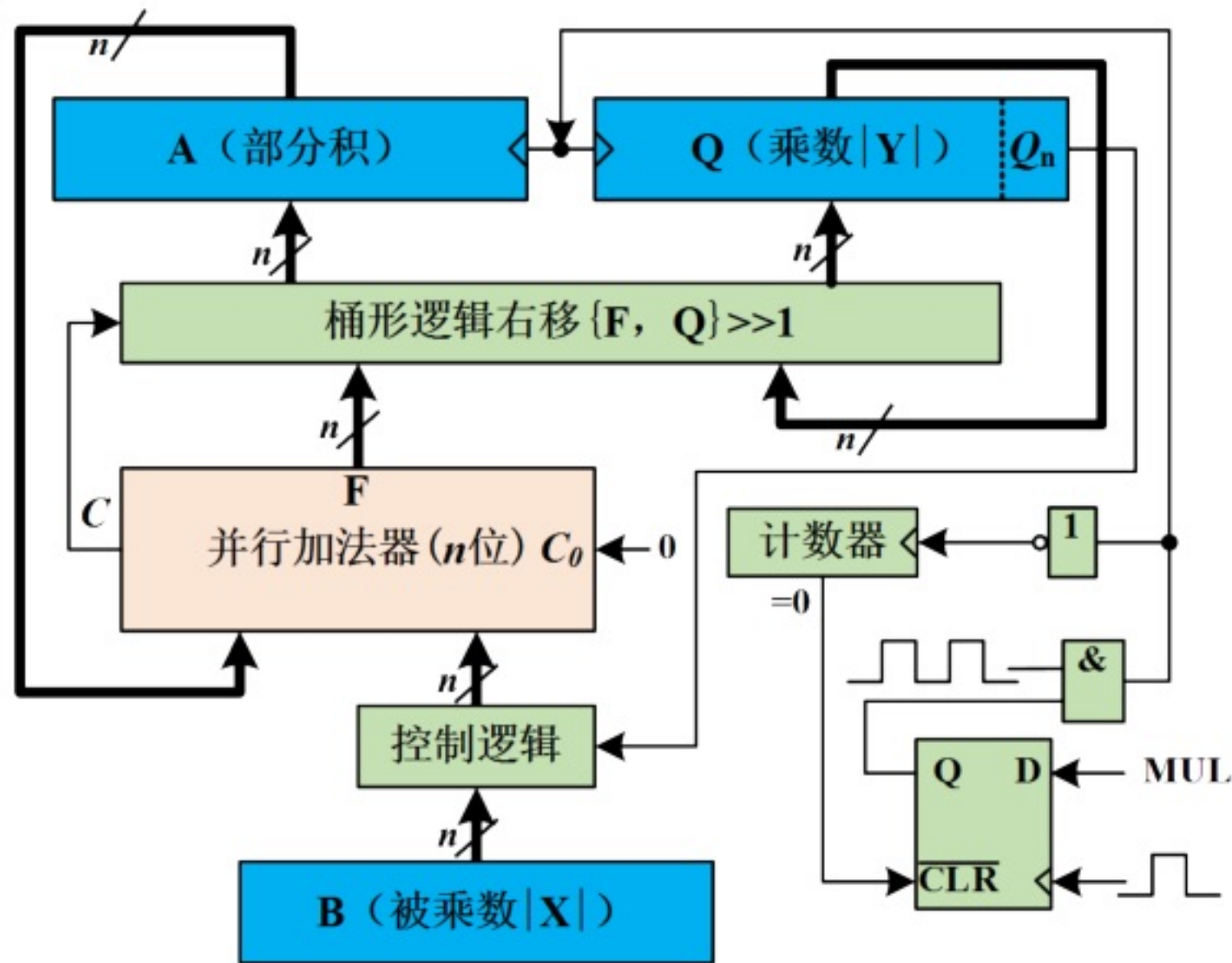
❖ ③将符号位置入积的绝对值左边最高位，得P的原码：

$[P]_{\text{原}}=1, 10001111$

部分积P	乘数 Y	操作说明
0, 0000	1 1 0 1	
+ 0, 1011		$Y_4=1, + X $
0, 1011	1 1 0 1	
0, 0101	1 1 1 0	右移一位
+ 0, 0000		$Y_3=0, +0$
0, 0101	1 1 1 0	
0, 0010	1 1 1 1	右移一位
+ 0, 1011		$Y_2=1, + X $
0, 1101	1 1 1 1	
0, 0110	1 1 1 1	右移一位
+ 0, 1011		$Y_1=1, + X $
1, 0001	1 1 1 1	
0, 1000	1 1 1 1	右移一位



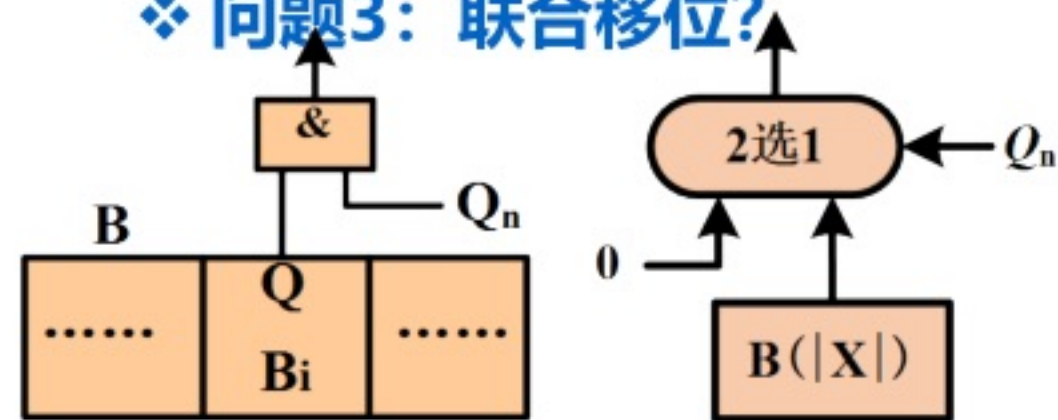
3、原码乘法的硬件实现：加法器、移位器



❖ 问题1：循环次数控制？

❖ 问题2：累加的位积不同？

❖ 问题3：联合移位？

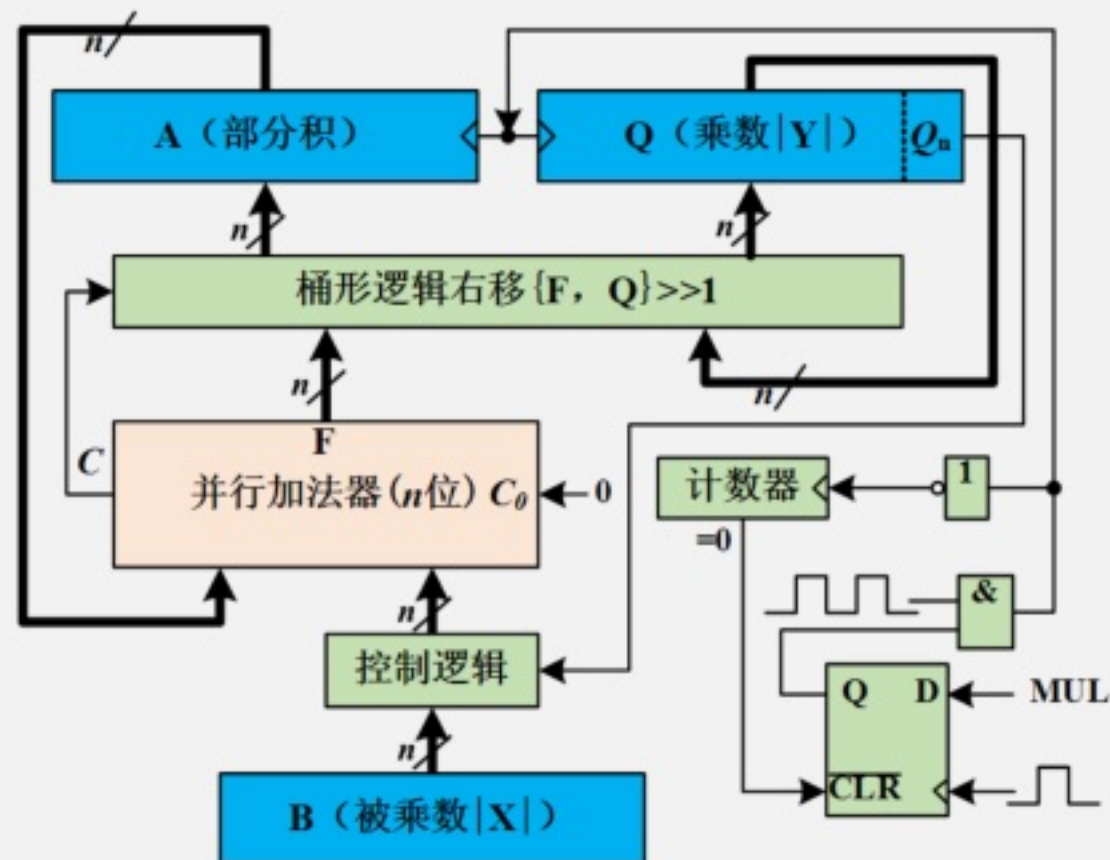


❖ 控制逻辑电路

- $Q_n=0$: $+0$ ($00\dots0\&B$)
- $Q_n=1$: $+|X|$ ($11\dots1\&B$)

3、原码乘法的硬件实现

- ❖ A: 累加寄存器
- ❖ B: 被乘数寄存器
- ❖ Q: 乘数寄存器
- ❖ 桶形移位器: F、Q联合逻辑右移
 - $A \leftarrow \{C, F_1 \dots F_{n-1}\}$ $Q \leftarrow \{F_n, Q_1 \dots Q_{n-1}\}$



初始:

- ❖ $A=0$
- ❖ $B=|X|$
- ❖ $Q=|Y|$
- ❖ 计数器=n

累加、右移n次

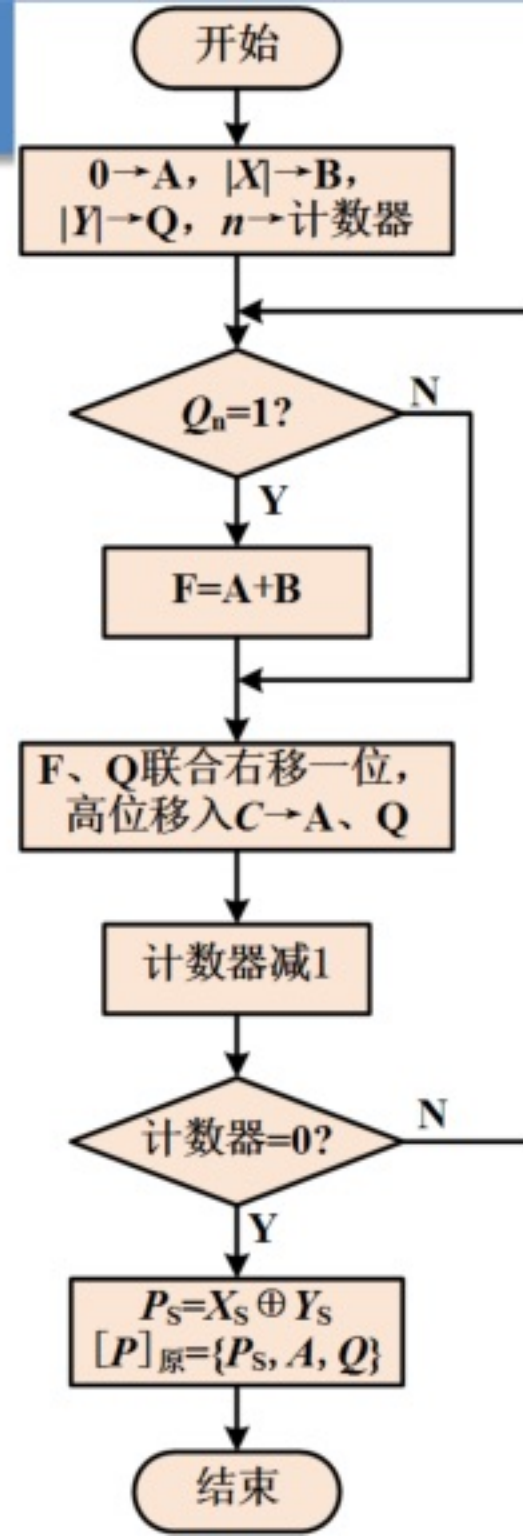
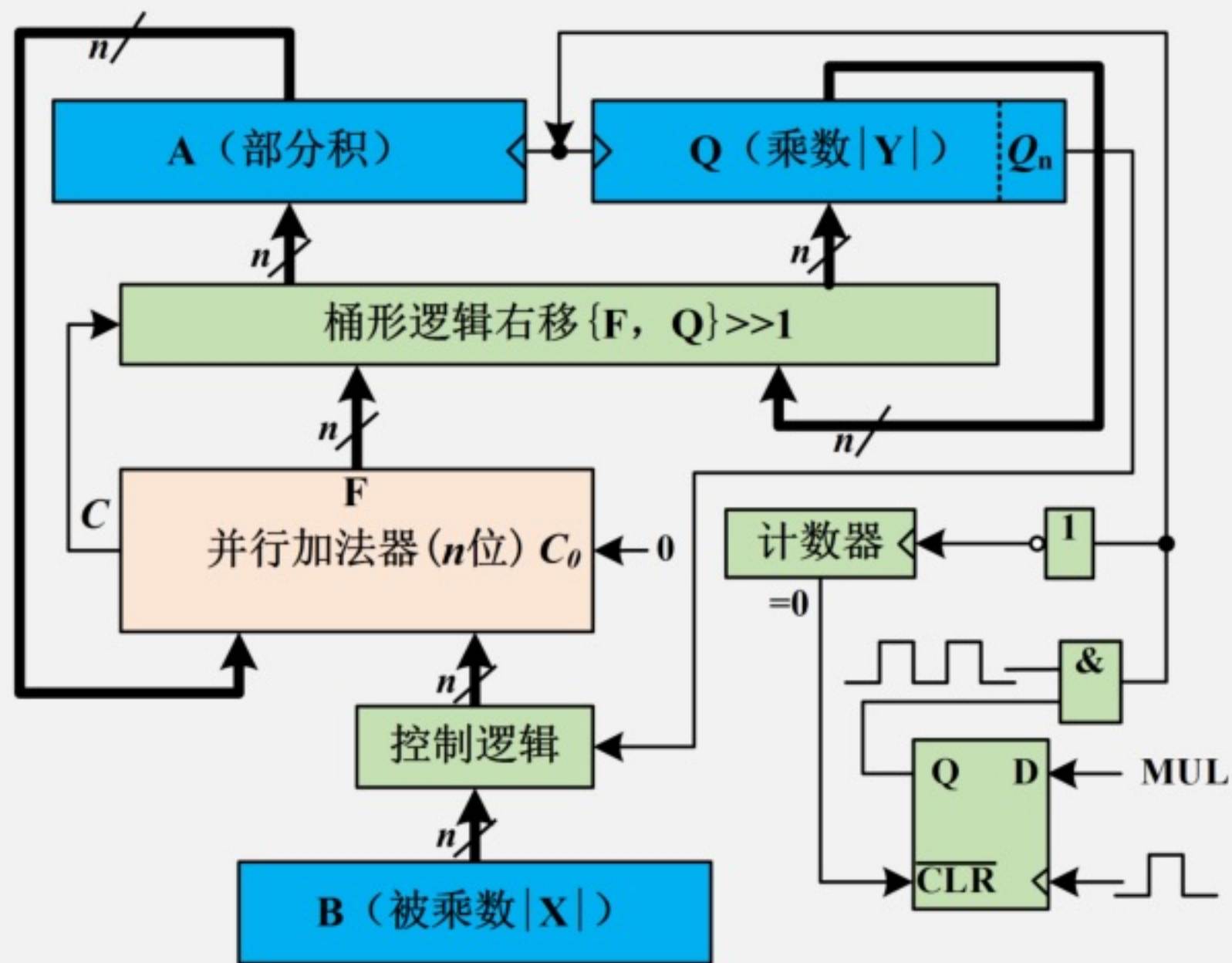


结果:

- ❖ A = 乘积绝对值高位
- ❖ $B = |X|$
- ❖ Q = 乘积绝对值低位
- ❖ 计数器=0



原码一位乘法流程：





4.3.3 补码串行乘法器

❖ 1、补码乘法算法

- (1) 补码乘法的校正算法
- (2) 补码乘法的Booth算法

❖ 2、补码乘法的硬件实现





(1) 补码乘法的校正算法

- ❖ $[X]_{\text{补}} = X_0 . X_1 \dots X_n$, $[Y]_{\text{补}} = Y_0 . Y_1 \dots Y_n$
- ❖ 则有: $[X \cdot Y]_{\text{补}} = [X]_{\text{补}} \cdot (0.Y_1 \dots Y_n) + Y_0 \cdot [-X]_{\text{补}}$
- ❖ 证明:
- ❖ 当被乘数X的符号任意, **Y为正数**时, 根据补码定义有:

$$[X]_{\text{补}} = 2 + X = 2^{n+1} + X \pmod{2} \quad [Y]_{\text{补}} = Y \quad \text{则:}$$

$$[X]_{\text{补}} \cdot [Y]_{\text{补}} = (2^{n+1} + X) \cdot Y = 2^{n+1} \cdot Y + X \cdot Y$$

$$= 2^{n+1} \cdot (0.Y_1 \dots Y_n) + X \cdot Y$$

$$= 2 \cdot (Y_1 \dots Y_n) + X \cdot Y = 2 + X \cdot Y \pmod{2} = [X \cdot Y]_{\text{补}}$$

- ❖ 即: $Y > 0$ 时,

- $[X \cdot Y]_{\text{补}} = [X]_{\text{补}} \cdot [Y]_{\text{补}} = [X]_{\text{补}} \cdot (0.Y_1 \dots Y_n)$



(1) 补码乘法的校正算法

❖ 当被乘数 X 的符号任意, Y 为负数时:

$[Y]_{\text{补}} = 2 + Y = 1.Y_1 \dots Y_n$, 则:

$$Y = [Y]_{\text{补}} - 2 = 0.Y_1 \dots Y_n - 1$$

$$\begin{aligned}[X \cdot Y]_{\text{补}} &= [X \cdot 0.Y_1 \dots Y_n - X]_{\text{补}} \\ &= [X \cdot 0.Y_1 \dots Y_n]_{\text{补}} + [-X]_{\text{补}}\end{aligned}$$

因为 $0.Y_1 \dots Y_n > 0$, 所以:

$$[X \cdot 0.Y_1 \dots Y_n]_{\text{补}} = [X]_{\text{补}} \cdot (0.Y_1 \dots Y_n)$$

❖ 所以: $Y < 0$ 时,

$$\blacksquare [X \cdot Y]_{\text{补}} = [X]_{\text{补}} \cdot (0.Y_1 \dots Y_n) + [-X]_{\text{补}}$$





(2) 补码乘法的Booth算法

❖ 令 $Y_0=Y_S$, $Y_{n+1}=0$, 做出如下推导：

$$\begin{aligned}[X \cdot Y]_{\text{补}} &= [X]_{\text{补}} \cdot (0.Y_1 \dots Y_n) + Y_0 \cdot [-X]_{\text{补}} \\&= [X]_{\text{补}} \cdot (Y_1 \cdot 2^{-1} + Y_2 \cdot 2^{-2} + \dots + Y_n \cdot 2^{-n} - Y_0) \\&= [X]_{\text{补}} \cdot [Y_1 \cdot (2^0 - 2^{-1}) + Y_2 \cdot (2^{-1} - 2^{-2}) + \dots + Y_n \cdot (2^{-n+1} - 2^{-n}) - Y_0 \cdot 2^0] \\&= [X]_{\text{补}} \cdot [Y_1 \cdot 2^0 - Y_1 \cdot 2^{-1} + Y_2 \cdot 2^{-1} - Y_2 \cdot 2^{-2} + \dots + Y_n \cdot 2^{-n+1} - Y_n \cdot 2^{-n} - Y_0 \cdot 2^0] \\&= [X]_{\text{补}} \cdot [(Y_1 - Y_0) \cdot 2^0 + (Y_2 - Y_1) \cdot 2^{-1} + (Y_3 - Y_2) \cdot 2^{-2} + \dots + (Y_n - Y_{n-1}) \cdot 2^{-n+1} - Y_n \cdot 2^{-n}] \\&= [X]_{\text{补}} \cdot [(Y_1 - Y_0) \cdot 2^0 + (Y_2 - Y_1) \cdot 2^{-1} + (Y_3 - Y_2) \cdot 2^{-2} + \dots + (Y_n - Y_{n-1}) \cdot 2^{-n+1} + (Y_{n+1} - Y_n) \cdot 2^{-n}] \\&= [X]_{\text{补}} \cdot (a_0 \cdot 2^0 + a_1 \cdot 2^{-1} + a_2 \cdot 2^{-2} + \dots + a_{n-1} \cdot 2^{-n+1} + a_n \cdot 2^{-n})\end{aligned}$$

其中，将乘数Y的补码在最末位添加一位附加位 Y_{n+1} （初始为0）， $a_i = Y_{i+1} - Y_i$ ，
 $i=0, 1, \dots, n-1, n$



Booth算法的运算规则

假设 $[Y]_{\text{补}} = Y_0 . Y_1 \dots Y_n$

- ① 被乘数 X 、乘数 Y ：均以补码的形式参加乘法运算，运算的结果是积的补码
- ② 部分积、 X 、 Y ：采用单符号位；部分积初值为0；运算前，在乘数 Y 的补码末位后添加一位附加位 Y_{n+1} ，初始为0
- ③ 根据 $Y_n Y_{n+1}$ 的值，按照下表进行累加右移操作，右移时遵循补码的移位规则
- ④ 累加 $n+1$ 次，右移 n 次，即最后一次不右移。

Y_n	Y_{n+1}	$a_n = Y_{n+1} - Y_n$	操作
0	0	0	+0, 右移一位
0	1	1	+ $[X]_{\text{补}}$, 右移一位
1	0	-1	+ $[-X]_{\text{补}}$, 右移一位
1	1	0	+0, 右移一位

Booth算法举例

- ❖ 【例4.7】 $X=+0.1011$, $Y=-0.1101$, 用补码一位乘法的Booth算法计算 $P=X \cdot Y$ 。
- ❖ ① 写出 $[X]_{\text{补}}$ 、 $[Y]_{\text{补}}$ 和 $[-X]_{\text{补}}$, 并在 $[Y]_{\text{补}}$ 的末尾添加附加位 $Y_{n+1}=0$
 - $[X]_{\text{补}}=00.1011$ $[Y]_{\text{补}}=1.0011 \underline{0}$
 - $[-X]_{\text{补}}=11.0101$
- ❖ ② 列式计算: 累加右移 $n+1$ 次, 每次累加的位积 = $(Y_{n+1}-Y_n) * [X]_{\text{补}}$; 最后一次不右移; 部分积和乘数联合右移
- ❖ ③ 结果就是积的补码:

$$[X \cdot Y]_{\text{补}} = 1.0111 \ 0001$$

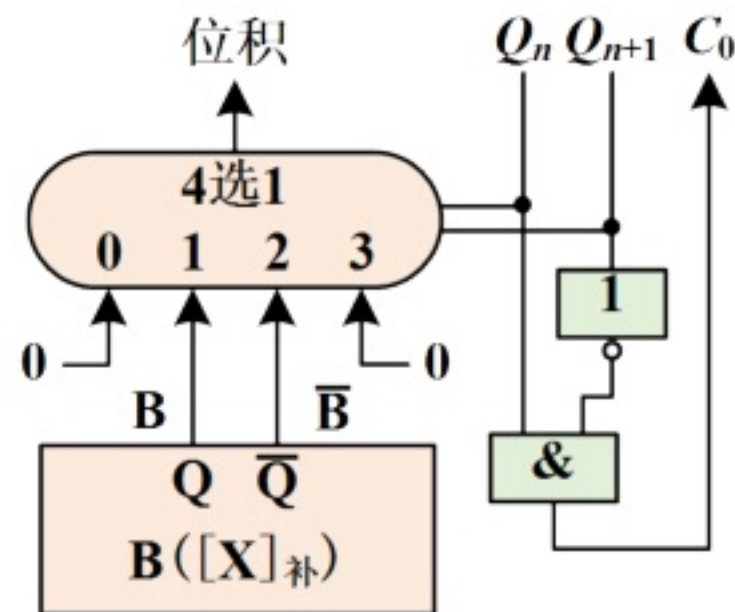
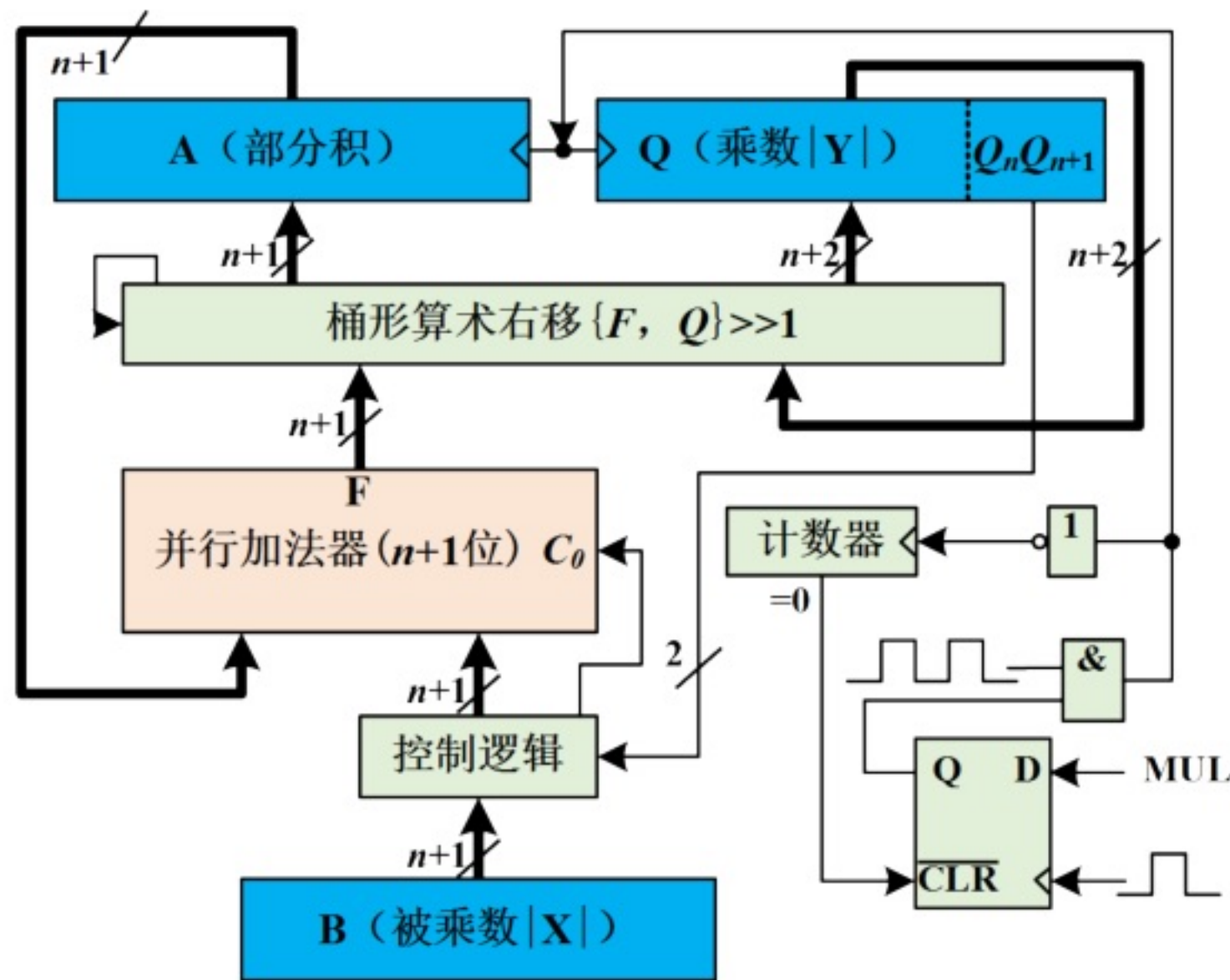
$$X \cdot Y = -0.1000 \ 1111$$

部分积	乘数Y ($Y_n Y_{n+1}$)	操作说明
00.0000	1.0 0 1 <u>1</u> 0	
+ 11.0101		$Y_4 Y_5 = 10$, $+[-X]_{\text{补}}$
11.0101		
11.1010	1 1.0 0 <u>1</u> 1	右移一位
+ 00.0000		$Y_3 Y_4 = 11$, $+0$
11.1010		
11.1101	0 1 1.0 <u>0</u> 1	右移一位
+ 00.1011		$Y_2 Y_3 = 01$, $+ [X]_{\text{补}}$
00.1000		
00.0100	0 0 1 <u>1</u> 0 0	右移一位
+ 00.0000		$Y_1 Y_2 = 00$, $+0$
00.0100		
00.0010	0 0 0 1 <u>1</u> 0	右移一位
+ 11.0101		$Y_0 Y_1 = 10$, $+[-X]_{\text{补}}$
11.0111	0 0 0 1	



2、补码乘法的硬件实现

Booth乘法的硬件实现



控制逻辑电路

- ❖ $Q_n Q_{n+1}$ 选择4个位积: 0 、 B 、 $\overline{B}$ 、 0
- ❖ 当 $Q_n Q_{n+1} = 10$ 时, 输出 $C_0 = 1$
- ❖ 位积是 $\overline{B} + 1 = [-X]_{\text{补}}$



补码乘法的Booth算法流程

❖ A、B: $n+1$ 位寄存器

❖ Q: $n+2$ 位寄存器

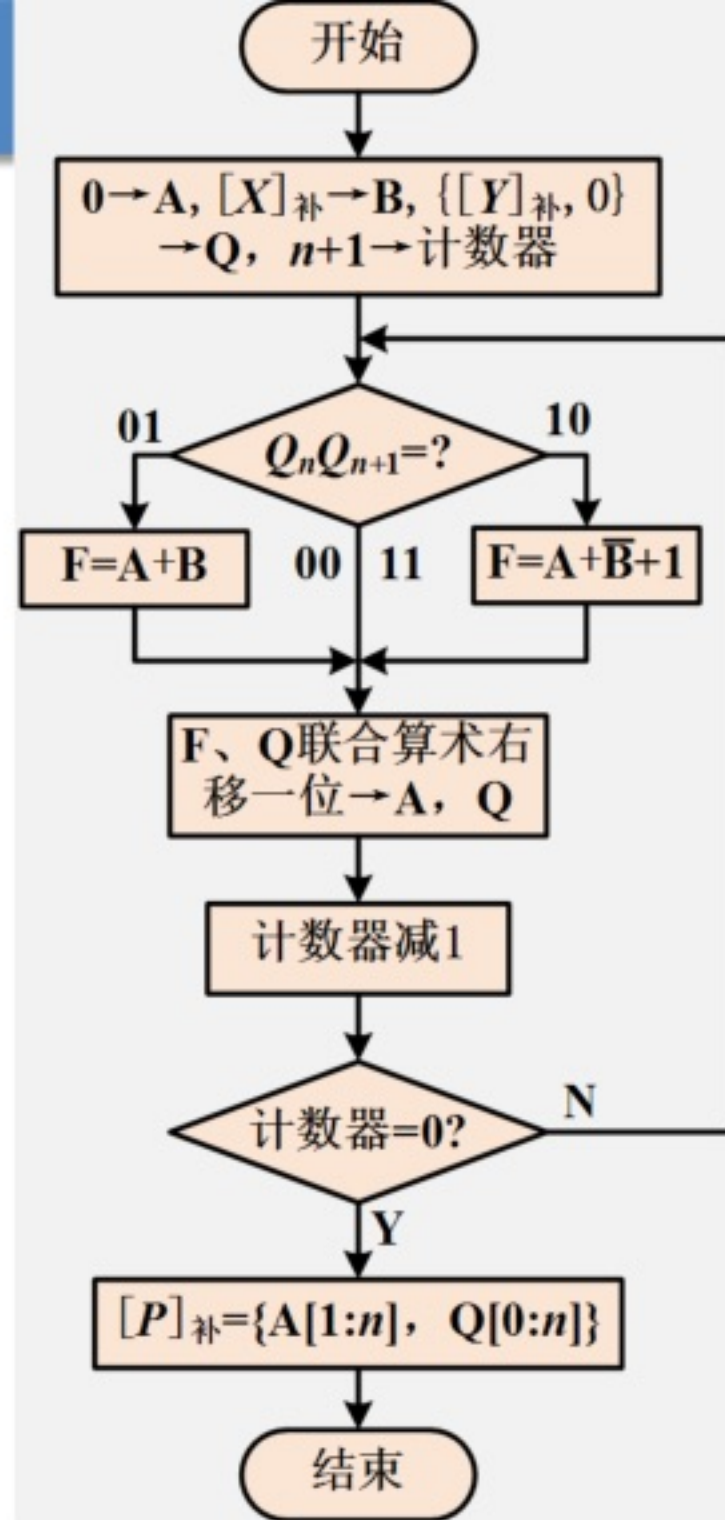
❖ 初始化:

- $A=0$ 、 $B=[X]_{\text{补}}$
- $Q=\{[Y]_{\text{补}}, 0\}$

❖ 计算: 累加右移 $n+1$ 次

❖ 结果:

- $[P]_{\text{补}}=\{A[1:n], Q[0:n]\}$
- 即: 去掉A的最高位, Q的最低位





4.3.4 阵列乘法器

❖ 原理类似于二进制手工算法

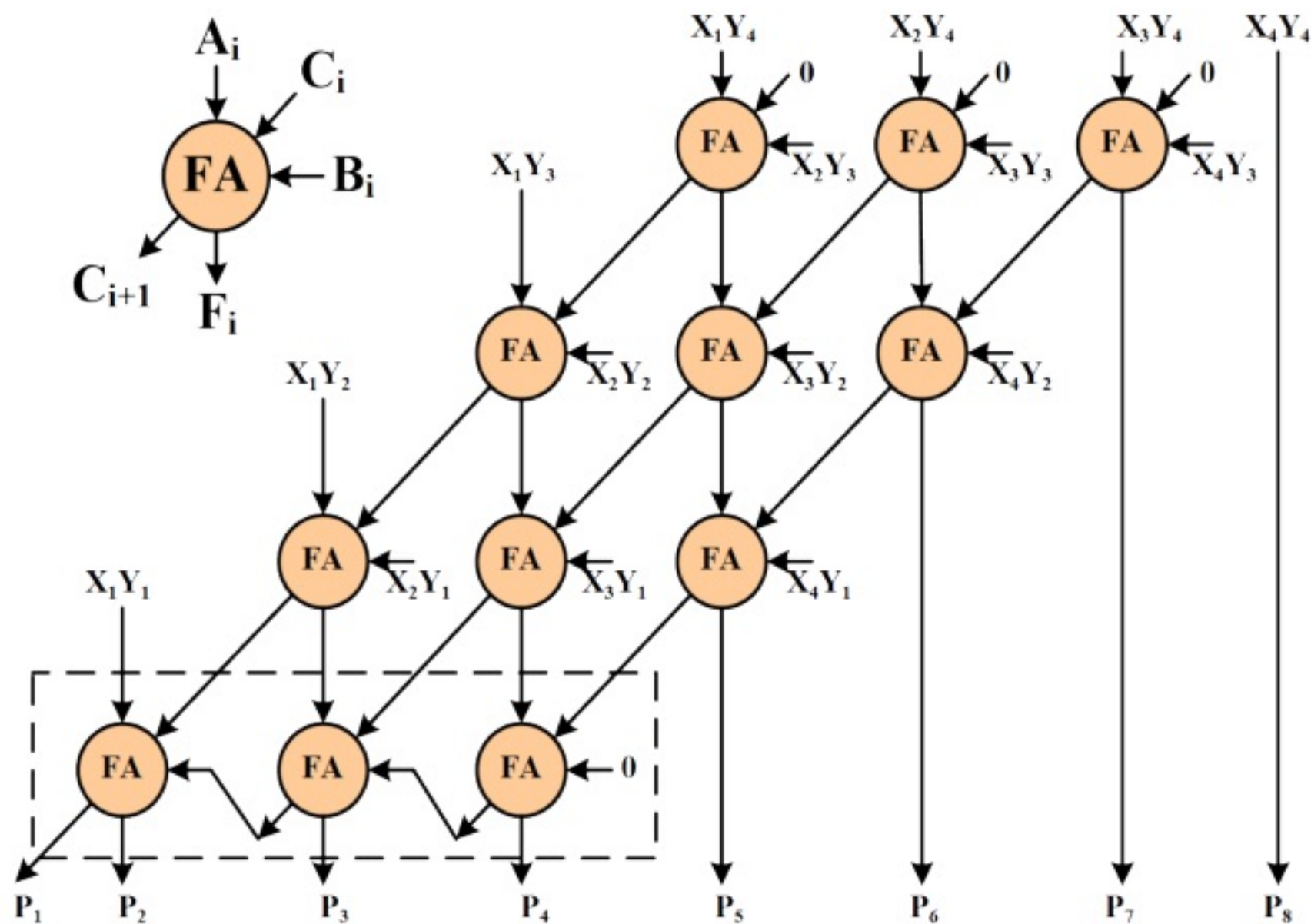
- 位积的每一位 X_iY_j 用一个与门产生。
- 每一次的累加都用单独一组FA实现。
- 本次累加的FA之间的进位，送至下一次再累加。

				X_1	X_2	X_3	X_4
			$\times$	Y_1	Y_2	Y_3	Y_4
				X_1Y_4	X_2Y_4	X_3Y_4	X_4Y_4
		X_1Y_3	X_2Y_3	X_3Y_3	X_4Y_3		
	X_1Y_2	X_2Y_2	X_3Y_2	X_4Y_2			
+	X_1Y_1	X_2Y_1	X_3Y_1	X_4Y_1			
P_1	P_2	P_3	P_4	P_5	P_6	P_7	P_8

				1101
		$\times$		1011
			1101	
			1101	
			0000	
			1101	
				10001111



1、绝对值阵列乘法器



❖ $X_i Y_j$: 对应一个与门

❖ 每个FA的进位: 向斜线方向传递

- 本行所有FA产生的进位传递到下一行的高位FA, 下一次再执行加法

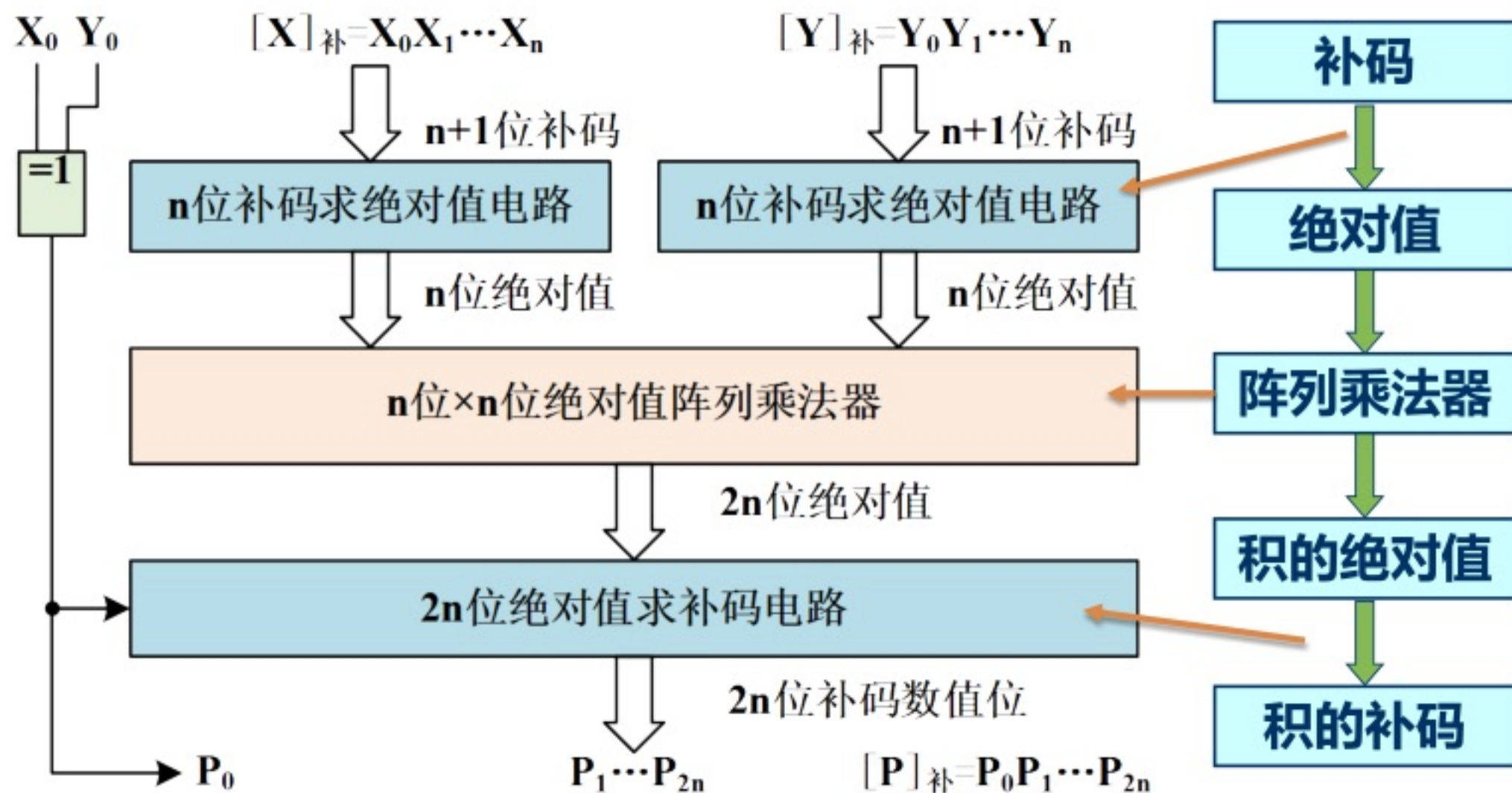
❖ 进位保留加法器

CSA(Carry Save Adder): 减少延迟

- 每一行全加器构成的加法器: 能对三个数执行没有进位依赖的加法, 产生等长的和数与进位

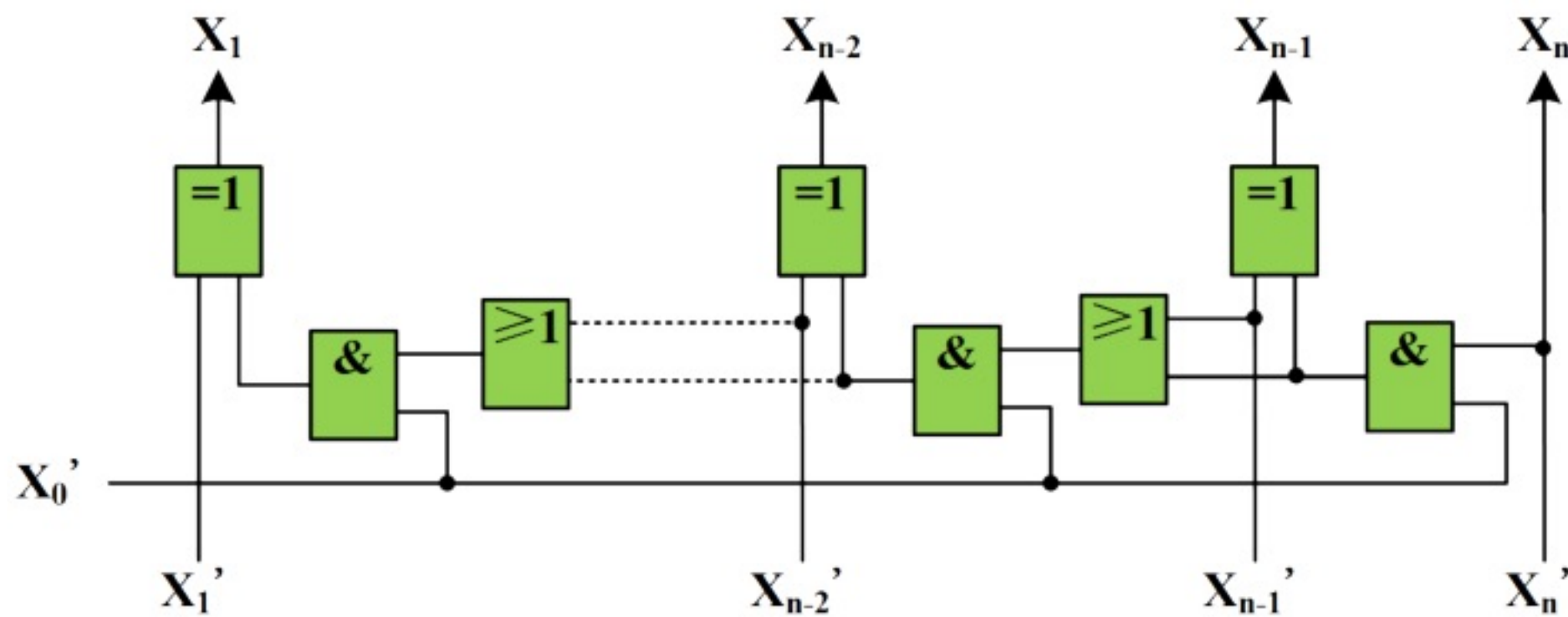


2、补码阵列乘法器





补码→绝对值电路

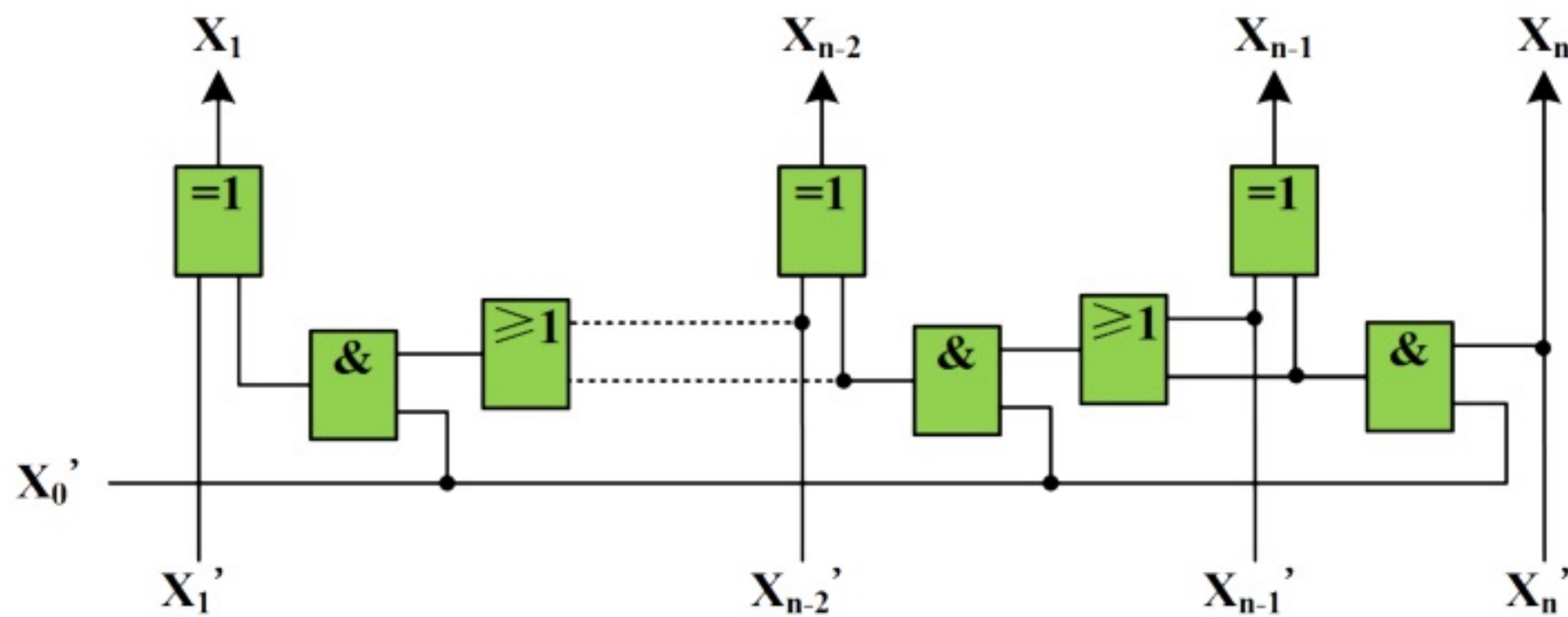


❖ 输入: $[X]_{\text{补}} = X_0' X_1' \dots X_{n-1}' X_n'$

❖ 得到: $|X| = X_1 \dots X_{n-1} X_n$



绝对值→补码电路



❖ 输入: $|X| = X_1' \dots X_{n-1}'X_n'$, 符号位 X_0'

$X < 0 : X_0' = 1, X \geq 0 : X_0' = 0$

❖ 得到: $[X]_{\text{补}} = X_0' X_1 \dots X_{n-1} X_n$





4.4 定点数的除法运算

❖ 定点除法运算：使用原码和补码来实现

❖ 计算机对参加除法运算的数据有要求：

- ①除数不为0
- ②为防止结果溢出：
 - 定点小数进行除法时：被除数的绝对值（ n 位）要小于除数的绝对值（ n 位）
 - 定点整数进行除法时：被除数绝对值（ $2n$ 位）的高 n 位要小于除数的绝对值（ n 位）

❖ 4.4.1 原码串行除法器

❖ 4.4.2 阵列除法器





4.4.1 原码串行除法器

原码除法的硬件实现





1、手工除法算法

❖ $X = +0.1011$, $Y = -0.1101$, 求 $X \div Y$

❖ 手工除法:

- 判断被除数是否大于除数, 决定商0/1。
- 若商1, 则减除数 (得部分余数), 再添加一位数或者添加0
- 若商0, 则直接添加一位数或者添加0

整数

小数

部分余数

❖ 改进手工算法 $\rightarrow$ 适合机器运算:

- 计算机通过做减法测试来实现判断: 结果大于等于0, 表明够减, 商1; 结果小于0, 表明不够减, 商0。
- 计算机将部分余数左移一位, 再直接与不右移的除数相减。

❖ 即: 减 (+除数相反数的补码)

左移, 构成循环

$$\begin{array}{r} 0.1101 \\ 0.1101 \overline{) 0.10110} \\ \underline{1101} \\ 10010 \\ \underline{1101} \\ 10100 \\ \underline{1101} \\ 0111 \end{array}$$





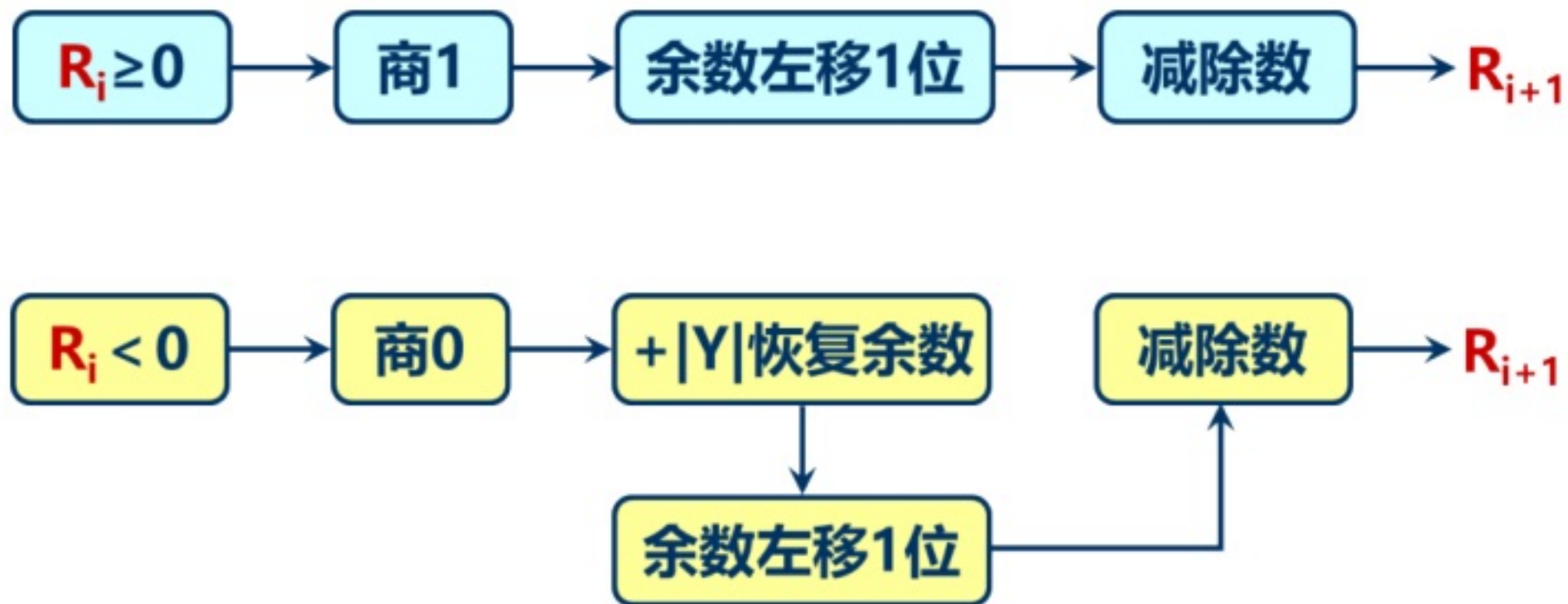
2、原码恢复余数算法

若为定点整数，一般是2n位除以n位

- ❖ $[X]_{\text{原}} = X_S . X_1 X_2 \dots X_n$, $[Y]_{\text{原}} = Y_S . Y_1 Y_2 \dots Y_n$
- ❖ Q: $X \div Y$ 的商, Q_S : 商的符号, R: $X \div Y$ 的余数, R_S : 余数的符号
- ❖ 原码除法运算的规则是:
 - ① $Q_S = X_S \oplus Y_S$, $R_S = X_S$, $|Q| = (|X| - |R|) \div |Y|$
 - ② 余数和被除数、除数均采用双符号位; 初始余数为 $|X|$, 初始商值为全0。
 - ③ 部分余数减去 $|Y|$ (通过 $+ [-|Y|]_{\text{补}}$ 来实现):
 - 结果符号位为0: 够减, 上商1, 余数左移一位
 - 结果符号位为1: 不够减, 上商0, 先加 $|Y|$ 恢复余数, 然后余数左移一位
 - ④ 循环操作步骤③, 共做 $n+1$ 次, 最后一次不左移
 - 若最后一次上商0, 则必须 $+ |Y|$ 恢复余数;
 - 若为定点小数除法, 余数则为最后计算得到的余数右移 n 位的值



2、原码恢复余数算法



举例1

❖ 【例4.8】 $X=+0.1011$, $Y=-0.1101$
，用原码恢复余数算法计算 $X \div Y$ 。

❖ ① 写出原码和绝对值等

- $[X]_{\text{原}}=0.1011$ $[Y]_{\text{原}}=1.1101$
- $|X|=0.1011$ $|Y|=0.1101$
- $[-|Y|]_{\text{补}}=11.0011$
- $Q=0$ $Q_s=X_s \oplus Y_s=1$ $R_s=0$

❖ ② 列式计算：减/恢复余数、右移

❖ ③ 置Q和R的符号位：

得 $[Q]_{\text{原}}=1.1101$

$[R]_{\text{原}}=0.00000111$



被除数/余数	商Q	操作说明
00.1011	0 0 0 0 0	
+ 11.0011		$+[- Y]_{\text{补}}$
11.1110	0 0 0 0 0	$R_0 < 0$, 上商0
+ 00.1101		$+ Y $ 恢复余数
00.1011	0 0 0 0 0	
01.0110	0 0 0 0 0	左移一位
+ 11.0011		$+[- Y]_{\text{补}}$
00.1001	0 0 0 0 1	$R_1 > 0$, 上商1
01.0010	0 0 0 1 0	左移一位
+ 11.0011		$+[- Y]_{\text{补}}$
00.0101	0 0 0 1 1	$R_2 > 0$, 上商1
00.1010	0 0 1 1 0	左移一位
+ 11.0011		$+[- Y]_{\text{补}}$
11.1101	0 0 1 1 0	$R_3 < 0$, 上商0
+ 00.1101		$+ Y $ 恢复余数
00.1010	0 0 1 1 0	
01.0100	0 1 1 0 0	左移一位
+ 11.0011		$+[- Y]_{\text{补}}$
00.0111	0 1 1 0 1	$R_4 > 0$, 上商1

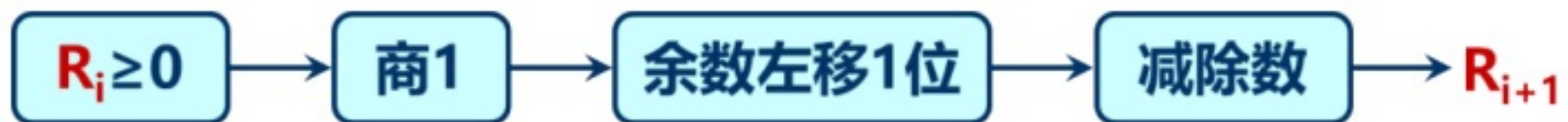


3、原码不恢复余数算法

- ❖ 又称为**加减交替法**：当不够减时，不恢复余数，用加上除数 ($+|Y|$) 的办法来继续求下一位商，其他操作不变。
- ❖ 加减交替法的规则如下：
 - **余数为正**：商上**1**，求下一位商的办法，是余数左移一位，再**减去除数**；
 - **余数为负**：商上**0**，求下一位商的办法，是余数左移一位，再**加上除数**。
 - 若最后一次上商为0，而又需得到正确余数，则最后一次仍需恢复余数。
- ❖ **Why?**



3、原码不恢复余数算法

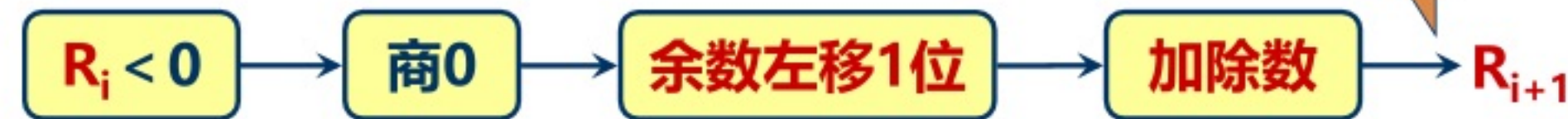


$$R_{i+1} = 2R_i - |Y|$$



余数左移1位

$$R_{i+1} = 2(R_i + |Y|) - |Y| = 2R_i + 2|Y| - |Y| = 2R_i + |Y|$$



举例2

❖ 【例4.9】 $X=+0.1011$, $Y=-0.1101$
，用原码不恢复余数算法计算 $X \div Y$ 。

❖ ① 写出原码和绝对值等

- $[X]_{\text{原}}=0.1011$ $[Y]_{\text{原}}=1.1101$
- $|X|=0.1011$ $|Y|=0.1101$
- $[-|Y|]_{\text{补}}=11.0011$
- $Q=0$ $Q_S=X_S \oplus Y_S=1$ $R_S=0$

❖ ② 列式计算：加/减、右移

❖ ③ 置Q和R的符号位：

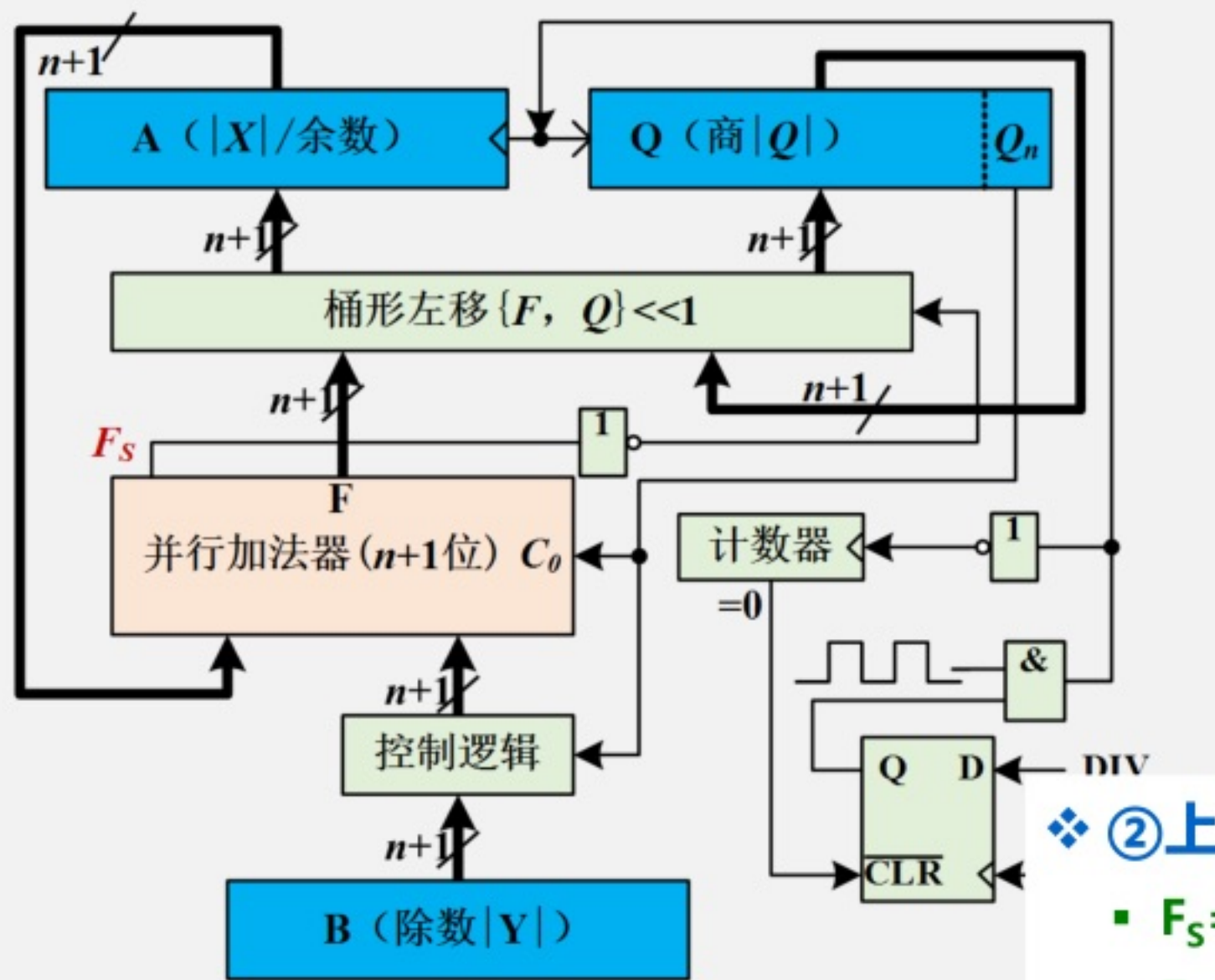
$$[Q]_{\text{原}}=1.1101$$

$$[R]_{\text{原}}=0.00000111$$

被除数/余数	商Q	操作说明
00.1011	0 0 0 0 0	
+ 11.0011		$+[- Y]_{\text{补}}$
11.1110	0 0 0 0 0	$R_0 < 0$, 上商0
11.1100	0 0 0 0 0	左移一位
+ 00.1101		$+ Y $
00.1001	0 0 0 0 1	$R_1 > 0$, 上商1
01.0010	0 0 0 1 0	左移一位
+ 11.0011		$+[- Y]_{\text{补}}$
00.0101	0 0 0 1 1	$R_2 > 0$, 上商1
00.1010	0 0 1 1 0	左移一位
+ 11.0011		$+[- Y]_{\text{补}}$
11.1101	0 0 1 1 0	$R_3 < 0$, 上商0
11.1010	0 1 1 0 0	左移一位
+ 00.1101		$+ Y $
00.0111	0 1 1 0 1	$R_4 > 0$, 上商1



4、原码除法的硬件实现



❖ ①如何控制加减交替?

❖ ②如何上商?

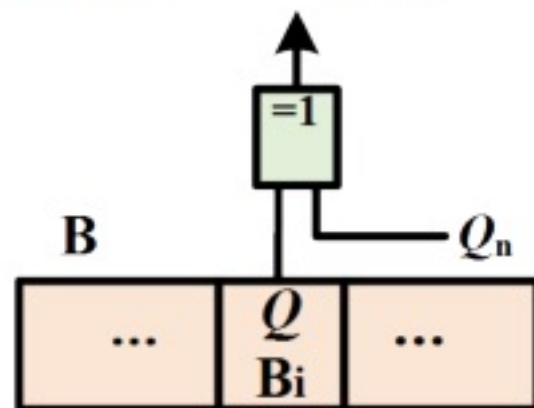
❖ ③加减与移位次数如何控制, 结果在哪里?

❖ ①加减交替: 控制逻辑

- $Q_n = 1$ 表明够减 → 下一次做减法 → B 中的|Y|取反 ($11...11 \oplus B$)
- $Q_n = 0$ 表明不够减 → 下一次做加法 → B 中的|Y|不取反 ($00...00 \oplus B$)

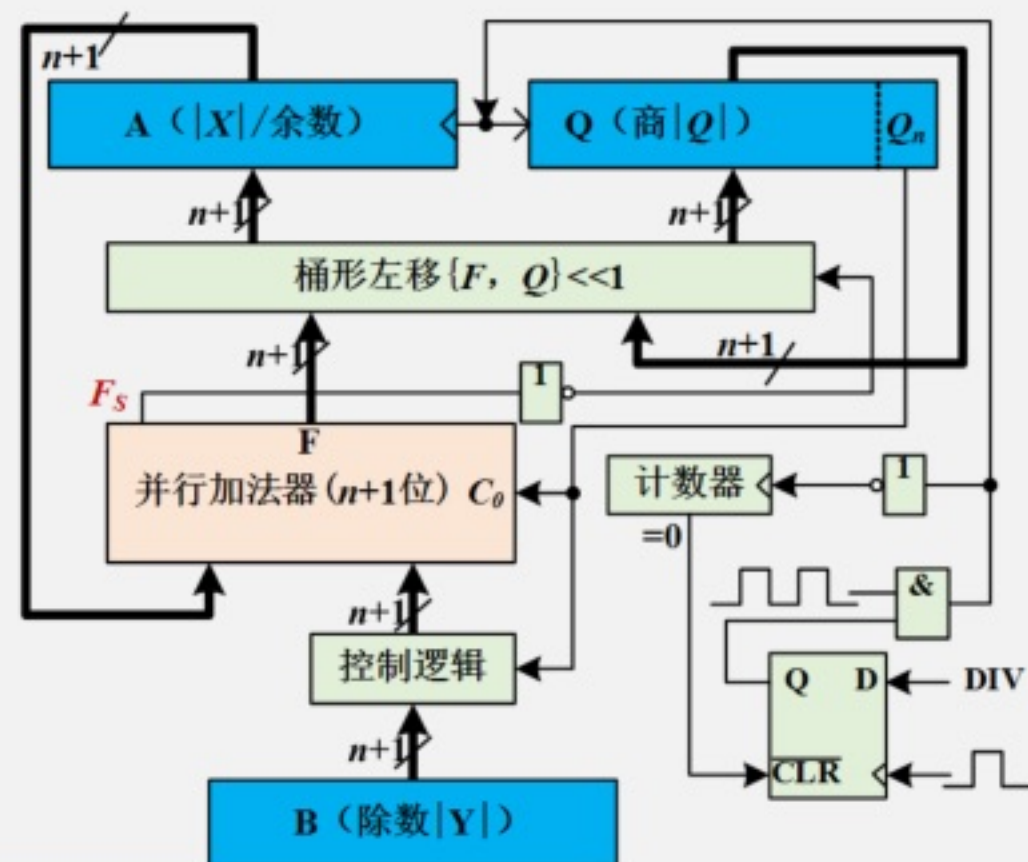
❖ ②上商: F_s 取反

- $F_s = 0$ 表明够减 → $Q_n = 1$
- $F_s = 1$ 表明不够减 → $Q_n = 0$



4、原码除法的硬件实现

- ❖ A: $n+1$ 位累加寄存器
- ❖ B: $n+1$ 位除数寄存器
- ❖ Q: $n+1$ 位商寄存器
- ❖ 桶形移位器: F、Q联合逻辑左移
 - $A \leftarrow \{F_1 \dots F_n, Q_0\}$ $Q \leftarrow \{Q_1 \dots Q_n, \overline{F_S}\}$



初始:

- ❖ $A = |X|$
- ❖ $B = |Y|$
- ❖ $Q = 00 \dots 01$
- ❖ 计数器 = $n+1$

加减、商 $n+1$ 次



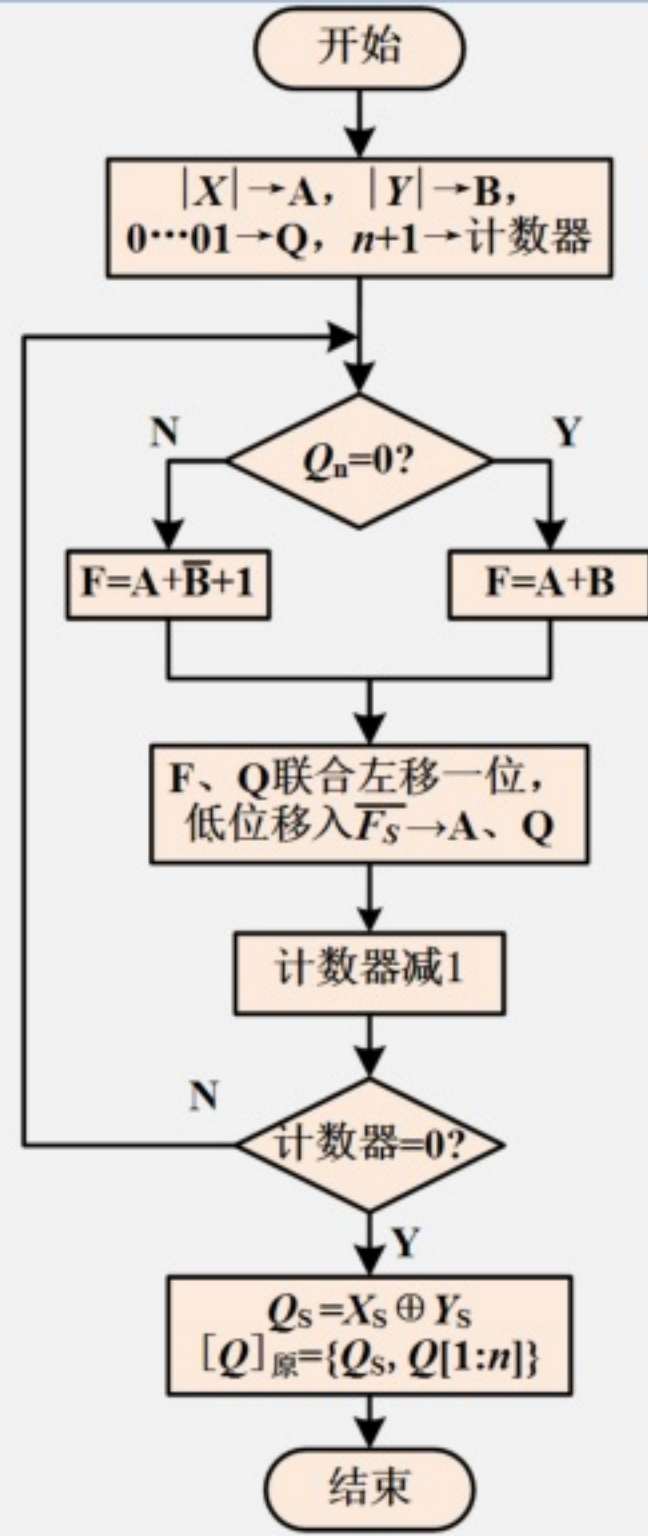
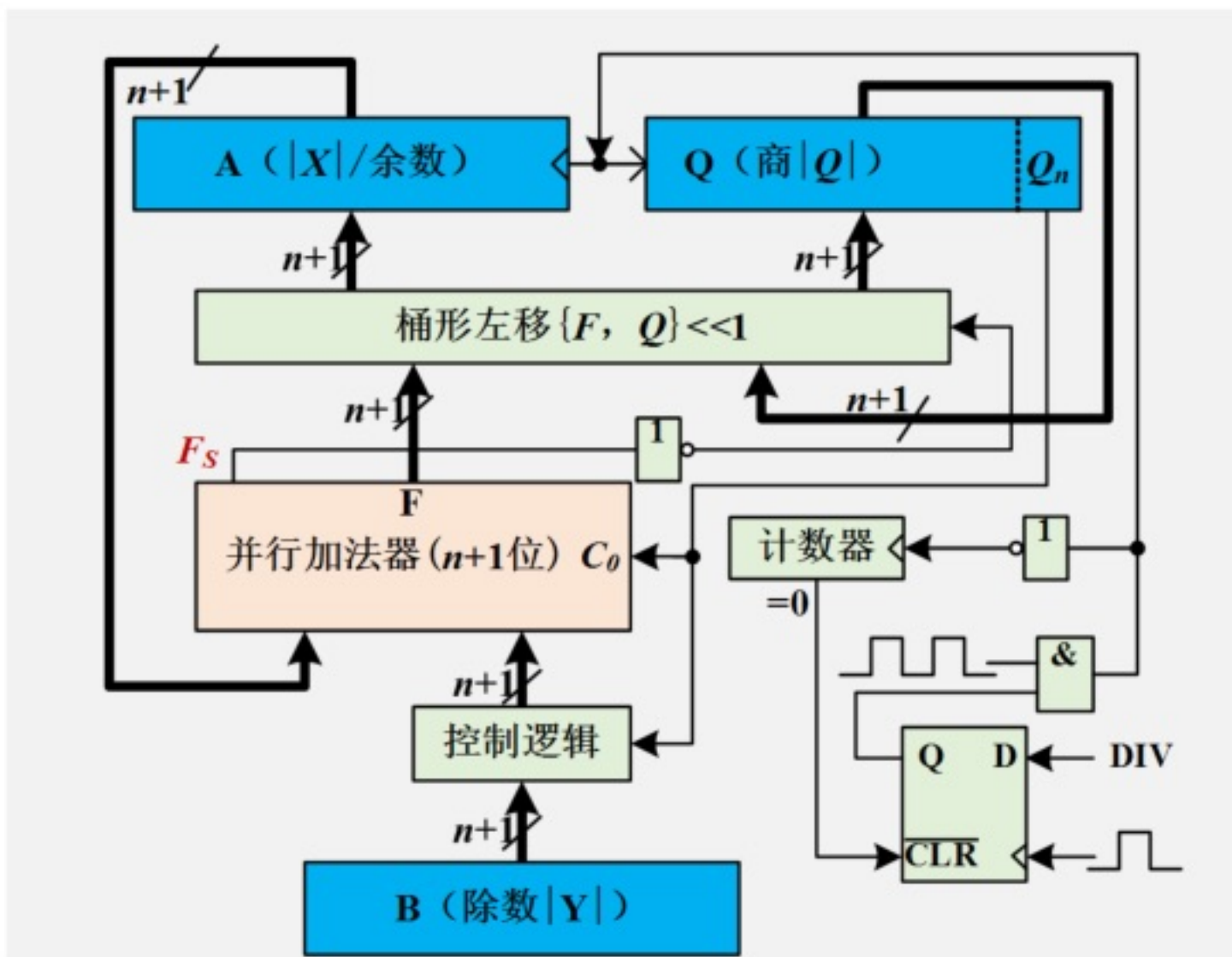
左移 n 次

结果:

- ❖ $A =$ 余数绝对值低位
- ❖ $B = |Y|$
- ❖ $Q =$ 商绝对值
- ❖ 计数器 = 0

4、原码除法的硬件实现

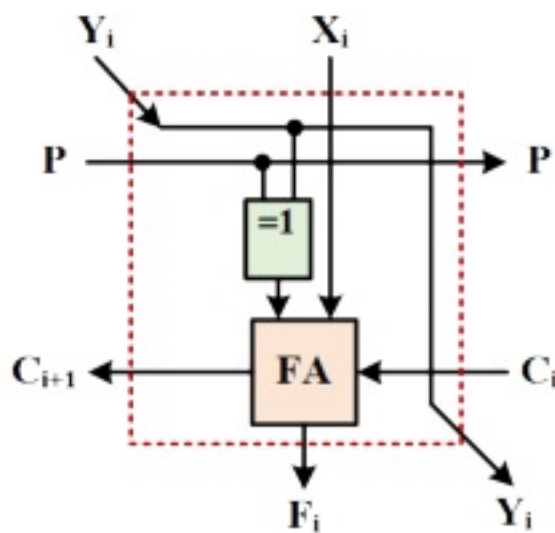
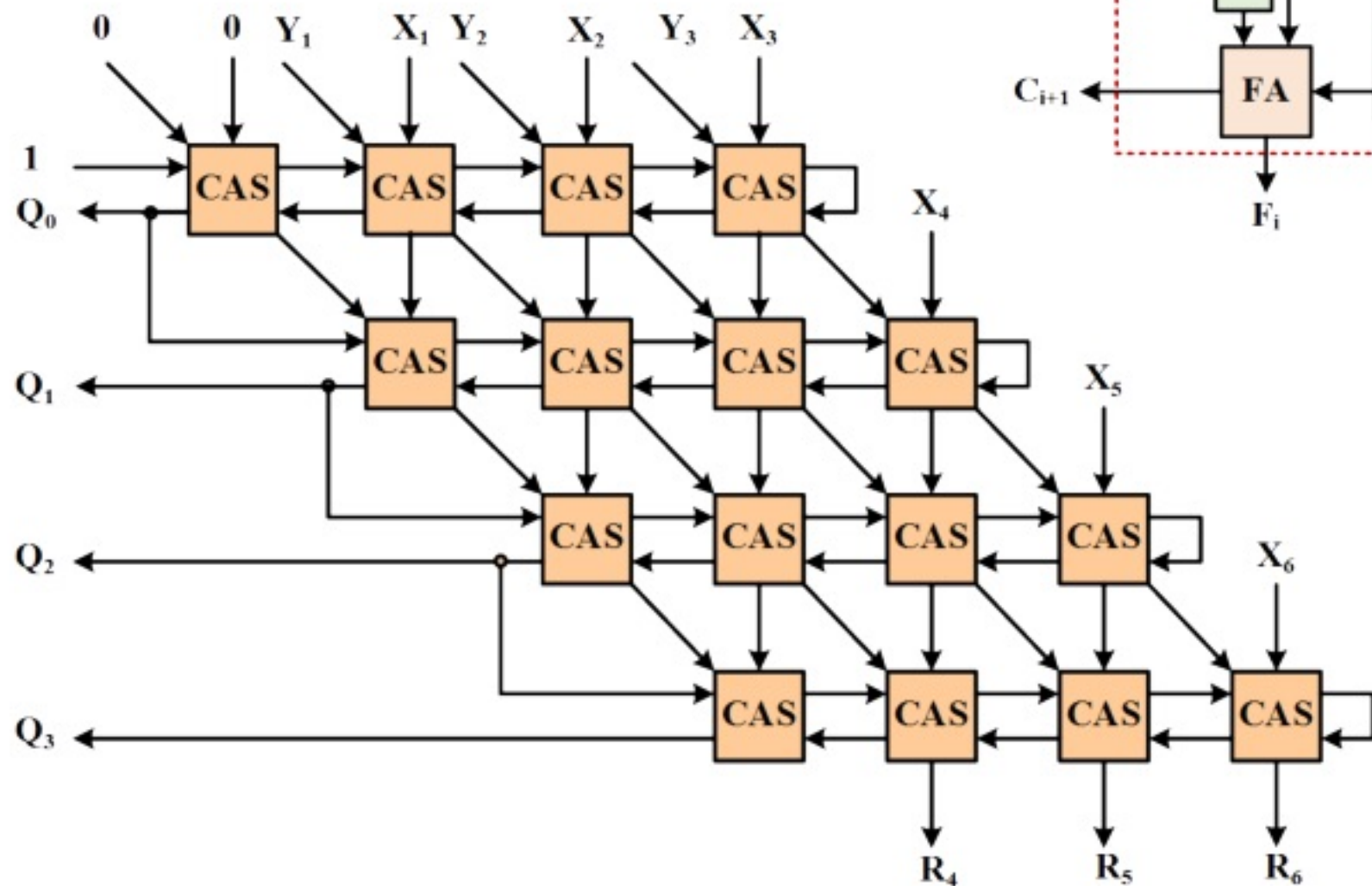
❖ 原码不恢复余数除法流程





4.4.2 阵列除法器

$X \div Y$



❖ 定点整数:

- $X = X_1 X_2 X_3 X_4 X_5 X_6$
- $Y = Y_1 Y_2 Y_3$
- 商 $Q = Q_1 Q_2 Q_3$ ($Q_0 = 0$)
- 余数 $R = R_4 R_5 R_6$

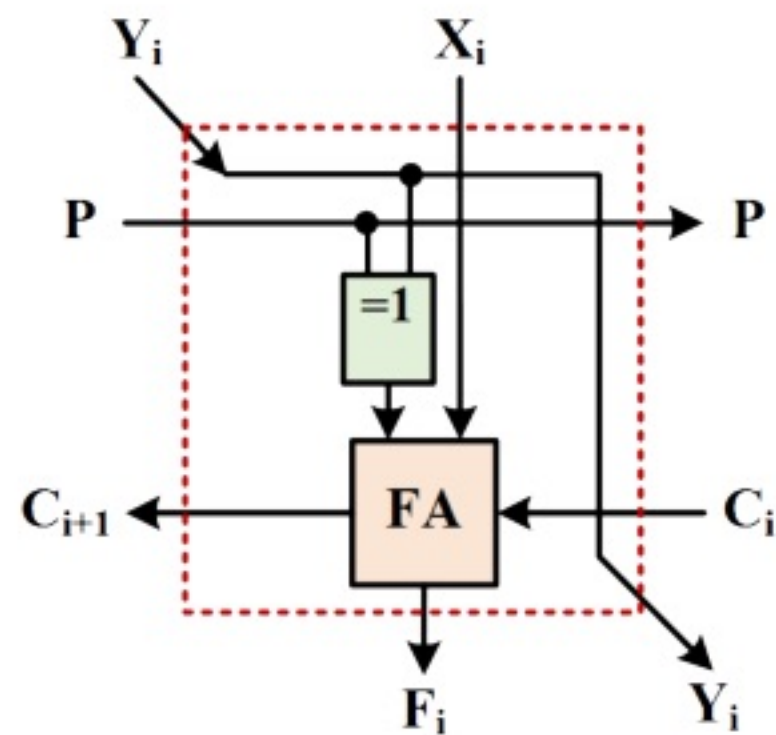
❖ 定点小数

- $X = 0.X_1 X_2 X_3 X_4 X_5 X_6$
- $Y = 0.Y_1 Y_2 Y_3$
- 商 $Q = 0.Q_1 Q_2 Q_3$ ($Q_0 = 0$)
- 余数 $R = 0.000 R_4 R_5 R_6$

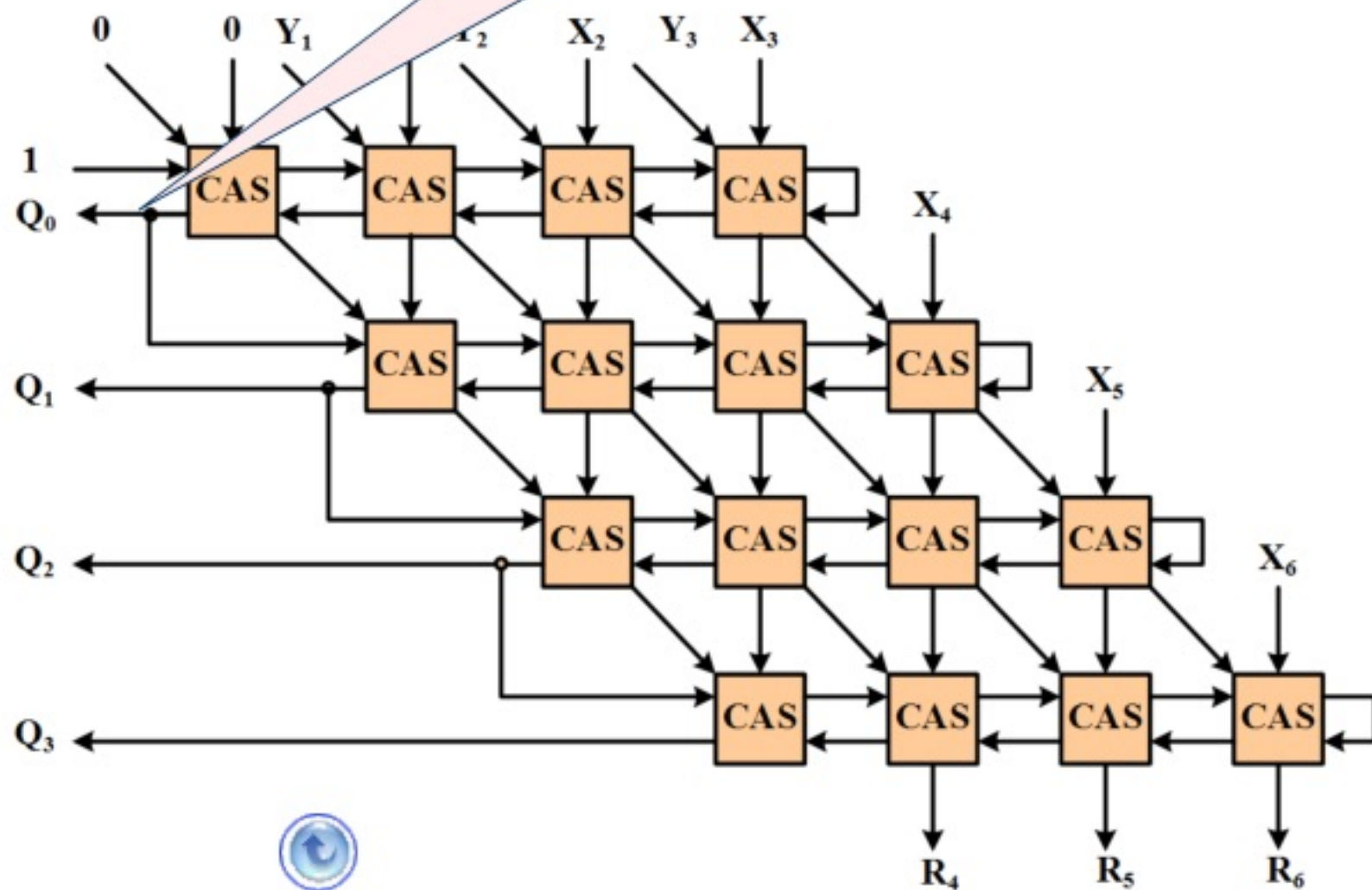


4.4.2 阵列除法器

- ❖ 构成的基本部件：**可控加减单元CAS**
- ❖ 算法：**加减交替算法**
- ❖ $P=1$ 做减法
- ❖ $P=0$ 做加法



$C=0$: 有借位, 商0, 下一次做加法, $P=0$
 $C=1$: 没有借位, 商1, 下一次做减法, $P=1$





4.5 定点运算器的组成与结构

- ❖ 4.5.1 定点运算器的组成
- ❖ 4.5.2 运算器结构与数据通路
- ❖ 4.5.3 多端口寄存器堆
- ❖ 4.5.4 标志寄存器





4.5.1 定点运算器的组成

❖ **运算器的基本功能**：在控制器的控制下，完成数据信息的**算术逻辑运算**

❖ **基本组成包括**：

Arithmetic & Logic Unit

▪ **算术逻辑运算单元ALU**：核心部件，实现算术运算和逻辑运算

▪ **暂存器**：用来存放参与计算的数据及运算结果，它只对硬件设计者可见，即只被控制器硬件逻辑控制或微程序所访问

General Purpose Registers

▪ **通用寄存器堆**：用于存放程序中用到的数据，它可以被软件设计者所访问。

GPR
GPRs
GR

▪ **内部总线**：用于连接各个部件的信息通道。

RF (Register File)

▪ **其他可选电路**

❖ **设计定点运算器**，如何确定各**部件的功能和组织方式**是关键，取决于：

- 指令系统
- 机器数及其运算原理
- 机器字长
- 体系结构（连接方式等）





❖1、总线连接的运算器结构

- (1) 单总线结构
- (2) 双总线结构

❖2、直接连接的运算器结构

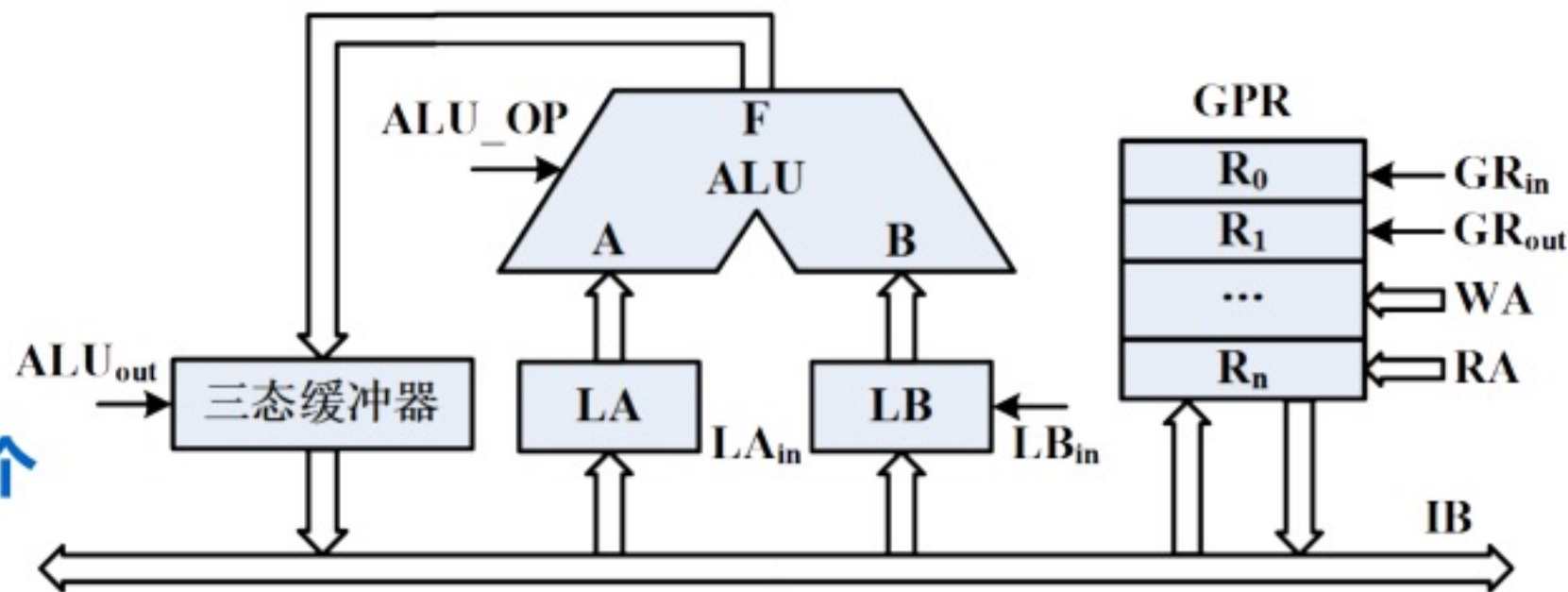




(1) 单总线结构

❖ 构成:

- 内部总线IB
- ALU
- 2个暂存器: LA和LB
- 通用寄存器堆GPR: $n+1$ 个

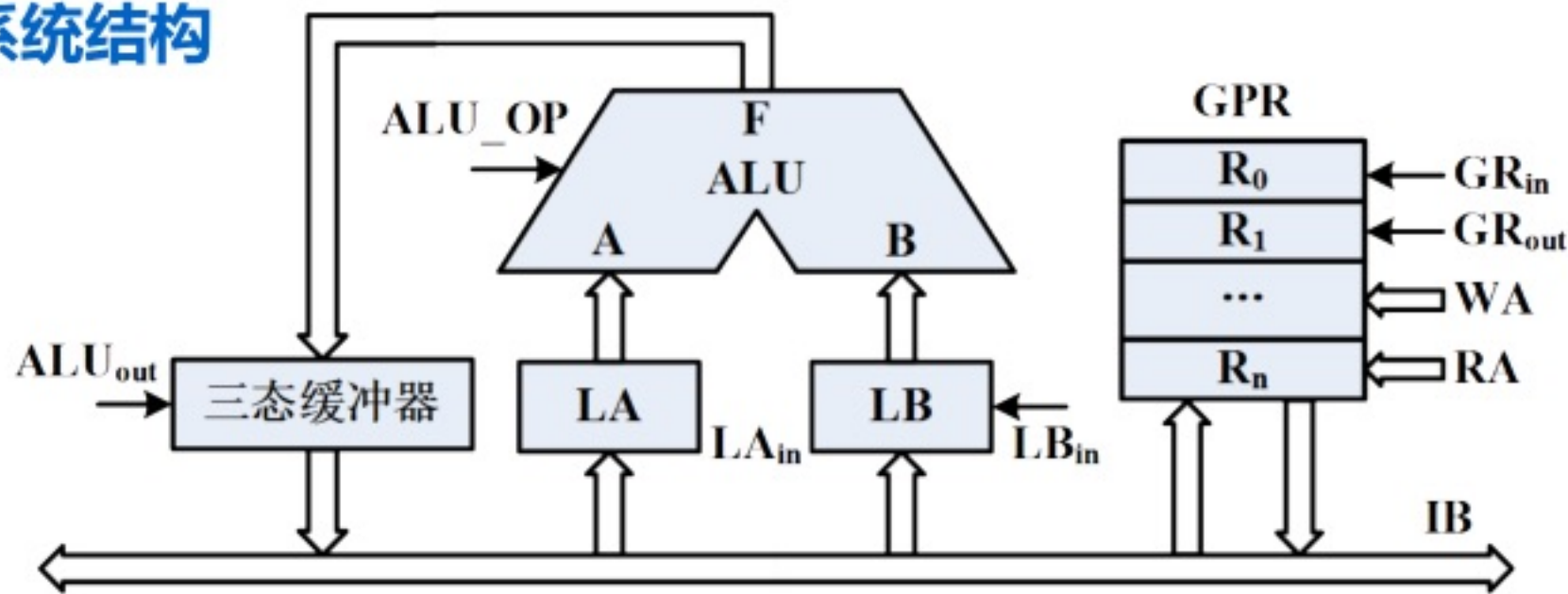


- ❖ $LA_{in}=1$: IB上数据写入LA
- ❖ $LB_{in}=1$: IB上数据写入LB
- ❖ ALU_OP : 功能码/运算码, 选择ALU执行的运算
- ❖ $ALU_{out}=1$: ALU的运算结果 (即输出端F) $\rightarrow$ IB
- ❖ RA : 执行读操作的寄存器地址
- ❖ WA : 执行写操作的寄存器地址
- ❖ $GR_{out}=1$: 读出RA指定的寄存器数据送到IB上 ($Reg[RA] \rightarrow IB$)
- ❖ $GR_{in}=1$: 将IB上的数据写入由写地址WA选中的寄存器 ($IB \rightarrow Reg[WA]$)



(1) 单总线结构

❖ 数据通路：指数据在硬件系统结构上流动的过程与控制步骤



❖ 在总线上传送数据的数据通路：需要一个CPU周期

- 源部件将数据送上总线；目的部件从总线上接收数据
- 任何要把数据送上总线的源部件，其数据输出端必须以三态输出的形式与总线连接，并由使能信号控制
- 对于某个总线系统来说，各个总线源部件的数据输出使能信号不能同时有效：任何时刻只能允许一个部件将数据送上总线，以保证分时享用总线



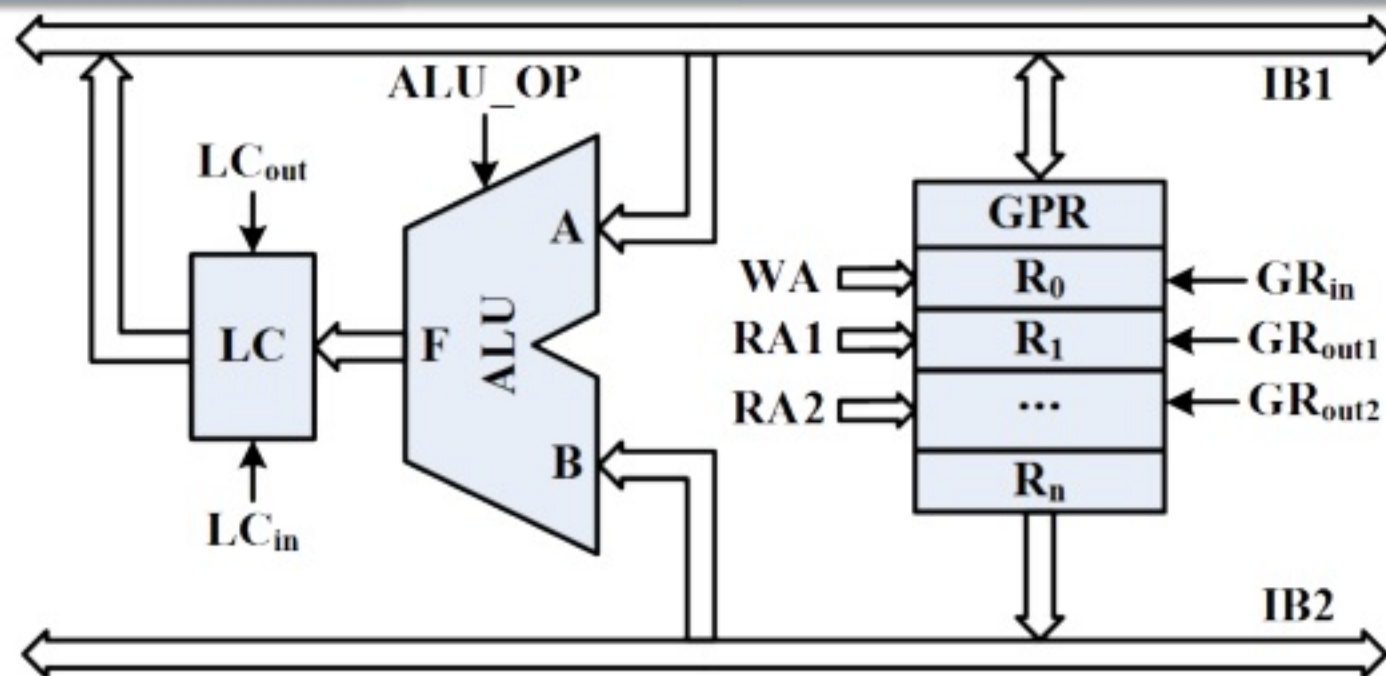
- ❖ 需要3个CPU周期



(2) 双总线结构

❖ 构成:

- 2条内部总线: IB1、IB2
- ALU
- 暂存器: LC
- 通用寄存器堆GPR: $n+1$ 个



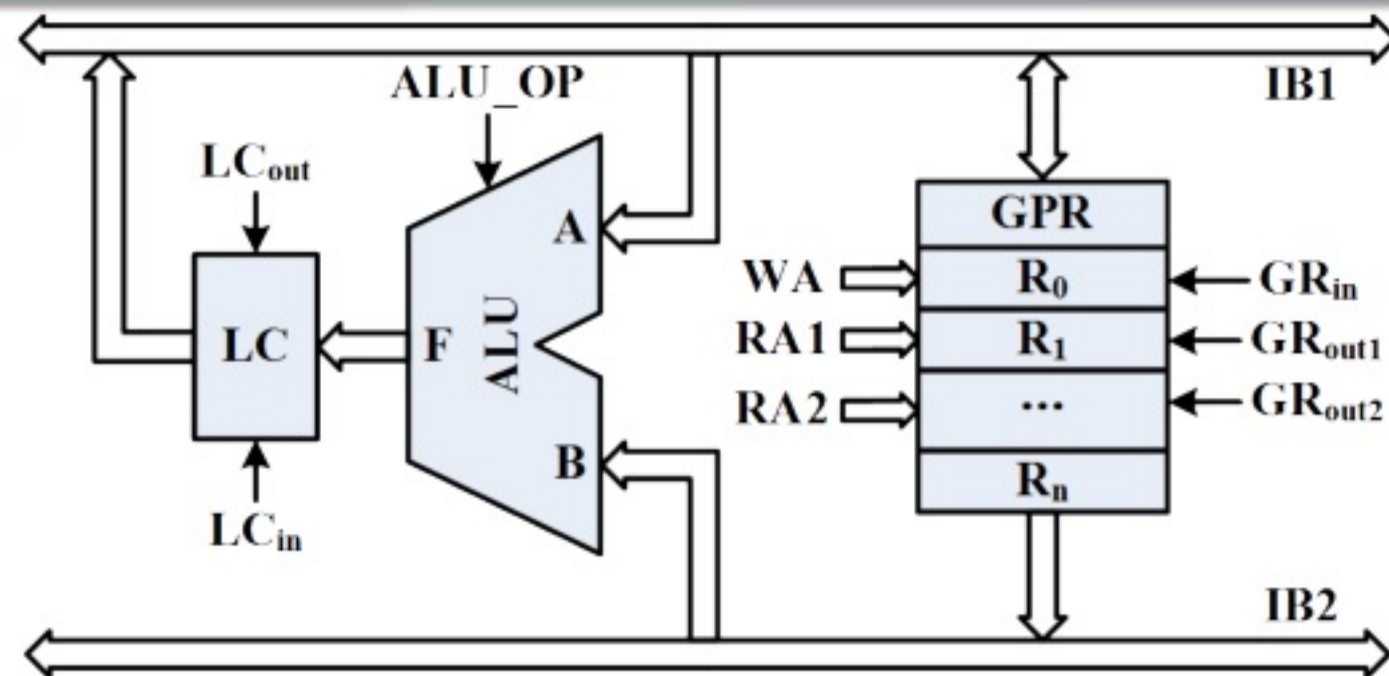
- ❖ $LC_{in}=1$: ALU运算结果写入LC, 即 $F \rightarrow LC$
- ❖ $LC_{out}=1$: LC中数据送上总线IB1, 即 $LC \rightarrow IB1$
- ❖ ALU_OP : ALU功能码/运算码
- ❖ $RA1$: 执行读出数据到IB1的寄存器地址
- ❖ $RA2$: 执行读出数据到IB2的寄存器地址
- ❖ WA : 执行写操作的寄存器地址

- ❖ $GR_{out1}=1$: 读出RA1指定的寄存器数据送到IB1上, 即 $Reg[RA1] \rightarrow IB1$
- ❖ $GR_{out2}=1$: 读出RA2指定的寄存器数据送到IB2上, 即 $Reg[RA2] \rightarrow IB2$
- ❖ $GR_{in}=1$: 将IB1上的数据写入由写地址WA选中的寄存器, 即 $IB1 \rightarrow Reg[WA]$

(2) 双总线结构

❖ 【例4.11】假设在双总线运算器中，同样完成指令ADD R₃, R₂, R₇的运算操作 $R_2 + R_7 \rightarrow R_3$ ，分析数据通路，需要几个CPU周期？

❖ 需要2个CPU周期



步骤	操作	发送的信号
1	$R_2 \rightarrow IB1, R_7 \rightarrow IB2, F = A + B, F \rightarrow LC$	$RA1 = 2, RA2 = 7, ALU_OP = \text{加法功能码}, GR_{out1} = 1, GR_{out2} = 1, LC_{in} = 1$
2	$LC \rightarrow R_3$	$LC_{out} = 1, GR_{in} = 1, WA = 3$

❖ 思考：如果LC暂存器输出端连接到IB2上，能否完成上述操作？

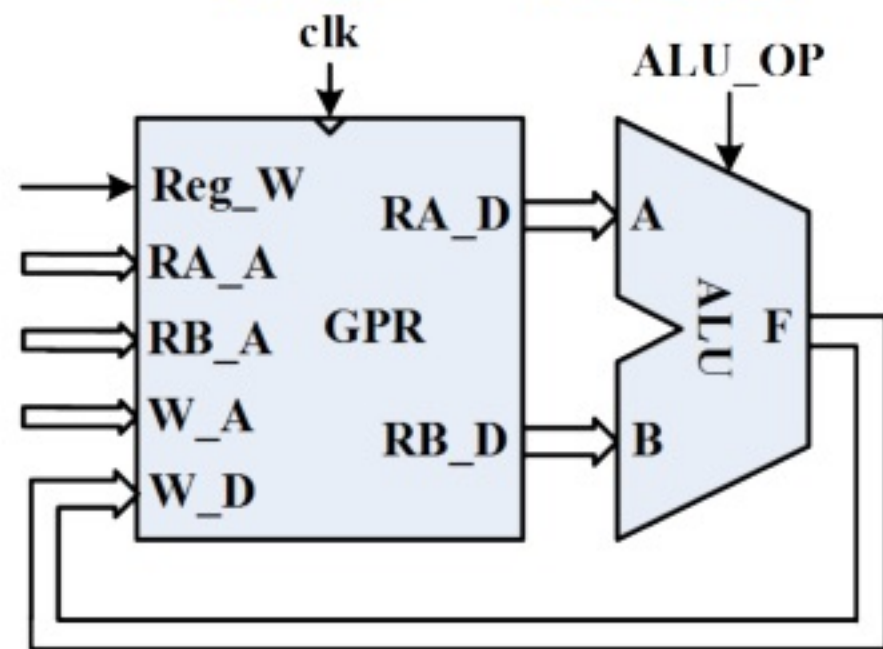
❖ 总结：在分析总线结构的运算器数据通路时，基本的原则：在一个CPU周期内，某条总线上的数据必须是唯一的，且不能保留至下一个CPU周期。



2、直接连接的运算器结构

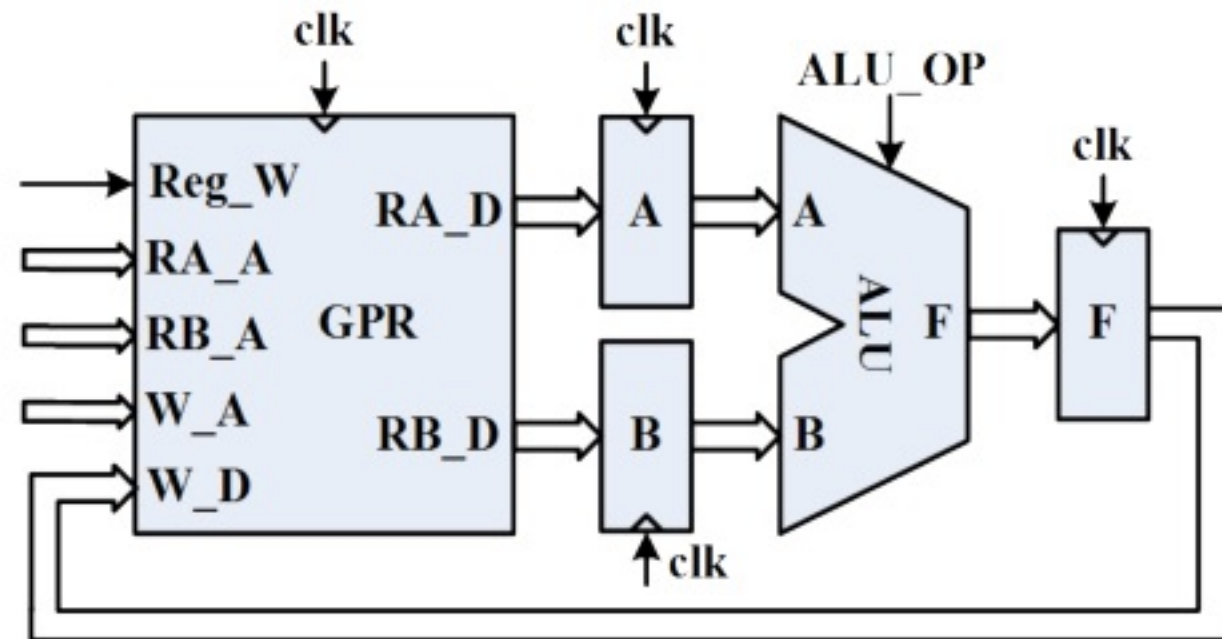
- ❖ 特征：按照运算操作的过程与控制需求，部件之间由专用的独立线路进行连接
- ❖ 特点：线路比较复杂，但是传输效率高，适用于**流水线CPU**

❖ 单周期CPU的运算器



- ❖ 每条指令只需一个CPU周期完成

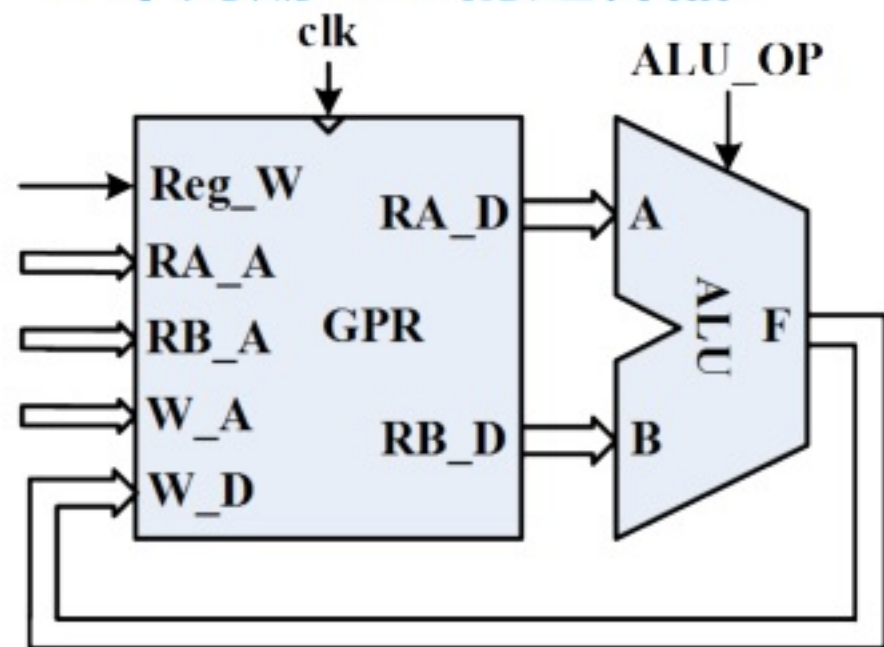
❖ 多周期CPU的运算器



- ❖ 每条指令需要多个CPU周期完成

2、直接连接的运算器结构

❖ 单周期CPU的运算器

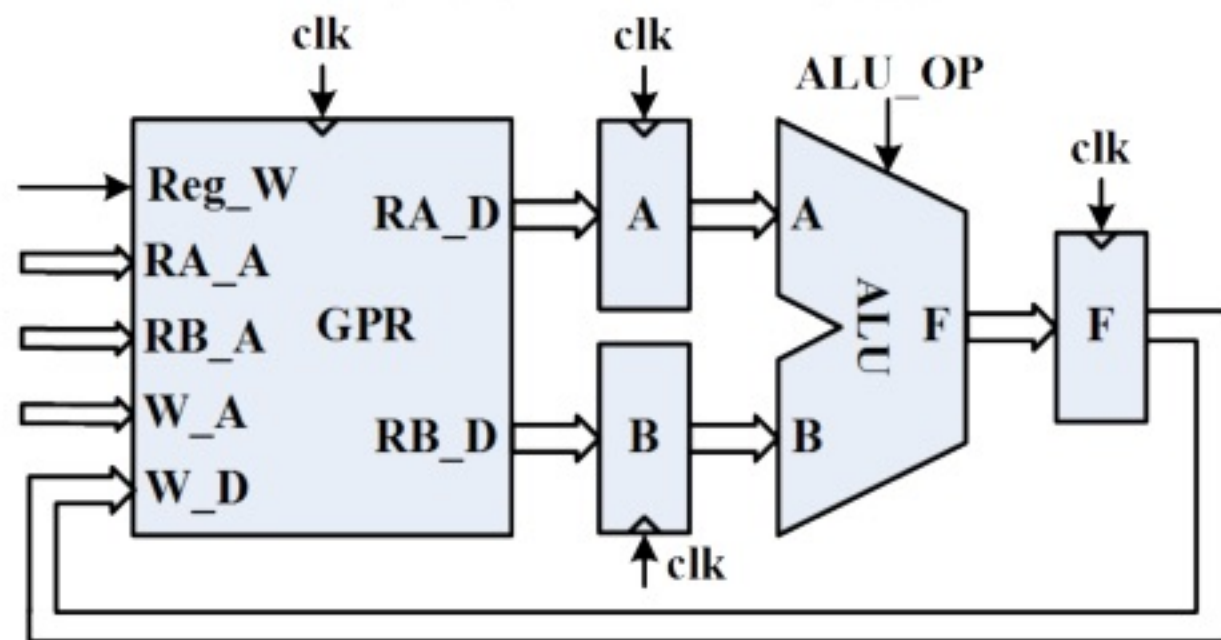


寄存器堆：
有3个端口

❖ 两个读端口A和B:

- 地址: RA_A、RA_B
- 读出的数据: RA_D、RB_D
- 读操作无需时钟, 即:
 - $RA_D = \text{Reg}[RA_A]$
 - $RB_D = \text{Reg}[RB_A]$

❖ 多周期CPU的运算器

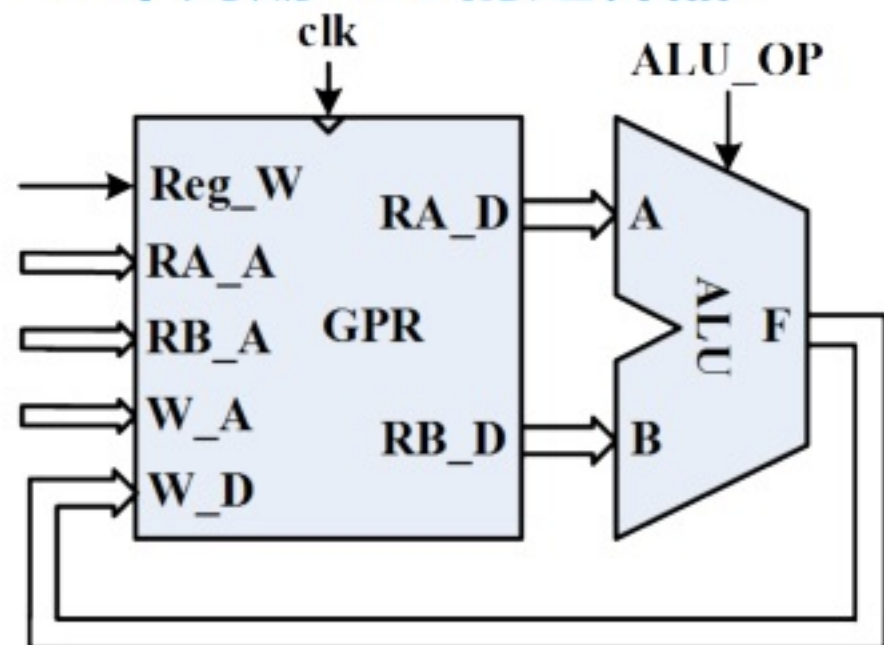


❖ 一个写端口W:

- 写地址: W_A
- 写入的数据: W_D
- 写信号Reg_W: 在clk来临时, 如果Reg_W=1, 则执行写入操作: $W_D \rightarrow \text{Reg}[W_A]$

2、直接连接的运算器结构

❖ 单周期CPU的运算器

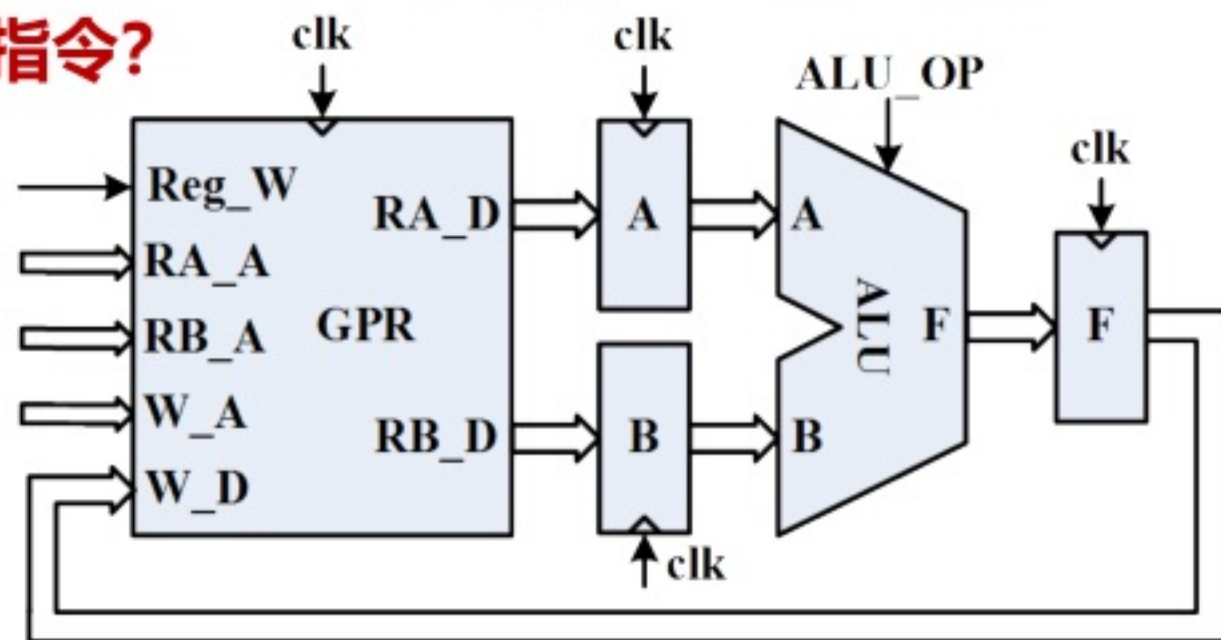


❖ 只需1个clk周期:

- $RA_A=2, RB_A=7, W_A=3$
- ALU_OP =加法功能码
- $Reg_W=1$, 在clk的边沿完成 $R_2+R_7 \rightarrow R_3$ 操作

❖ 执行ADD R_3, R_2, R_7 指令?

❖ 多周期CPU的运算器



❖ 需3个节拍 (clk周期):

- 第1个clk (读寄存器): $R_2 \rightarrow A, R_7 \rightarrow B$: $RA_A=2, RB_A=7$
- 第2个clk (执行): $A+B \rightarrow F$: ALU_OP =加法功能码
- 第3个clk (写回): $F \rightarrow R_3$: $W_A=3, Reg_W=1$





4.5.3 多端口寄存器堆

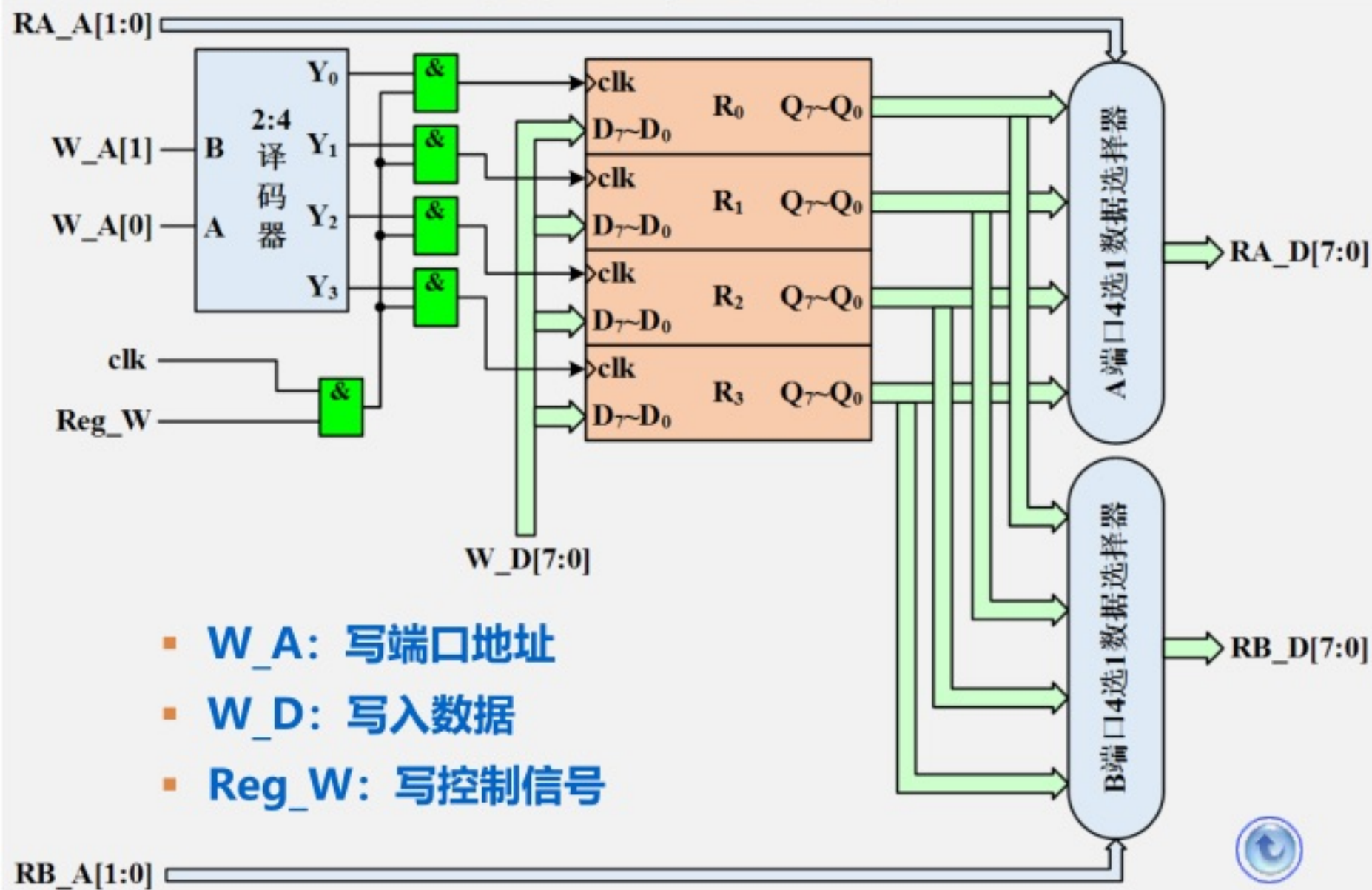
❖ 特征：能够对一组寄存器同时读出或者写入多个数据

❖ 三端口寄存器堆
可以同时进行

- 读A端口
- 读B端口
- 写入数据

❖ 寄存器堆：4×8位

- $R_0 \sim R_3$ ，地址2位
- RA_A ：A口地址
- RA_D ：A口读出数据
- RB_A ：B口地址
- RB_D ：B口读出数据



- W_A ：写端口地址
- W_D ：写入数据
- Reg_W ：写控制信号





4.5.4 标志寄存器

❖ 标志寄存器FR：又称为状态寄存器

PSW：程序状态字

- 用来保存ALU操作结果的某些状态，可作为外界对运算结果进行分析的依据，也可以用于判断程序是否要转移的条件

❖ 最基本的5种运算结果标志：

①ZF：结果为零标志

- 运算结果为全0，ZF置1
- 运算结果不全为0，ZF置0

$$ZF = \overline{F_n + F_{n-1} + \dots + F_0}$$

②CF：进位/借位标志位，CF标志只对无符号数运算有意义

- 加法运算时：C=1则CF置1（表示有进位），否则置0
- 减法运算时：C=0则CF置1（表示不够减，有借位），否则置0

最高位的进位

$$CF = \overline{ADD/SUB} \cdot C + \overline{ADD/SUB} \cdot \bar{C}$$



4.5.4 标志寄存器

③ **OF**: 溢出标志, 反映有符号数加减运算所得结果是否溢出; **OF**标志只对带符号数运算有意义。

- 运算溢出: $OF=1$
- 运算没有溢出: $OF=0$

$$OF = \overline{ADD/SUB}(\overline{X_S} \overline{Y_S} F_S + X_S Y_S \overline{F_S}) + \overline{ADD/SUB}(\overline{X_S} Y_S F_S + X_S \overline{Y_S} \overline{F_S})$$

$$OF = C_1 \oplus C_S$$

$$OF = F_{S1} \oplus F_{S2}$$

④ **SF**: 符号标志, 记录运算结果的符号。

- 为运算结果的最高位 $SF = F_n$
- 运算结果为正数时: $SF=0$; 为负数时: $SF=1$ 。(无溢出时才正确)

⑤ **PF**: 奇偶标志, 反映运算结果中“1”的个数的奇偶性

- 结果中“1”的个数为偶数: $PF=1$
- 结果中“1”的个数为奇数: $PF=0$

$$PF = \overline{F_n \oplus F_{n-1} \oplus \dots \oplus F_0}$$



4.5.4 标志寄存器

- ❖ ①不同的CPU：设置FR及其标志的方法不同（有/没有？有哪些？）
 - MIPS、RISC-V CPU：不设置程序员可见的标志寄存器
 - x86、ARM CPU：设置程序员可访问的标志寄存器，分别是FLAG和CPSR寄存器
 - FR包含的标志也不尽相同：ARM中有N、Z、C、V四种标志，对应SF、ZF、CF和OF；
 - x86 CPU的标志要复杂得多
- ❖ ②不同CPU对标志的解释可能不一致
 - X86 CPU：对CF标志的解释和前述一致
 - ARM CPU：对CF标志的解释不同——标志C就是硬件产生的最高位进位C
- ❖ ③每条指令执行完成后，是否要修改FR的各个标志值？
 - 传送类指令：不影响各标志位，即不修改标志寄存器各位的值
 - 运算类指令：一般会影响标志位，但是不同的运算指令影响的标志位会有不同
- ❖ ④CPU并不知道一个数值是有符号的还是无符号的；CPU在运算操作之后，会机械地设置各种状态标志；由程序员自己来选择使用哪些标志和忽略哪些标志



4.5.4 标志寄存器

- ❖ 【例4.12】下面两条Intel x86指令将80H加80H后送到AL，试写出运算结果及其标志位。

```
MOV AL, 80H; 80H→AL  
ADD AL, 80H; AL+80H→AL
```

$$\begin{array}{r} 1000\ 0000 \\ +\ 1000\ 0000 \\ \hline 0000\ 0000 \\ \text{C}=1 \end{array}$$

- ❖ 运算结果为：AL=00H
- ❖ ZF=1
- ❖ CF=1
- ❖ OF=1
- ❖ SF=0
- ❖ PF=1

- ❖ 假设上面两条指令后用JC L1指令（等价于JB指令）发生跳转，则表明程序员想处理无符号数



4.5.4 标志寄存器

- ❖ 【例4.13】某32位模型计算机X上，有一个C语言编写的程序片段如下：
- 假设初始 $a[5] = 0x7FFF\ FFFF$ ， $b[5] = 0xFFFF\ FFFF$ ，则：
 - (1) 执行完上述程序后， $b[5]$ 的结果是多少？请以16进制表示，写出推导或计算过程。
 - (2) $\text{if } (b[i] < a[i])$ 这条判断语句，在机器指令与硬件上是通过 $b[i] - a[i]$ 来实现的；那么执行 $b[5] - a[5]$ 后，标志位CF、OF、ZF、SF的值是多少？

```
int i,a[10],b[10];
for (i=0;i<10;i++)
    if (b[i]<a[i]) b[i]=a[i]+20;
```

❖ (1) $b[5]$ 的结果是多少？

- ①确认 $b[5] < a[5]$ 是否为真？
 - 数组定义为int类型：补码表示的**有符号数**
 - $b[5]$ 补码是全1，真值是-1； $a[5]$ 是最大正数
 - 比较结果：真
- ②执行 $b[5] = a[5] + 20 = 0x8000\ 0013$

	0111	1111	1111	1111	1111	1111	1111	1111
+	0000	0000	0000	0000	0000	0000	0001	0100
+	1000	0000	0000	0000	0000	0000	0001	0011



4.5.4 标志寄存器

❖ 【例4.13】某32位模型计算机X上，有一个C语言编写的程序片段如下：

- 假设初始 $a[5] = 0x7FFF\ FFFF$ ， $b[5] = 0xFFFF\ FFFF$ ，则：
- (2) if ($b[i] < a[i]$)这条判断语句，在机器指令与硬件上是通过 $b[i] - a[i]$ 来实现的；那么执行 $b[5] - a[5]$ 后，标志位CF、OF、ZF、SF的值是多少？

```
int i,a[10],b[10];  
for (i=0;i<10;i++)  
    if (b[i]<a[i]) b[i]=a[i]+20;
```

❖ (2) $b[5] - a[5] = b[5] + [-a[5]]_{\text{补}}$

- $[-a[5]]_{\text{补}} = 0x8000\ 0001$
- ZF=0：结果=0x80000000，不全为0
- CF=0：做减法，C=0
- OF=0：C=1，次高位有进位
- SF=1：最高位=1

	1111	1111	1111	1111	1111	1111	1111	1111	1111
+	1000	0000	0000	0000	0000	0000	0000	0000	0001
+	1000	0000	0000	0000	0000	0000	0000	0000	0000





4.6 浮点运算及运算器

❖ 浮点数是由定点机器数组合表示的，浮点运算是在定点机器数基础上实现的

$$❖ X = M_x \times 2^{E_x}$$

$$❖ Y = M_y \times 2^{E_y}$$

	运算	预处理		阶码	尾数	结果
❖ $X+Y$?	加法	0操作数检查	对阶	-	加法	规格化、舍入
❖ $X-Y$?	减法	0操作数检查	对阶	-	减法	规格化、舍入
❖ $X*Y$?	乘法	0操作数检查	-	加法	乘法	规格化、舍入
❖ $X\div Y$?	除法	0操作数检查	-	减法	除法	规格化、舍入



4.6 浮点运算及运算器

❖ 浮点数运算:

- 参加运算的数据: 规格化
- 运算的结果: 转换为规格化的

❖ 4.6.1 浮点加减运算

❖ 4.6.2 浮点乘法运算

❖ 4.6.3 浮点除法运算

❖ 4.6.4 浮点运算器结构





4.6.1 浮点加减运算

- ❖ 假设手工计算 $9.97 \times 10^1 + 4.53 \times 10^{-1}$
- ❖ 第1步：保证它们的指数是相同的（小数点对齐）
- ❖ 第2步：对有效数位做加减运算

算法一：指数对齐 10^1

$$\begin{aligned} & 9.97 \times 10^1 + 4.53 \times 10^{-1} \\ &= 9.97 \times 10^1 + 0.0453 \times 10^1 \\ &= (9.97 + 0.0453) \times 10^1 \\ &= 10.0153 \times 10^1 \\ &= 1.00153 \times 10^2 \end{aligned}$$

算法二：指数对齐 10^{-1}

$$\begin{aligned} & 9.97 \times 10^1 + 4.53 \times 10^{-1} \\ &= 997 \times 10^{-1} + 4.53 \times 10^{-1} \\ &= (997 + 4.53) \times 10^{-1} \\ &= 1001.53 \times 10^{-1} \\ &= 1.00153 \times 10^2 \end{aligned}$$

解释

小数点对齐，使指数一样
有效数值部分相加
相加结果为非规格化
把结果变为规格化表示

- ❖ 结果一致，前提是：手工计算不受表示范围的约束
- ❖ 实际上：数据存储和计算都要受到计算机物理部件的处理位数的限制，超出部分全部丢失



4.6.1 浮点加减运算

❖ 假设：浮点数的尾数部分有效数值只能保存3位（整数位1位，小数位2位）

算法一：指数对齐 10^1

$$\begin{aligned} & 9.97 \times 10^1 + 4.53 \times 10^{-1} \\ &= 9.97 \times 10^1 + 0.04 \times 10^1 \\ &= (9.97 + 0.04) \times 10^1 \\ &= 10.01 \times 10^1 \\ &= 1.001 \times 10^2 \\ &= 1.00 \times 10^2 \end{aligned}$$

算法二：指数对齐 10^{-1}

$$\begin{aligned} & 9.97 \times 10^1 + 4.53 \times 10^{-1} \\ &= 7.00 \times 10^{-1} + 4.53 \times 10^{-1} \\ &= (7.00 + 4.53) \times 10^{-1} \\ &= 11.53 \times 10^{-1} \\ &= 1.153 \times 10^0 \\ &= 1.15 \times 10^0 \end{aligned}$$

解释

小数点对齐，使指数一样
有效数值部分相加
相加结果为非规格化
把结果变为规格化表示
对结果进行舍入

❖ 结果不一致：差别很大，算法一更为精确

❖ 原因：有效数值部分9.97进行了左移操作，使得有效数值的高位数字全部舍弃了，从而导致计算结果偏差较大

❖ 处理：选择误差小的那一种方法（算法一：指数对齐大的那个）



4.6.1 浮点加减运算

❖ 假设两个浮点数X和Y

$$X = M_X \times 2^{E_X}$$

$$Y = M_Y \times 2^{E_Y}$$

❖ 对齐小数点：使得阶码小的那个数变得与阶码大的那个数的阶码相等

❖ 然后：对尾数（有效数位）做加减运算

$$Z = X \pm Y = (M_X \cdot 2^{(E_X - E_Y)} \pm M_Y) \times 2^{E_Y} \quad E_X \leq E_Y$$



浮点加减运算步骤

(1) 0操作数检查：以尽可能的简化操作

(2) 对阶：

❖ 目标是：对齐小数点

❖ 原则是：小阶对大阶

- 求阶差 $\Delta E = E_X - E_Y$ ，若 $\Delta E \neq 0$ ，即 $E_X \neq E_Y$ 时需要对阶。
- 若 $\Delta E > 0$ ，则 $E_X > E_Y$ ， M_Y 每右移一位， $E_Y + 1$ ，直至 $E_Y = E_X$
- 若 $\Delta E < 0$ ，则 $E_X < E_Y$ ， M_X 每右移一位， $E_X + 1$ ，直至 $E_X = E_Y$

(3) 尾数相加减

- 若做减法，先将操作数Y的尾数取相反数，再与Y的尾数相加
- 对阶时，右移多出的数位不丢弃（因为一般设置位数更宽的ALU进行计算，同时设置更宽的寄存器来存放中间运算结果）



浮点加减运算步骤

(4) **结果规格化**: 尾数运算的结果可能出现两种**非规格化**情况:

- **尾数溢出**: 需要**右规 (1次)**, 即尾数右移1位, 阶码 + 1
- **|尾数| < 2⁻¹**: 需要**左规**, 即尾数左移1位, 阶码 - 1, 左规可能多次, 直到尾数变为规格化形式。

有前
导零

- 先判断是否溢出, 如果没有, 再判断是否有前导零

(5) **舍入**: 将尾数多余的位数舍去, 变成规定的浮点数格式

- **舍入策略**: 舍去多余位数的方法, **简单的舍入策略**:
 - 截断法: 多余的全舍去
 - 0舍1入法: 多余位的最高位为1时, 在尾数最低位上+1, 否则舍去多余位
 - 末位恒置1: 始终在舍入结果的最末位置1



IEEE 754标准的舍入模式

❖ IEEE 754标准：规定了4种可选的舍入模式

- ① **向上舍入** (总是朝 $+\infty$) : 为正数时, 只要多余位不全为0, 就向最低有效位进1; 为负数时, 则采用简单的截断法
- ② **向下舍入** (总是朝 $-\infty$) : 为正数时, 只要多余位不全为0, 就简单地截尾; 为负数时, 则向最低有效位进1
- ③ **向0舍入**: 即朝数轴的原点方向舍入, 就是无论正数还是负数, 都采用简单的截尾, 从而使得绝对值总是变小。这种方法容易累积误差
- ④ **就近舍入**: 即舍入到最接近的数, “四舍五入”。多余位 $= R_1 R_2 \dots R_n$
 - 当 $R_1=0$: 截尾 (即舍去);
 - 当 $R_1=1$, 且 $R_2 \dots R_n$ 不全为0: 向最低有效位进1;
 - 当 $R_1 R_2 \dots R_n = 10 \dots 0$: 即等于一半 (中点), 则若最低有效位为0就截尾, 若最低有效位为1就进1, 以使得最低有效位总是为0

❖ 中间结果设置了: 保护位(guard)、舍入位(round)、粘滞位(sticky)



浮点数计算的精度

- ❖ 浮点数不能精确表示每一个实数
- ❖ 运算时需要舍入处理，再次损失精度
- ❖ 原因：浮点除法运算上的bug
 - 浮点数有效位的第12位到52位可能出错
- ❖ 事件过程：
 - 1994年9月：数学教授Thomas Nicely发现了这个bug
 - 向Intel的技术支持部门询问未果，将这个bug发布到网上
 - Intel回应：声称这个bug只影响理论数学家，用户平均27000年才会碰见1次这样的错误
 - IBM研究院反驳：声称平均每24天电子表格用户就会遇到这样一次错误
 - 12月21日：Intel发布了道歉声明，并召回了有错误的芯片
 - “Intel相信技术上的一个极小错误已经开始了它的影响”
- ❖ 计算机算术运算：对精度和范围要有足够的重视，否则一个极小的错误就有可能导致严重的后果

❖ 1994年，Intel Pentium处理器

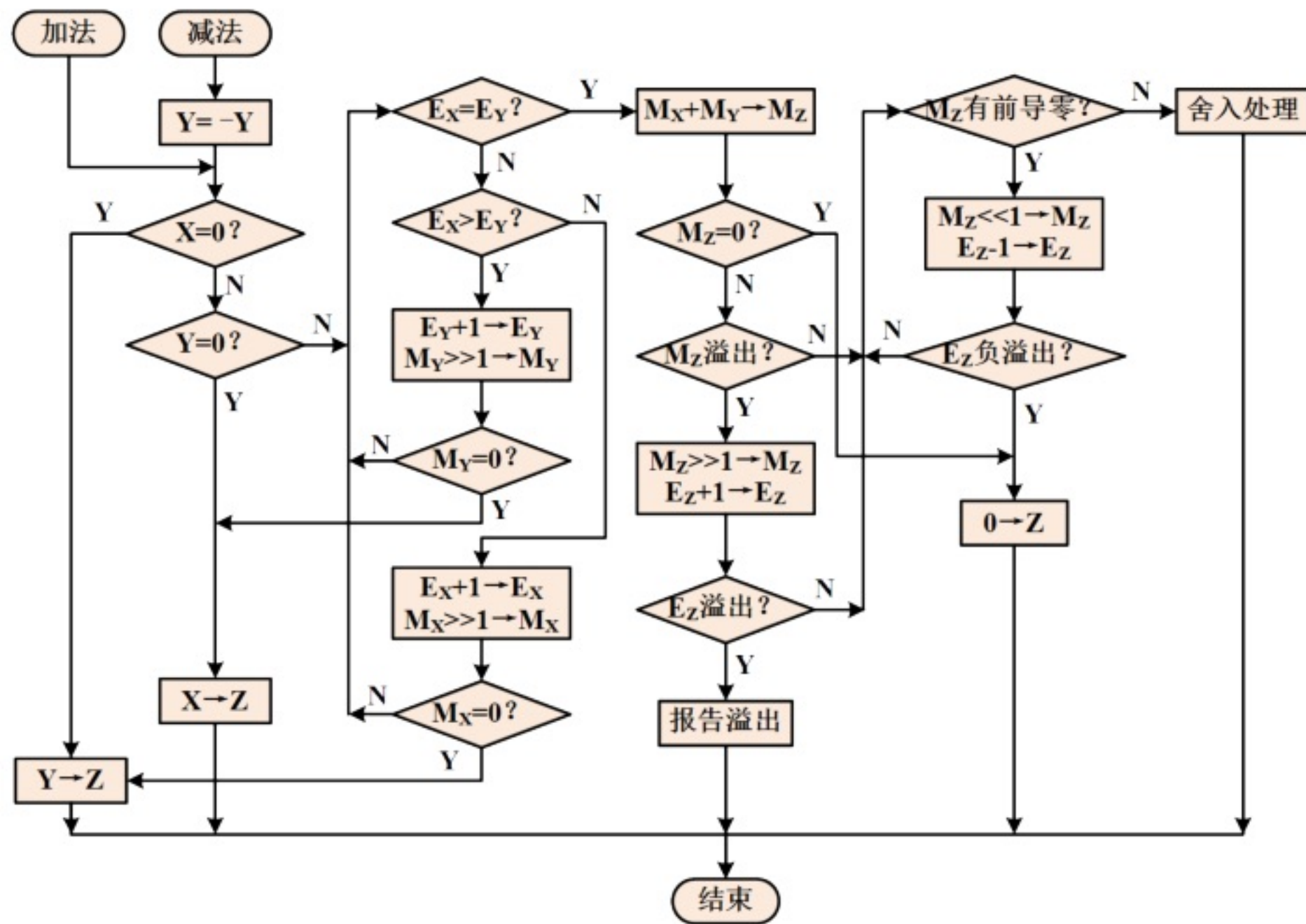
❖ 大规模召回事件

❖ 损失约5亿美元





浮点加减运算流程





浮点加减运算例题

- ❖ 【例4.14】：某浮点数格式为：12位浮点数，阶码4位（含阶符），尾数8位（含数符），用补码表示，阶码在前，尾数（含数符）在后，已知X和Y如下，采用0舍1入法舍入，求 $Z=X+Y$ 。

$$X = -0.1001011 \times 2^{001} \quad Y = 0.1100101 \times 2^{-010}$$

- ❖ 按照格式先写出它们的规格化浮点数

- ❖ $[X]_{\text{浮}} = 0, 001 \quad 1.0110101$

$$[Y]_{\text{浮}} = 1, 110 \quad 0.1100101$$

(1) 对阶

- $\Delta E = E_X - E_Y = [E_X]_{\text{补}} + [-E_Y]_{\text{补}} = 00, 001 + 00, 010 = 00, 011$
- $\Delta E = 3 > 0$ ，将 M_Y 右移3位， E_Y 加3：
- $[Y]_{\text{浮}} = 0, 001 \quad 0.0001100 \quad (101)$



浮点加减运算例题

(2) 尾数相加:

$$[M_Z]_{\text{补}} = 1.1000001 \quad (101)$$

$$\begin{array}{r} [M_X]_{\text{补}} \quad 11.0110101 \\ + [M_Y]_{\text{补}} \quad 00.0001100(101) \\ \hline [M_{X+Y}]_{\text{补}} \quad 11.1000001(101) \end{array}$$

(3) 结果规格化:

❖ 首先判断: 是否溢出?

- 因为双符号位为11, 没有溢出

❖ 再检查: 有无前导零?

- 因为 $[M_Z]_{\text{补}}$ 符号位和最高有效位相同=1.1, 故有一个前导零, 须左规一位

❖ 左规一位: 尾数左移1位, 阶码+1

- $[M_Z]_{\text{补}} = 1.0000011 \quad (01)$
- $[E_Z]_{\text{补}} = 00, 001 + 11, 111 = 00, 000$

(4) 舍入: 按照0舍1入法, 尾数多余位舍去

$$\text{结果为: } [X+Y]_{\text{浮}} = 0, 000 \quad 1.0000011$$





4.6.2 浮点乘法运算

❖ 假设两个浮点数X和Y:

$$X = M_X \times 2^{E_X}$$

$$Y = M_Y \times 2^{E_Y}$$

$$Z = X \times Y = (M_X \cdot M_Y) \times 2^{(E_X + E_Y)}$$

❖ 浮点乘法运算步骤

❖ (1) 0操作数检查

❖ (2) 阶码相加: 可采用补码/移码做加法, 同时对结果判溢

- 发生正溢出: 需报告溢出
- 发生负溢出: 将结果置为机器零

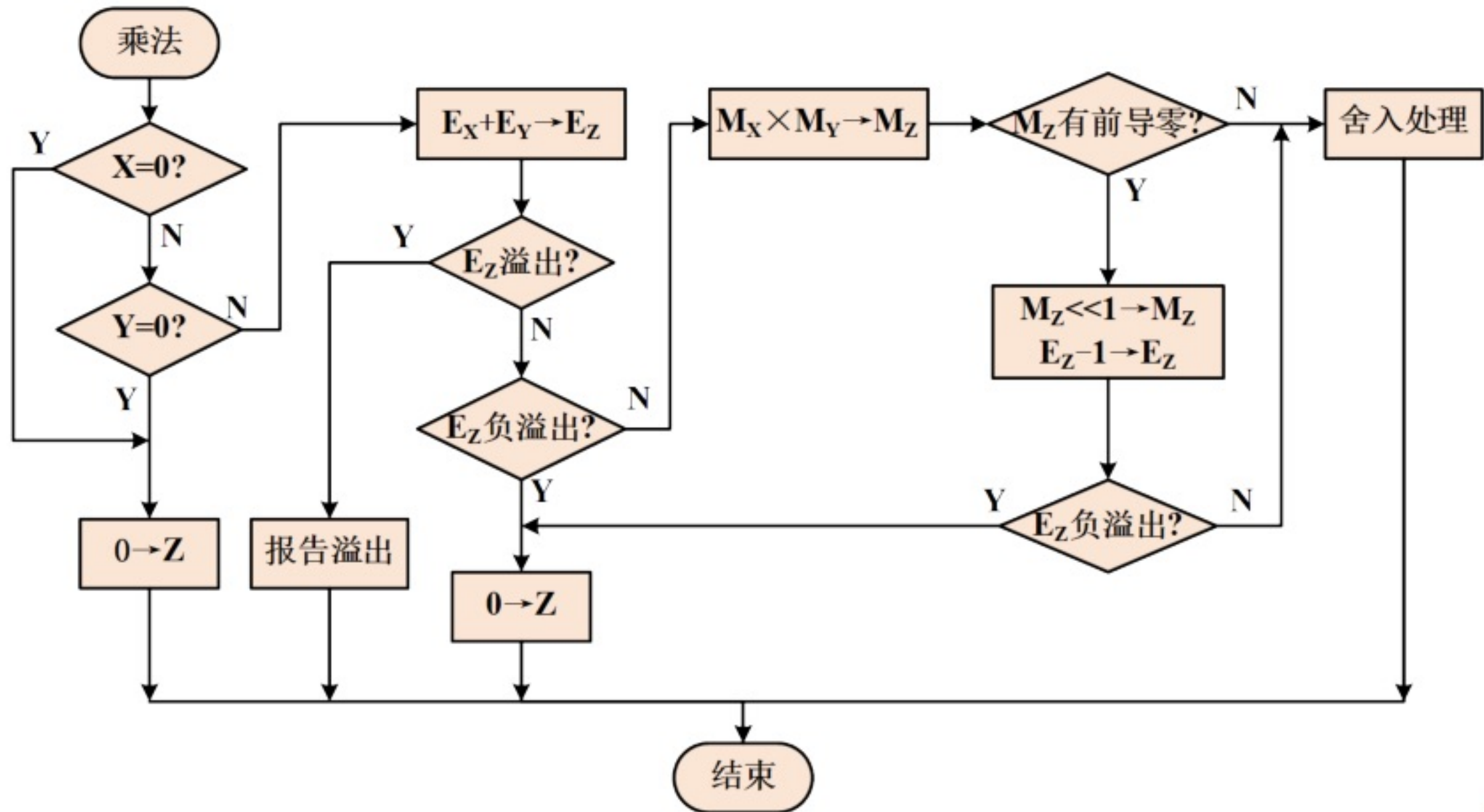
❖ (3) 尾数相乘

❖ (4) 结果规格化: 可能需要左规1位

❖ (5) 舍入处理: 尾数相乘的结果长度是尾数长度的两倍, 必须对低位舍入



浮点数乘法运算流程





浮点乘法运算例题

- ❖ 【例4.15】某浮点数表示格式为：10位浮点数，阶码4位（含阶符），用移码表示，尾数6位（含数符），用补码表示，阶码在前，尾数（含数符）在后，已知X和Y如下，求 $Z=X \cdot Y$ 。要求阶码用移码计算，尾数用补码Booth算法计算，采用0舍1入法舍入。

$$X = (-0.11001) \times 2^{011}$$

$$Y = 0.10011 \times 2^{-001}$$

- ❖ 按照格式首先写出X和Y的规格化浮点数：
- ❖ $[X]_{\text{浮}} = 1, 011 \quad 1.00111$
- ❖ $[Y]_{\text{浮}} = 0, 111 \quad 0.10011$

浮点乘法运算例题

1. 阶码相加

- 无溢出, $[E_Z]_{移} = 1, 010$

$$\begin{array}{r} [E_X]_{移} \quad 01, 011 \\ + [E_Y]_{补} \quad 11, 111 \\ \hline [E_X + E_Y]_{移} \quad 01, 010 \end{array}$$

2. 尾数相乘

- 采用补码Booth算法计算 $[M_X \cdot M_Y]_{补}$
- $[M_X]_{补} = 11.00111$ $[M_Y]_{补} = 0.10011$
- $[-M_X]_{补} = 00.11001$

$$[M_Z]_{补} = 1.10001 \ 00101$$

3. 结果规格化: $[M_Z]_{补}$ 有前导零, 左规一次

- M_Z 左移1位得: $[M_Z]_{补} = 1.00010 \ 01010$
- E_Z 减1得: $[E_Z]_{移} = 01, 010 + 11, 111 = 01, 001$

4. 舍入: 0舍1入

- 对尾数 M_Z 进行“舍”, 得 $[Z]_{浮} = 1, 001 \ 1.00010$

部分积	乘数 M_Y	操作说明
0.00000	0.1 0 0 1 <u>1</u> 0	=10, $+[-M_X]_{补}$
+ 0.11001		
0.11001	0.1 0 0 1 1 0	
0.01100	1 0.1 0 0 <u>1</u> 1	右移一位
+ 0.00000		=11, +0
0.01100	1 0.1 0 0 1 1	
0.00110	0 1 0.1 0 <u>0</u> 1	右移一位
+ 1.00111		=01, $+ [M_X]_{补}$
1.01101	0 1 0.1 0 0 1	
1.10110	1 0 1 0.1 <u>0</u> 0	右移一位
+ 0.00000		=00, +0
1.10110	1 0 1 0.1 0 0	
1.11011	0 1 0 1 0.1 <u>0</u>	右移一位
+ 0.11001		=10, $+ [-M_X]_{补}$
0.10100	0 1 0 1 0.1 0	
0.01010	0 0 1 0 1 <u>0</u> 1	右移一位
+ 1.00111		=01, $+ [M_X]_{补}$
1.10001	0 0 1 0 1	





4.6.3 浮点除法运算

❖ 假设两个浮点数X和Y:

$$X = M_X \times 2^{E_X}$$

$$Y = M_Y \times 2^{E_Y}$$

❖ (1) 0操作数检查

- 除数为0: 报告除法出错 / 商无穷大
- 被除数为0: 商为0

❖ (2) 阶码相减: 结果可能溢出:

- 发生正溢出: 则需报告浮点数溢出;
- 发生负溢出: 则将结果置为机器零。

❖ (3) 尾数相除

❖ (4) 结果规格化: 可能需要右规1次

- 当 $|M_X| \geq |M_Y|$ 时, 需要右规1次

❖ (5) 舍入处理

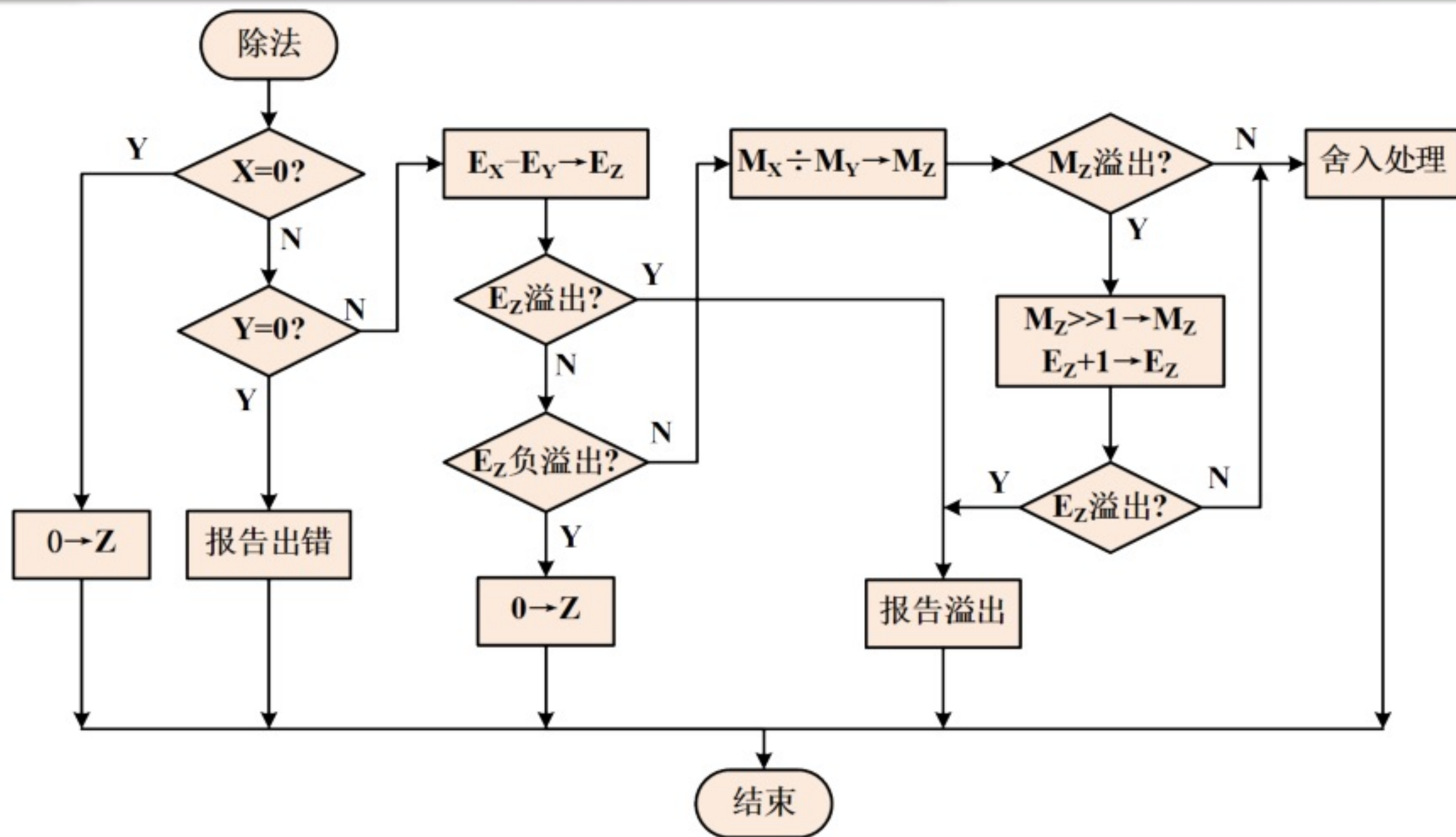
$$Z = X \div Y = (M_X \div M_Y) \times 2^{(E_X - E_Y)}$$

❖ 也可以采用:

- (1) 0操作数检查
- (2) 若 $|M_X| \geq |M_Y|$, M_X 右移1位, 阶码+1
- (3) 阶码相减
- (4) 尾数相除
- 结果规格化
- (5) 舍入处理



浮点数除法运算流程





浮点除法运算例题

❖ 【例4.16】对于例4.15中的浮点数X和Y，求 $Z=X\div Y$ 。要求阶码用移码计算，尾数用原码加减交替除法计算。

- 浮点数格式为：10位浮点数，阶码4位，包含1位阶符，用移码表示，尾数6位，包含1位数字符，用补码表示，阶码在前，尾数（包括数符）在后
- $X = (-0.11001) \times 2^{011}$ $Y = 0.10011 \times 2^{-001}$

❖ 按照浮点数的格式分别写出它们的表示形式为：

- $[X]_{\text{浮}} = 1, 011 \quad 1.00111$ $[Y]_{\text{浮}} = 0, 111 \quad 0.10011$

1. 阶码相减

- $[E_Z]_{\text{移}} = [E_X]_{\text{移}} + [-E_Y]_{\text{补}} = 01, 100$
- 最高符号位为0：无溢出
- $[E_Z]_{\text{移}} = 1, 100$

$[E_X]_{\text{移}}$	01, 011
$+ [-E_Y]_{\text{补}}$	00, 001
$[E_X + E_Y]_{\text{移}}$	01, 100



浮点除法运算例题

2. 尾数相除

- 采用原码加减交替法计算 $|M_x| \div |M_y|$
- $|M_x| = 00.11001$ $|M_y| = 00.10011$
- $[-|M_y|]_{\text{补}} = 11.01101$

$$|M_z| = |M_x| \div |M_y| = 1.01010$$

3. 结果规格化: $|M_x| > |M_y|$, 须右规1位

- $|M_z|$ 右移1位: $|M_z| = 0.10101 \ 0$
- E_z 加1得: $[E_z]_{\text{移}} = 01,101$ 无溢出

4. 舍入: 0舍1入

- “舍”, 得 $|M_z| = 0.10101$
- $[M_z]_{\text{补}} = 1.01011$

最后: $[Z]_{\text{浮}} = 1,101 \ 1.01011$



被除数/余数	商Q	操作说明
00.11001	0 0 0 0 0 0	
+ 11.01101		$+[- M_y]_{\text{补}}$
00.00110	0 0 0 0 0 1	$R_0 > 0$, 上商1
00.01100	0 0 0 0 1.0	左移一位
+ 11.01101		$+[- M_y]_{\text{补}}$
11.11001	0 0 0 0 1.0	$R_1 < 0$, 上商0
11.10010	0 0 0 1.0 0	左移一位
+ 00.10011		$+ M_y $
00.00101	0 0 0 1.0 1	$R_2 > 0$, 上商1
00.01010	0 0 1.0 1 0	左移一位
+ 11.01101		$+[- M_y]_{\text{补}}$
11.10111	0 0 1.0 1 0	$R_3 < 0$, 上商0
11.01110	0 1.0 1 0 0	左移一位
+ 00.10011		$+ M_y $
00.00001	0 1.0 1 0 1	$R_4 > 0$, 上商1
00.00010	1.0 1 0 1 0	左移一位
+ 11.01101		$+[- M_y]_{\text{补}}$
11.01111	1.0 1 0 1 0	$R_5 < 0$, 上商0



4.6.4 浮点运算器结构

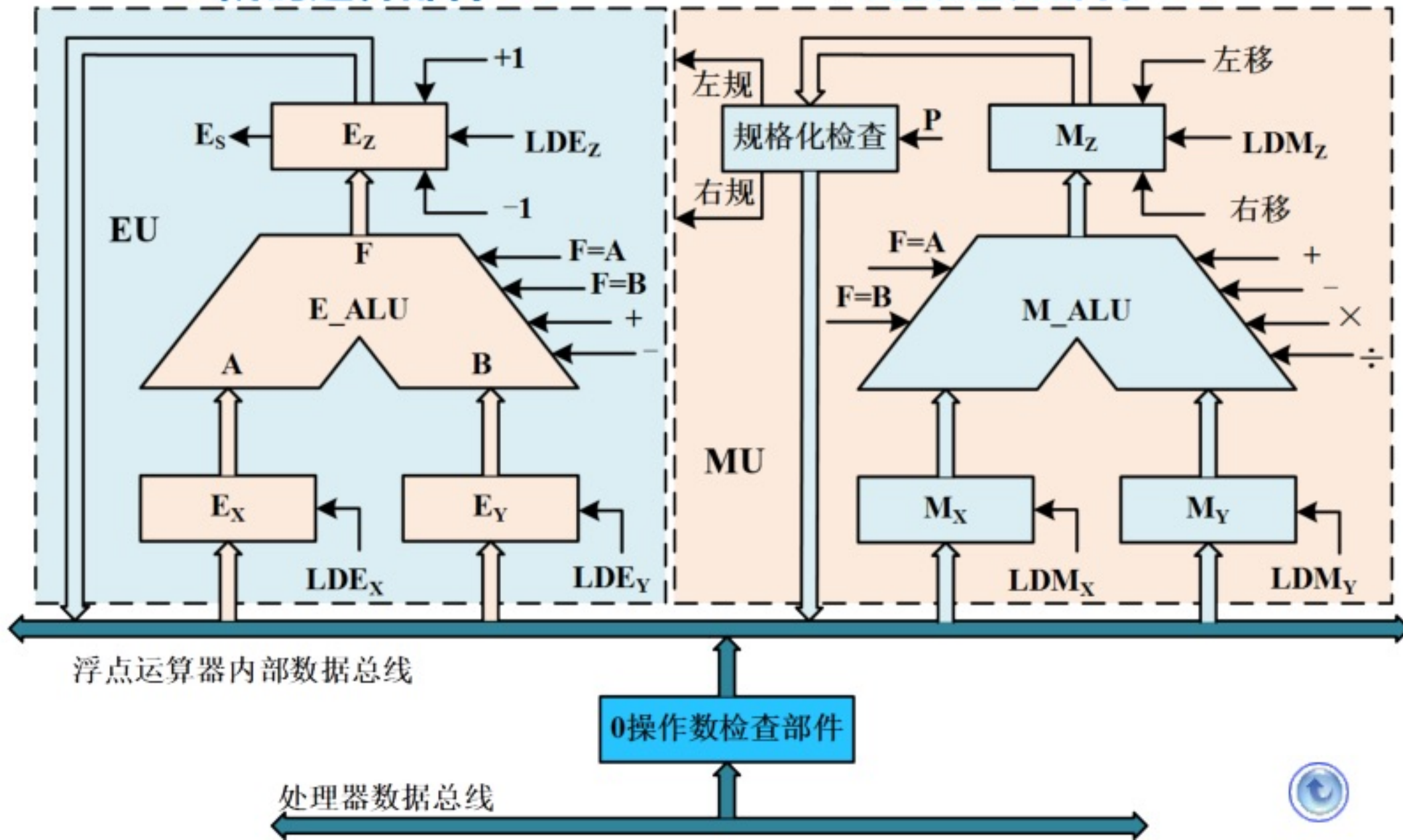
- ❖ 浮点运算复杂程度：远远大于定点运算
- ❖ 是否设置浮点运算器？根据性能要求和需要来确定
- ❖ 在没有浮点运算器的机器中，基于一定的定点运算部件，可以按照上述浮点运算的算法用软件来实现，速度较慢
- ❖ 浮点运算器：由两个松散连接的定点运算部件组成
 - 阶码运算部件：整数运算，具有加减运算和+1、-1的功能
 - 尾数运算部件：小数运算，具有加减乘除四则运算和移位的功能



4.6.4 浮点运算器结构

❖ 阶码运算部件EU

❖ 尾数运算部件MU





4.7 现代计算机的运算器

❖ 4.7.1 超级计算机及微处理器中的运算器

- 1、超级计算机的运算器
- 2、微处理器中的运算器

❖ 4.7.2 Intel 80387浮点运算器

❖ 4.7.3 运算操作的流水线





1、超级计算机的运算器

- ❖ **超级计算机特征：**极大的数据存储容量、极快的数据处理速度
- ❖ 不同时代的超级计算机的性能不能相提并论
- ❖ **1968年：**IBM System 360 Model 91超级计算机
 - 每秒完成**1660万次**加法运算，平均每20ns的时钟周期就执行一条浮点指令
 - 两个同时运行的浮点算术单元，每个单元：两级流水加法器、12b×56b的流水乘法器
 - **浮点数：**32或64位，阶码的底为16
 - **浮点加法：**需要2个时钟周期，浮点加法器由阶码加法器、前/后移位器、阶码调节器、56bit尾数加法器模块构成，尾数加法器采用三级的超前进位设计
 - **浮点乘法：**需要6个时钟周期，使用基4 Booth算法，硬件CSA树型乘法器
 - **浮点除法：**需要18个时钟周期，使用基于牛顿迭代的收敛算法（基于乘法器构造）
- ❖ **1976年：**美国克雷公司推出世界上首台运算速度达**每秒2.5亿次**的超级计算机
- ❖ 早期的高性能计算机，就计算能力而言，远不及现今任何一款笔记本电脑



1、超级计算机的运算器

❖ 超级计算机结构演变：

- 20世纪80年代：通常由小到中等数量的强大向量处理器组成的**向量多处理机**
 - **Cray X-MP/Model 24**巨型机：由两个相同的CPU组成，每个CPU都具有寻址、标量和向量处理部件，其向量处理能力很强
- 20世纪90年代中期：拥有数千个**微处理器**的**集群（并行）计算机**

❖ 中国超级计算机：天河系列、曙光系列、神威系列、深腾系列

❖ 2009年：**天河一号**诞生于天津，是**中国首台超级计算机**

- 由6144个CPU和5120个GPU组成，运算峰值速度达每秒4700万亿次
- 中国自主研发的芯片、操作系统、编译系统、并行程序开发环境与科学计算可视化系统
- 2010年11月，全球超级计算机500强排行榜中，“天河一号”排名第一
- 在各个领域获得了大量成功应用，已经服役超过10年，成为**国产功勋超级计算机**





1、超级计算机的运算器



- ❖ 2013年“天河二号”诞生于广州
- ❖ 天河系列超级计算机：六度夺得全球TOP500冠军，运算速度达 3.386×10^{16} 次/秒
- ❖ “神威·太湖之光”：诞生于无锡，2016年6月20日TOP500强排名第一
 - 浮点运算速度： 9.3×10^{16} 次/秒；采用64位自主申威指令系统，支持256位向量指令集
 - 使用了40960个中国自主研发的申威26010众核处理器构成
 - 异构众核架构：4个运算控制核心和256个运算核心
- ❖ 2022年上半年公布的TOP500榜单中：中国部署的超级计算机数量继续位列全球第一，达到173台，占总体份额的34.6%
 - “神威·太湖之光”和“天河二号”分列榜单第六、第九位
 - 上海交通大学部署的“思源一号”排名第138位，在全球高校部署的超算系统中名列前茅
 - 联想集团交付161台，是目前世界最大的超级计算机制造商





2、微处理器中的运算器

❖ 早期计算机:

- 整数运算ALU: 集成在主处理器中
- 浮点运算部件与浮点寄存器: 集成在浮点协处理器FPU上, 与主处理器连接并协同工作

❖ 现代计算机:

- 整数运算器和FPU: 都集成在微处理器中, 一般各自都有自己独立的寄存器组
- RISC-V: 整数寄存器x0~x31, 供整数指令访问; 浮点寄存器f0~f31, 供浮点指令访问
- 浮点运算器: 设有自己的标志寄存器, 例如RISC-V的fcsr, Intel 80x87的状态寄存器

C语言	RISC-V的汇编语言机器语言
<pre>int x,y,z; z=x+y;</pre>	<pre>//假设x,y,z分别在整数寄存器x18、x19和x20中 add x20,x18,x19</pre>
<pre>float a,b,c; c=a+b;</pre>	<pre>//假设a,b,c分别在浮点寄存器f18、f19和f20中 fadd.s f20,f18,f19</pre>



2、微处理器中的运算器

❖ 现代计算机的浮点运算器：

- 设置了**乘加**浮点指令：完成 $c=c+(a \times b)$ 的运算，将舍入处理从两次减少到一次，提高了浮点运算的精度；RISC-V: **fmadd**指令，4操作数的乘加指令，完成 $d=c+(a \times b)$ 的运算
- 加减乘除四则运算+开平方根、正弦、余弦、对数、指数等**超越函数运算**指令

❖ 现代微处理器：包括**整数运算部件**和**浮点运算部件**，视处理器的性能和支持的指令集的不同，可能**设置多个运算部件**

- **Pentium微处理器**：包含两个整数ALU和一个浮点运算器
- **Pentium MMX版本**：增加了单独的运算部件，支持64位整数向量的处理→MMX指令
- **Pentium III**：可将整数或者浮点数数据打包为128位的向量进行处理→SSE扩展指令，
- **Pentium 4微处理器**：包含3个整数运算部件和2个浮点处理部件→SSE2指令集
- **Core i7多核处理器**：每个核心都包含多个整数运算ALU和浮点运算器，数据封装到256位的向量→从AVX到AVX2的浮点指令集
- **ARM Cortex A8处理器**：包含两个普通的ALU部件、一个地址生成部件和一个乘法部件，还包含了一个NEON部件，支持128位SIMD扩展架构





4.7.2 Intel 80387浮点运算器

- ❖ **FPU: Floating Point Unit**, 浮点处理单元、浮点协处理器、浮点运算器
 - 特殊用途的微处理器: 专门执行浮点数等数据的算术运算和超越函数的运算
- ❖ **Intel 80x87**: 浮点协处理器, 8087、80287、80387、80487SX
 - 只截取和执行协处理器指令 (换码指令ESC), 与Intel 80x86处理器协同工作
 - 80x87可执行68~77条指令
- ❖ **Intel 80x86**: 微处理器, 截取和执行常规指令
 - 将协处理器指令变为换码指令 (ESC)
- ❖ **从80486DX至今**: x86系列微处理器中都包含与80387兼容的至少1个FPU
 - 浮点算术运算速度要高出50~100倍: Pentium 4的FADD指令只要占用0.5ns~3.5ns
- ❖ **主处理器和协处理器之间的数据交换**: 通过专用的I/O地址
 - 80187~80287: 00F8~00FFH
 - 80387~Pentium: 800000F8~800000FFH



4.7.2 Intel 80387浮点运算器

❖ 1、80387的性能

❖ 特点:

- ① 可与配套的CPU芯片异步并行工作
- ② 支持7种数据类型，浮点数的格式与运算完全符合IEEE 754标准
- ③ 具有高性能的80位字长的体系结构，支持80位临时实数
- ④ 具有出色的内部出错管理功能，能检测出6种错误
- ⑤ 可在80386/80486微机系统的实模式和保护模式两种工作模式下运行
- ⑥ 扩展了80386/80486的指令系统，提供超越函数指令



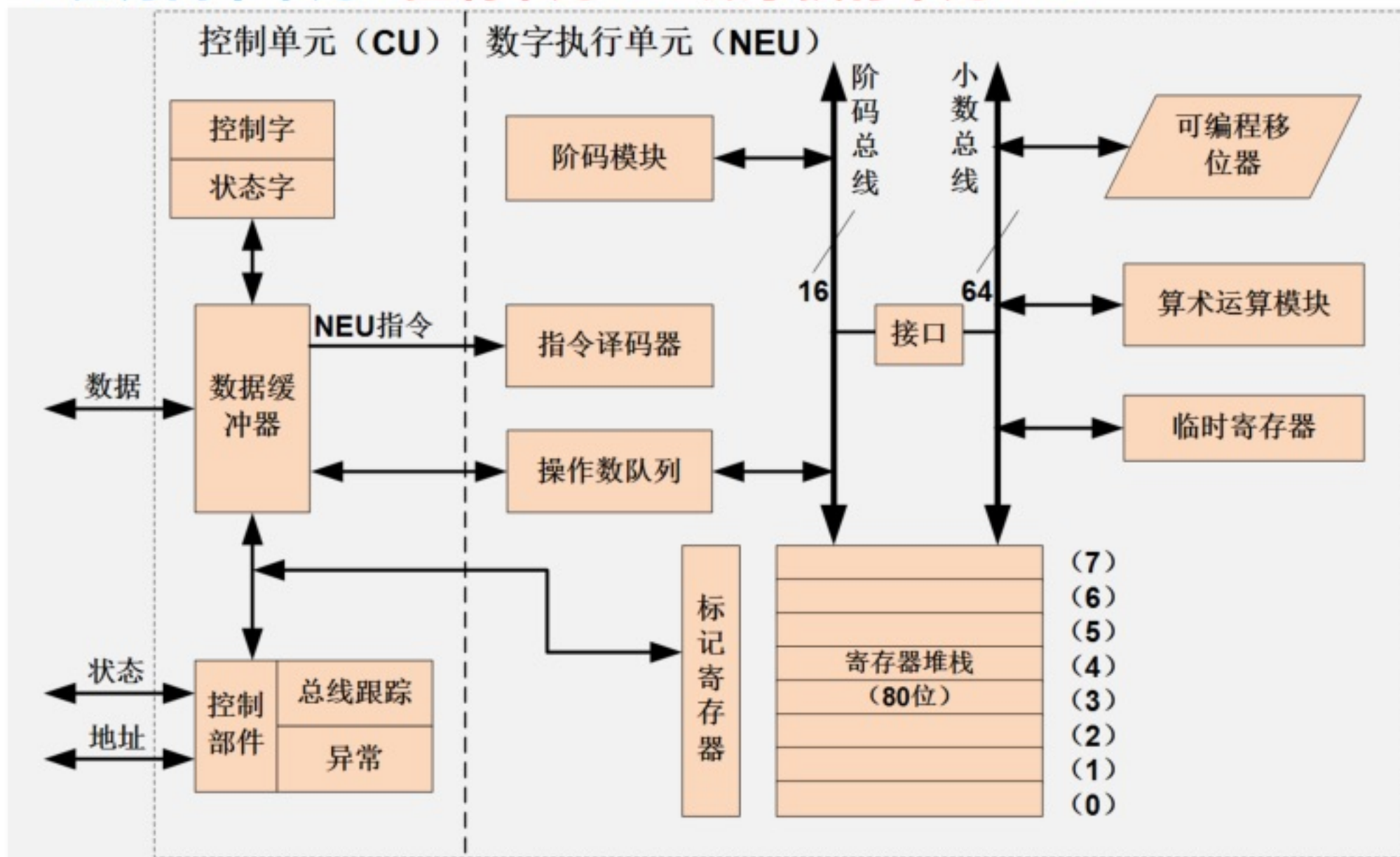
80387支持的7种数据类型及其格式

❖ S—符号位；3种浮点数均符合IEEE754标准

数据类型	数据格式	说明
单字整数（16位整数）	S 15位	二进制补码
短整数（32位整数）	S 31位	二进制补码
长整数（64位整数）	S 63位	二进制补码
短实数（32位浮点数）	S 阶码（8位） 尾数（23位）	单精度浮点数
长实数（64位浮点数）	S 阶码（11位） 尾数（52位）	双精度浮点数
临时实数（80位浮点数）	S 阶码（15位） 尾数（64位）	临时浮点数
十进制数串（80位）	S （7位） $d_{17} d_{16} \dots d_1 d_0$	18位BCD码

2、80387的结构

❖ 分为两个单元：控制单元CU+数字执行单元NEU



❖ **控制单元CU:**
将协处理器连接到微处理器系统总线上, 获取协处理器指令

❖ **数字执行单元NEU:**
执行所有协处理器指令



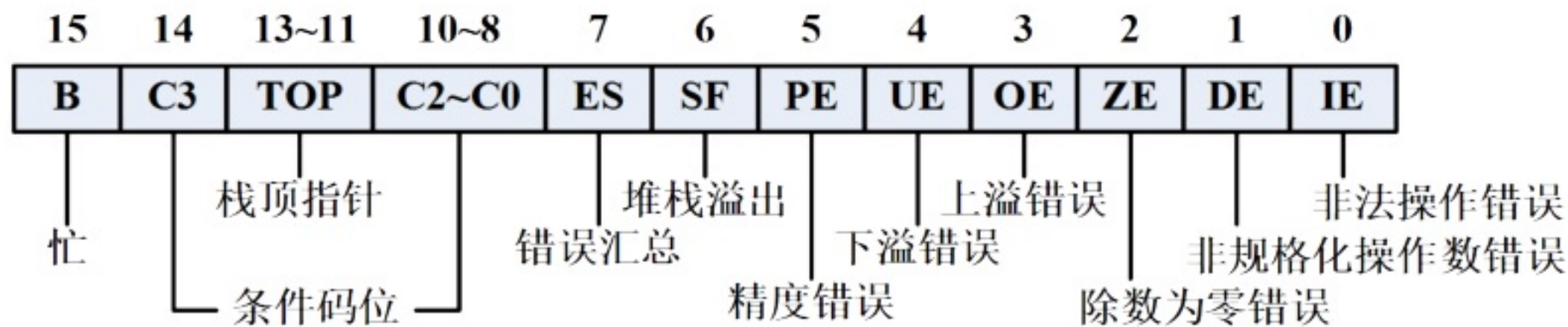
NEU的构成

❖ (1) 寄存器堆栈

- 8个80位寄存器构成“先进后出”堆栈，用于存储操作数和结果(80位的扩展精度浮点数)
- 8个寄存器编号为0~7，栈顶寄存器称为ST，它的编号由状态寄存器的TOP字段指出
- 用ST(i) (i=0~7) 访问相对于栈顶寄存器偏移量为i的寄存器
- 可通过80387的传送指令存取寄存器堆栈的数据

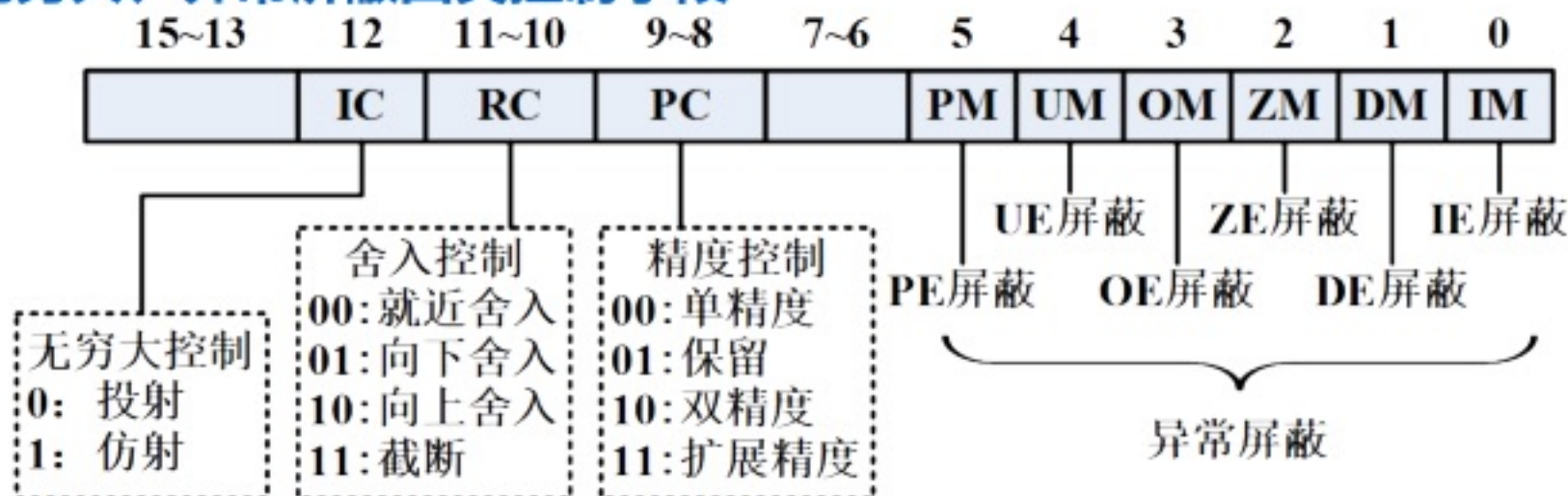
❖ (2) 状态寄存器

- 状态寄存器反映协处理器所有指令的运行情况
- FSTSW AX指令**: 状态寄存器→AX



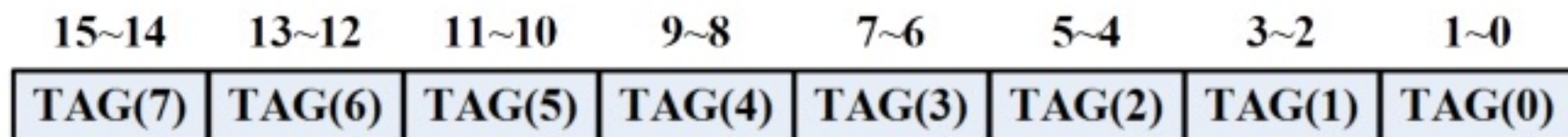
❖ (3) 控制寄存器

- 包括精度、舍入、无穷大、异常屏蔽四类控制字段



❖ (4) 标记寄存器

- 特征寄存器：表明堆栈中每个寄存器内容的状态特征
- 2位的TAG(i)：表示堆栈寄存器ST (i) 的4种状态



TAG值

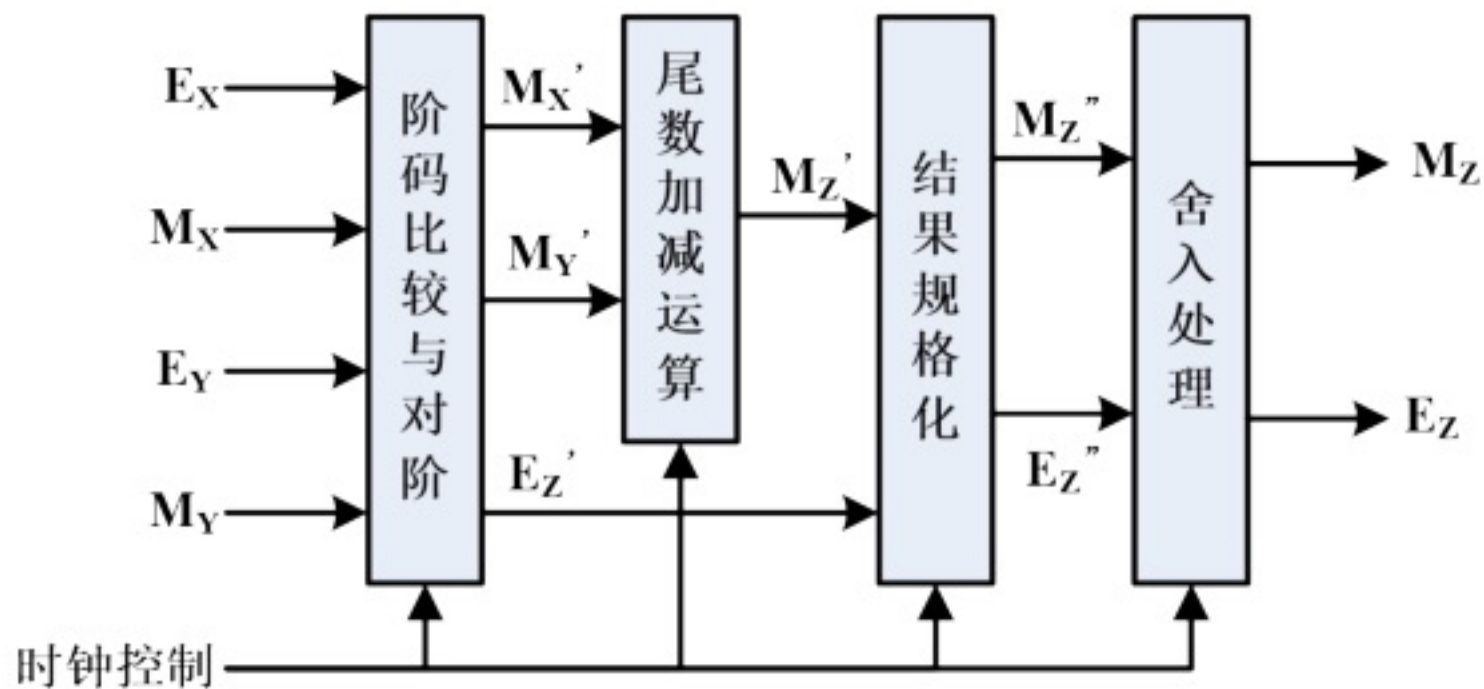
00	合法
01	零
10	非法或无穷大
11	空





4.7.3 运算操作的流水线

- ❖ **流水线原理**: 把输入的任务分割为一系列的子任务, 将任务连续不断地输入流水线, 使**各子任务**能在流水线的各个阶段**并发地**执行。
- ❖ 计算机中流水处理**通过时间并行性**大大改善了计算机系统性能
- ❖ 处理器中: **在指令级采用流水线+在运算操作上采用流水线处理**



❖ 浮点数加减法运算流水线:

- 划分为4个步骤
- 每个步骤作为一个子任务
- 由专用部件实现各自的任務
- 处理结果送入下一步骤功能部件
- 同一时刻, 可以同时处理4组浮点数加减的运算请求



4.7.3 运算操作的流水线

❖ Pentium浮点流水线

- **流水线：8个操作步骤**——预取PF、首次译码D1、二次译码D2、取操作数EX、首次执行X1、二次执行X2、浮点寄存器写入WF和出错报告ER
- **浮点部件：7个部件组成**——浮点接口、寄存器组及控制部件FIRC、浮点指数功能部件FEXP、浮点乘法FMUL、浮点加法FADD、浮点除法FDIV以及舍入处理部件PFRND
- 执行速度是80486的10倍多，每个时钟执行一条浮点指令

❖ Pentium 4微处理器：指令流水线达到了20级，其中运算相关的阶段有：

- 两个RF阶段(读整数寄存器和浮点数寄存器)、执行Ex(整数和浮点运算)、Flgs (计算标志位)

❖ Intel Core i7处理器：流水线深度14级

- 指令译码产生的微操作分发到6个ALU等功能部件上执行，执行结果存入保留站和寄存器

❖ ARM Cortex A8处理器：整数指令流水线有13级

- **整数运算：**E0~E5共6个阶段(取寄存器-桶形移位-ALU运算-饱和算术运算-转移控制-写回)
- **NEON浮点处理器：**10级的流水线，运算和处理有2-6级不等的7种流水级





本章小结

- ❖ **运算器主要功能：**算术运算和逻辑运算
- ❖ **运算器逻辑结构和功能：**取决于指令系统、系统结构、数据编码和运算方法等
- ❖ **高级语言程序中的运算操作符：**经过编译器翻译为**机器语言**在硬件上执行
- ❖ **计算机硬件的运算操作：**分为算术运算、按位运算、移位运算和逻辑运算4类
- ❖ **移位运算：**分为逻辑移位、算术移位和循环移位3类
- ❖ **定点机器数的加减法运算：**通过**补码**来实现
 - 和的补码等于补码的和，差的补码等于被减数的补码与减数相反数的补码之和
 - 补码符号位和数值位一起参加加法运算，将减法转化为加法来运算，方便了硬件设计
- ❖ **定点机器数的乘法运算：**可以采用**原码和补码**来实现
 - **串行乘法算法：**原码一位乘法、补码Booth乘法等
 - **乘法器件：**可以基于并行加法器和右移移位器，采用串行乘法算法构造乘法器，也可以基于进位保留加法器CSA构造高速的阵列乘法器



本章小结

❖ 定点机器数的除法运算：可以采用原码和补码来实现

- 原码除法：恢复余数算法、不恢复余数（加减交替）算法
- 除法硬件实现：可以基于并行加法器和左移移位器，采用串行除法算法构造除法器，也可以基于可控加减运算单元CAS构造阵列除法器，以提高除法运算的速度

❖ 定点运算器构成：核心部件是算术逻辑运算单元ALU

- 部件：算术逻辑部件ALU、暂存器、通用寄存器堆、标志寄存器、内部总线和其他可选电路
- 连接方式：通过片内总线进行连接、部件之间直接连接
- 多端口寄存器堆：能同时执行多个寄存器的读和写，用于多总线型运算器和直接连接运算器
- 标志寄存器：描述运算结果状态的寄存器，用做条件分支指令判别依据
 - 结果为零标志ZF、进位/借位标志CF、溢出标志OF、符号标志SF、奇偶标志PF等



本章小结

❖ 浮点数加减运算

- 步骤：0操作数检查、对阶、尾数加减法、结果规格化、舍入处理

❖ 浮点数乘法运算

- 步骤：0操作数检查、阶码相加、尾数相乘、结果规格化、舍入处理

❖ 浮点数除法运算

- 步骤：0操作数检查、阶码相减、尾数相除、结果规格化、舍入处理

❖ 浮点运算器：由阶码运算和尾数运算两个定点运算部件构成

❖ 现代计算机的运算器：整数运算器和浮点运算器，且大多采用流水线技术实现，以提高运算性能

❖ 浮点数的精度：始终是一个不能忽视的重要问题

❖ 本章重点：定点数和浮点数的运算方法





The End !